TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA ĐIỆN – ĐIỆN TỬ

PHAN THÀNH ĐẠT

THIẾT KẾ VÀ PHÁT TRIỂN MŨ BẢO HIỂM IOT HỖ TRỢ GIÁM SÁT AN TOÀN GIAO THÔNG VÀ CẢNH BÁO NGUY CƠ TAI NẠN

ĐỒ ÁN TỔNG HỢP
KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

TP. HỒ CHÍ MINH, NĂM 2025

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA ĐIỆN – ĐIỆN TỬ

PHAN THÀNH ĐẠT - 42100168

THIẾT KẾ VÀ PHÁT TRIỂN MŨ BẢO HIỂM IOT HỖ TRỢ GIÁM SÁT AN TOÀN GIAO THÔNG VÀ CẢNH BÁO NGUY CƠ TAI NẠN

ĐỒ ÁN TỔNG HỢP
KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

Người hướng dẫn
TS. Huỳnh Thanh Thiện

TP. HỒ CHÍ MINH, NĂM 2025

LỜI CẢM ƠN

Tôi xin chân thành cảm ơn Trường Đại học Tôn Đức Thắng, Khoa Điện - Điện tử đã tạo điều kiện thuận lợi, cung cấp môi trường học tập chất lượng, giúp tôi tích lũy kiến thức và phát triển bản thân trong suốt quá trình học tập. Cảm ơn các Thầy Cô trong Khoa đã luôn tận tâm truyền đạt những kiến thức quý báu, đóng góp to lớn vào sự trưởng thành của tôi.

Tôi đặc biệt xin gửi lời cảm ơn sâu sắc đến Thầy TS. Huỳnh Thanh Thiện, Giảng viên hướng dẫn đồ án tổng hợp của tôi. Thầy đã luôn tận tình hỗ trợ, chỉ dạy và cung cấp những định hướng chính xác giúp tôi hoàn thành tốt đồ án này. Sự quan tâm và hướng dẫn của Thầy là nguồn động lực lớn lao trong suốt quá trình nghiên cứu và thực hiện đồ án.

Tôi cũng xin gửi lời cảm ơn đến các thành viên trong Hội đồng bảo vệ đồ án, những người đã dành thời gian quý báu để lắng nghe, đánh giá và đưa ra những nhận xét bổ ích giúp tôi hoàn thiện đồ án tốt nghiệp của mình.

Cuối cùng, tôi xin chân thành cảm ơn gia đình, bạn bè và những người thân yêu đã luôn đồng hành, động viên và hỗ trợ tôi vượt qua mọi khó khăn trong suốt thời gian học tập. Sự hy sinh và tình cảm của họ là yếu tố không thể thiếu giúp tôi hoàn thành đồ án này.

TP. Hồ Chí Minh, ngày 09 tháng 07 năm 2025

Tác giả

Phan Thành Đạt

Công trình được hoàn thành tại Trường Đại học Tôn Đức Thắng

Cán bộ hướng dẫn khoa học: .....

.....

(Ghi rõ học hàm, học vị, họ tên và chữ ký)

Đồ án tổng hợp được bảo vệ tại **Hội đồng đánh giá Đồ án tổng hợp của Trường Đại học Tôn Đức Thắng** vào ngày… /…/……

Xác nhận của Chủ tịch Hội đồng đánh giá Đồ án tốt nghiệp/tổng hợp và Trưởng khoa quản lý chuyên ngành sau khi nhận Đồ án tốt nghiệp/tổng hợp đã được sửa chữa (nếu có).

CHỦ TỊCH HỘI ĐỒNG	**TRƯỞNG KHOA**
……	……

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH
TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của **TS. Huỳnh Thanh Thiện.** Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Đồ án tổng hợp còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Đồ án tổng hợp của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện.

TP. Hồ Chí Minh, ngày 09 tháng 07 năm 2025

Tác giả

Phan Thành Đạt

THIẾT KẾ VÀ PHÁT TRIỂN MŨ BẢO HIỂM IOT HỖ TRỢ GIÁM SÁT AN TOÀN GIAO THÔNG VÀ CẢNH BÁO NGUY CƠ TAI NẠN

TÓM TẮT

An toàn giao thông là một yếu tố quan trọng trong việc giảm thiểu tai nạn và bảo vệ người tham gia giao thông. Mặc dù nhiều biện pháp an toàn đã được triển khai, nhưng việc đảm bảo người lái xe tuân thủ các quy định về đội mũ bảo hiểm vẫn còn là một thách thức lớn. Trong bối cảnh đó, mũ bảo hiểm thông minh tích hợp công nghệ IoT và các cảm biến hiện đại hứa hẹn sẽ là một giải pháp đột phá trong việc giám sát an toàn giao thông.

Đề tài này trình bày quy trình thiết kế và phát triển mũ bảo hiểm thông minh, được trang bị các cảm biến để giám sát việc đội mũ của người lái xe và phát hiện các hành vi nguy hiểm trong quá trình di chuyển. Mũ bảo hiểm sử dụng công nghệ học sâu để nhận diện và cảnh báo khi người lái xe không đội mũ bảo hiểm hoặc đội mũ sai cách. Bên cạnh đó, các cảm biến gia tốc và va chạm được tích hợp để phát hiện các tình huống nguy hiểm như tăng tốc đột ngột hoặc va chạm, từ đó cung cấp cảnh báo khẩn cấp ngay lập tức.

Mô hình mũ bảo hiểm này cũng có khả năng kết nối với ứng dụng di động hoặc nền tảng web, cho phép người dùng theo dõi tình trạng an toàn giao thông trong thời gian thực và nhận thông tin liên quan đến các hành vi lái xe và nguy cơ tai nạn. Hệ thống không chỉ giúp nâng cao ý thức tuân thủ các quy định về an toàn giao thông mà còn đóng góp tích cực vào việc giảm thiểu số lượng tai nạn và cải thiện chất lượng cuộc sống cho người tham gia giao thông.

DESIGN AND DEVELOPMENT OF AN IOT HELMET FOR TRAFFIC SAFETY MONITORING AND ACCIDENT RISK WARNING

ABSTRACT

Traffic safety plays a crucial role in reducing accidents and protecting road users. Despite the implementation of various safety measures, ensuring that drivers comply with helmet regulations remains a major challenge. In this context, smart helmets integrated with IoT technology and advanced sensors offer a promising solution for enhancing traffic safety monitoring.

This study presents the design and development process of a smart helmet equipped with sensors to monitor helmet usage and detect risky behaviors while driving. The helmet employs deep learning techniques to identify and issue alerts when the rider is not wearing a helmet or wearing it incorrectly. In addition, integrated accelerometer and collision sensors enable the detection of dangerous situations such as sudden acceleration or impact, thereby providing instant emergency warnings.

The smart helmet prototype can also connect to a mobile application or web platform, allowing users to monitor traffic safety conditions in real time and access information regarding driving behaviors and accident risks. The proposed system not only raises awareness of compliance with traffic safety regulations but also contributes to reducing accident rates and improving the quality of life for road users.

MỤC LỤC

DANH MỤC HÌNH VẼ x

DANH MỤC BẢNG BIỂU xii

DANH MỤC CÁC CHỮ VIẾT TẮT xiii

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 1

1.1 Mục đích thực hiện đề tài 1

1.2 Yêu cầu kỹ thuật 2

1.3 Ý tưởng và phương pháp thực hiện 3

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 5

2.1 Tổng quan về mũ bảo hiểm thông minh và IoT 5

2.2 Ứng dụng học sâu trong nhận diện hình ảnh 5

2.3 Đánh giá mô hình học sâu trong nhận diện ảnh 6

2.4 Đánh giá thuật toán học máy trong phân tích dữ liệu 7

CHƯƠNG 3. THIẾT KẾ VÀ PHÁT TRIỂN MŨ BẢO HIỂM IOT HỖ TRỢ GIÁM SÁT AN TOÀN GIAO THÔNG VÀ CẢNH BÁO NGUY CƠ TAI NẠN 9

3.1 Mô hình hệ thống tổng thể 9

3.1.1 Mục tiêu kỹ thuật và kiến trúc tổng thể 9

3.1.2 Sơ đồ khối hệ thống 10

3.2 Thiết kế phần cứng 12

3.2.1 Khối cảm biến và nhận diện 12

3.2.2 Khối truyền thông và hiển thị 15

3.2.3 Khối thực thi *17*

3.2.4 Khối xử lý dữ liệu điều khiển *18*

3.2.5 Khối nguồn *21*

3.3 Thiết kế phần mềm **22**

3.3.1 Lưu đồ giải thuật hệ thống *22*

3.3.2 Xây dựng chương trình nhận diện hình ảnh 23

3.3.3 Xây dựng chương trình giám sát hệ thống *26*

3.3.4 Cấu trúc giao tiếp giữa các cảm biến và bộ xử lý *29*

3.3.5 Giao thức truyền thông giữa Raspberry Pi và ESP32 ..... *31*

3.4 Xây dựng giao diện người dùng **33**

3.4.1 Định hướng thiết kế giao diện người dùng *33*

3.4.2 Kiến trúc ứng dụng di động *34*

3.4.3 Các thành phần giao diện chính 36

CHƯƠNG 4. KẾT QUẢ THI CÔNG VÀ THỰC NGHIỆM **39**

4.1 Kết quả thiết kế hệ thống **39**

4.2 Kiểm thử hệ thống và vận hành thực tế **43**

4.3 Đánh giá hiệu suất hệ thống **47**

4.4 Một số hạn chế trong quá trình thi công **50**

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN **52**

5.1 Kết luận **52**

5.2 Hướng phát triển trong tương lai **53**

DANH MỤC TÀI LIỆU THAM KHẢO **54**

A. Tài liệu tham khảo: **54**

Tiếng Việt **54**

Tiếng Anh **54**

PHỤ LỤC **1**

Phụ lục A: Danh sách linh kiện sử dụng **A-1**

A.1 Danh sách linh kiện sử dụng **A-1**

A.2 Thông số kỹ thuật các cảm biến chính **A-2**

Phụ lục B: Mã nguồn phát triển **B-1**

B.1 Mã nguồn hệ thống 1 ..... **B-1**

B.2 Mã nguồn hệ thống 2 ..... **B-15**

DANH MỤC HÌNH VẼ

Hình 3-1 Sơ đồ khối hệ thống.....10

Hình 3-2 Camera với Raspberry Pi.....12

Hình 3-3 Cảm biến lực và bộ chuyển đổi ADC ADS1115 13

Hình 3-4 Cảm biến Hall với Raspberry Pi.....14

Hình 3-5 Cảm biến gia tốc MPU6050.....14

Hình 3-6 Module GSM SIM900A và GPS NEO6M.....16

Hình 3-7 Màn hình LED LCD I2C.....16

Hình 3-8 Buzzer và module relay 5VDC.....17

Hình 3-9 Raspberry Pi Zero 2W.....18

Hình 3-10 Vi điều khiển ESP32.....19

Hình 3-11 Lưu đồ giải thuật hệ thống 23

Hình 3-12 Learning Curve của quá trình huấn luyện mô hình MobileNet SSD 24

Hình 3-13 Kết quả nhận diện của mô hình 25

Hình 3-14 Sơ đồ luồng xử lý của hệ thống 27

Hình 3-15 Biểu đồ dao động của mũ bảo hiểm.....28

Hình 3-16 Kiến trúc giao tiếp của hệ thống 32

Hình 3-17 Nền tảng React Native 35

Hình 3-18 Giao diện Dashboard và Menu 36

Hình 3-19 Giao diện Alert History và Helmet Status 37

Hình 3-20 Giao diện Location Map và Settings 38

Hình 4-1 Linh kiện sử dụng ở hệ thống 1 39

Hình 4-2 Linh kiện sử dụng ở hệ thống 2 40

Hình 4-3 Mô hình 3D của hộp đựng linh kiện.....41

Hình 4-4 Mô hình nón bảo hiểm hoàn thiện.....42

Hình 4-5 Mô hình giả lập hệ thống trên phương tiện.....42

Hình 4-6 Giai đoạn chạy thử hệ thống 44

Hình 4-7 Kiểm thử chức năng giám sát 45

Hình 4-8 Kiểm thử chức năng giám sát bất thường 46

Hình 4-9 Hiệu suất hoạt động của CPU&RAM của Raspberry Pi Zero 2W 49

Hình 4-10 Biểu đồ sử dụng CPU và RAM của Raspberry Pi Zero 2W 50

DANH MỤC BẢNG BIỂU

Bảng 3-1 Công suất tiêu ước tính của hệ thống.....22

Bảng 3-2 Tóm tắt giao thức giao tiếp 29

Bảng 4-1 Hiệu suất quan sát được của hệ thống 47

Bảng A.1 Danh sách linh kiện sử dụng trong mô hình 1

Bảng A.2 Thông số kỹ thuật chính của các linh kiện sử dụng trong hệ thống....2

Bảng A.3 Thông số kỹ thuật ESP32 DevKit V1 3

Bảng A.4 Thông số kỹ thuật Raspberry Pi Zero 2W 4

DANH MỤC CÁC CHỮ VIẾT TẮT

ADC	Analog to Digital Converter
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CPU	Central Processing Unit
ESP32	Embedded Serial Processor 32-bit
FP	False Positive
FN	False Negative
FPS	Frames Per Second
FSR	Force Sensitive Resistor
GPS	Global Positioning System
GSM	Global System for Mobile Communications
HTTP	HyperText Transfer Protocol
I2C	Inter-Integrated Circuit
IoT	Internet of Things
LCD	Liquid Crystal Display
LED	Light Emitting Diode
MQTT	Message Queuing Telemetry Transport
OTA	Over-The-Air
RAM	Random Access Memory
REST	Representational State Transfer
SSD	Single Shot Multibox Detector
SMS	Short Message Service
TP	True Positive
TN	True Negative
UART	Universal Asynchronous Receiver Transmitter
Wi-Fi	Wireless Fideli

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Mục đích thực hiện đề tài

Mục đích của việc thiết kế và phát triển mũ bảo hiểm IoT hỗ trợ giám sát an toàn giao thông và cảnh báo nguy cơ tai nạn là xây dựng một hệ thống mũ bảo hiểm thông minh, tích hợp công nghệ Internet of Things (IoT), nhằm nâng cao an toàn giao thông và giảm thiểu nguy cơ tai nạn cho người tham gia giao thông. Tai nạn giao thông, đặc biệt là tai nạn liên quan đến người lái xe mô tô, vẫn là vấn đề nghiêm trọng và phổ biến, với nhiều vụ tai nạn do người lái xe không đội mũ bảo hiểm hoặc đội sai cách.

Đề tài này nhằm phát triển một hệ thống mũ bảo hiểm thông minh, tích hợp các cảm biến và thuật toán học sâu để giám sát việc đội mũ của người lái xe. Hệ thống này sẽ nhận diện người không đội mũ bảo hiểm và cảnh báo khi mũ đội không đúng cách, góp phần nâng cao nhận thức của người lái xe và thúc đẩy việc tuân thủ các quy định về an toàn giao thông. Đồng thời, mũ bảo hiểm còn được trang bị cảm biến gia tốc và cảm biến va chạm để phát hiện những hành vi nguy hiểm như tăng tốc đột ngột, giảm tốc nhanh hoặc va chạm. Khi xảy ra tai nạn hoặc tình huống nguy hiểm, hệ thống sẽ tự động gửi thông báo khẩn cấp đến người thân hoặc cơ quan chức năng đã được thiết lập trước đó, đồng thời cung cấp thông tin định vị để hỗ trợ nhanh chóng trong việc cứu hộ.

Mục tiêu của đề tài không chỉ là phát triển một giải pháp công nghệ để nâng cao an toàn giao thông mà còn nhằm thay đổi nhận thức cộng đồng về tầm quan trọng của việc tuân thủ các quy định an toàn. Hệ thống mũ bảo hiểm thông minh này sẽ giúp người lái xe không chỉ bảo vệ bản thân mà còn góp phần giảm thiểu tai nạn giao thông, bảo vệ tính mạng và sức khỏe cho cộng đồng. Thêm vào đó, qua việc ứng dụng công nghệ IoT và các cảm biến thông minh, hệ thống này sẽ mở ra những cơ hội phát triển thêm các ứng dụng liên quan trong tương lai.

1.2 Yêu cầu kỹ thuật

Đề tài yêu cầu thiết kế và phát triển mô hình mũ bảo hiểm IoT hỗ trợ giám sát an toàn giao thông và cảnh báo nguy cơ tai nạn như một công cụ điều chỉnh hành vi của người tham gia giao thông, nhằm đảm bảo sự tuân thủ các quy định an toàn và giảm thiểu rủi ro tai nạn. Cụ thể, người thực hiện cần phát triển mũ bảo hiểm với các yêu cầu sau:

Mô hình mũ bảo hiểm IoT cần được thiết kế với khả năng nhận diện hình ảnh người lái xe không đội mũ bảo hiểm thông qua ứng dụng học sâu. Hệ thống phải dùng mô hình học sâu (deep learning) để phân tích và nhận diện hình ảnh từ camera gắn trên mũ, nhằm xác định người lái xe có đội mũ bảo hiểm hay không. Cụ thể, khi người lái xe đội mũ bảo hiểm, hệ thống sẽ nhận diện hình ảnh khuôn mặt của người lái xe xuất hiện trong vùng quan sát của camera gắn trên mũ bảo hiểm với độ chính xác nhận diện khuôn mặt từ 50% trở lên và phải đảm bảo tính chính xác trong các tính huống người đội mũ trang bị kính mắt hoặc khẩu trang.

Mũ bảo hiểm cần phải tích hợp cảm biến lực hoặc các cảm biến khác để phát hiện người đội mũ đúng cách. Cảm biến này đo lường lực tác động lên mũ khi người lái xe đội hoặc xác nhận xem người sử dụng mũ đã gài quai an toàn hay chưa, từ đó xác định xem mũ có được đội đúng cách và đúng vị trí hay không. Nếu mũ được đội chắc chắn và đúng cách, lực phân bổ sẽ ổn định trên các điểm tiếp xúc giữa mũ và đầu người lái. Ngược lại, nếu mũ bị lệch hoặc không ôm sát đầu, hệ thống sẽ không cho phép xe khởi động, đảm bảo an toàn cho người lái. Đồng thời, mũ bảo hiểm phải được trang bị khả năng phát âm thanh thông báo khi người lái xe không đội mũ hoặc đội mũ chưa đúng cách. Cụ thể, khi hệ thống nhận biết trạng thái đội mũ của người lái xe chưa phù hợp, như mũ bị lệch hoặc không gài quai đúng cách, hệ thống sẽ phát ra âm thanh cảnh báo để người lái xe điều chỉnh lại mũ bảo hiểm trước khi có thể khởi động xe, giúp đảm bảo an toàn và nâng cao ý thức chấp hành các quy định an toàn giao thông.

Mũ bảo hiểm cần được trang bị cảm biến gia tốc và các cảm biến chuyển động khác để nhận diện các hành vi nguy hiểm của người lái xe, như tăng tốc đột ngột. Khi

hệ thống phát hiện người lái xe tăng tốc quá nhanh, ví dụ như gia tốc vượt quá $3m/s^2$ hoặc có thay đổi vận tốc bất thường, hệ thống sẽ phát ra cảnh báo âm thanh trực tiếp trên mũ bảo hiểm hoặc gửi thông báo qua ứng dụng di động. Đồng thời, hệ thống khi thiết kết sẽ phải lưu trữ các dữ liệu về hành vi lái xe nguy hiểm, bao gồm thông tin về các lần tăng tốc đột ngột và các tình huống vi phạm, để giúp cải thiện nhận thức và hành vi lái xe trong tương lai.

Mũ bảo hiểm và xe sẽ cần phải được trang bị cảm biến va chạm để nhận biết các tình huống va chạm trong quá trình di chuyển. Khi hệ thống phát hiện va chạm, cảm biến trên mũ bảo hiểm và xe sẽ xác định vị trí và mức độ nghiêm trọng của vụ va chạm thông qua sự thay đổi về lực tác động. Ngay lập tức, hệ thống sẽ phát ra cảnh báo khẩn cấp và tự động gọi điện thoại cho người thân của người lái xe để thông báo về tình huống tai nạn. Đồng thời, mô hình mũ bảo hiểm này sẽ cần sử dụng GPS với độ chính xác vị trí ±2.5 mét để xác định chính xác vị trí của người lái xe và gửi thông tin này đến người thân hoặc các cơ quan cứu hộ.

Hệ thống mũ bảo hiểm thông minh này sẽ phải phát triển để kết nối với ứng dụng di động hoặc website thông qua một web server để ghi lại và theo dõi các thông tin liên quan đến người lái xe. Mỗi khi người lái xe sử dụng mũ bảo hiểm, hệ thống sẽ tự động ghi nhận và lưu trữ lịch sử trạng thái đội mũ, bao gồm thông tin về việc mũ có được đội đúng cách hay không, thời gian sử dụng và các hành vi nguy hiểm. Ngoài ra, trạng thái an toàn của người lái xe cũng phải được theo dõi liên tục, với thông tin về các cảnh báo, va chạm hoặc tình huống nguy hiểm, giúp tạo ra một hồ sơ an toàn chi tiết. Tất cả dữ liệu này phải được lưu trữ trên thông qua ứng dụng di động hoặc web server, cho phép người dùng truy cập dễ dàng thông qua ứng dụng di động hoặc website để xem lại lịch sử, phân tích các hành vi lái xe và đánh giá mức độ an toàn của mỗi chuyến đi.

1.3 Ý tưởng và phương pháp thực hiện

Dự án được xây dựng với mục tiêu phát triển một mô hình mũ bảo hiểm thông minh có khả năng giám sát an toàn giao thông và hỗ trợ xử lý tình huống khẩn cấp,

hướng đến việc nâng cao ý thức và hành vi của người điều khiển phương tiện. Ý tưởng cốt lõi là thiết kế một thiết bị tích hợp các công nghệ hiện đại như xử lý ảnh, cảm biến và truyền thông không dây trong một hệ thống hoàn chỉnh. Hệ thống sử dụng camera kết hợp với thuật toán học sâu MobileNet SSD (Single Shot Multibox Detector) để nhận diện người đội mũ bảo hiểm một cách chính xác ngay cả trong môi trường thiếu ánh sáng. Ngoài ra, các cảm biến lực như FSR402 và RFP602 cùng với cảm biến từ trường Hall A3144 được tích hợp để kiểm tra việc đội mũ đúng cách và xác nhận người dùng đã gài quai an toàn. Sau khi xác nhận đầy đủ các điều kiện an toàn, Raspberry Pi sẽ đóng vai trò xử lý trung tâm, gửi tín hiệu điều khiển không dây đến vi điều khiển ESP32 để kích hoạt relay, cho phép xe được khởi động.

Không dừng lại ở chức năng nhận diện và cho phép khởi động xe, hệ thống còn được trang bị các cảm biến như MPU6050 (gia tốc kế và con quay hồi chuyển), SW420 (cảm biến rung) và SW520 (cảm biến nghiêng) nhằm nhận biết các hành vi nguy hiểm như tăng tốc đột ngột, rung lắc mạnh hoặc va chạm bất thường. Mỗi khi xảy ra tình huống có dấu hiệu tai nạn, hệ thống sẽ kích hoạt cơ chế cảnh báo thông minh: sử dụng module GPS để xác định vị trí người điều khiển và module SIM để gửi tin nhắn hoặc tự động gọi điện thoại đến người thân hoặc cơ quan chức năng cứu hộ. Trong trường hợp tin nhắn hoặc cuộc gọi không thể thực hiện do mất sóng hoặc sự cố mạng, hệ thống còn được tích hợp chức năng gửi email khẩn cấp kèm theo thông tin định vị và thời điểm xảy ra sự cố đến các địa chỉ liên hệ được thiết lập trước nhằm đảm bảo cảnh báo luôn được gửi đi kịp thời. Bên cạnh đó, toàn bộ dữ liệu về hành vi lái xe, trạng thái an toàn và lịch sử sử dụng thiết bị sẽ được lưu trữ một cách có hệ thống, cho phép người dùng truy cập thông qua ứng dụng di động hoặc website. Hệ thống còn cung cấp các báo cáo phân tích chi tiết về thói quen điều khiển phương tiện, hỗ trợ người dùng điều chỉnh hành vi không an toàn và nâng cao ý thức khi tham gia giao thông. Với khả năng tích hợp linh hoạt, độ chính xác cao và giao diện thân thiện, mô hình mũ bảo hiểm IoT này hứa hẹn sẽ là một giải pháp hữu ích trong việc giảm thiểu rủi ro tai nạn giao thông và bảo vệ tính mạng người điều khiển phương tiện.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về mũ bảo hiểm thông minh và IoT

Mũ bảo hiểm thông minh là một thiết bị bảo hộ được tích hợp thêm các thành phần điện tử và kết nối mạng, nhằm hỗ trợ người dùng nâng cao mức độ an toàn khi tham gia giao thông. Khác với mũ bảo hiểm truyền thống chỉ có chức năng bảo vệ cơ học, mũ thông minh có khả năng giám sát trạng thái đội mũ, phát hiện hành vi nguy hiểm và cảnh báo kịp thời khi xảy ra sự cố. Trong hệ thống được đề xuất, các tính năng chính bao gồm: nhận diện người đội mũ bằng hình ảnh từ camera, phát hiện tình trạng gài quai thông qua cảm biến từ trường, xác định va chạm và chuyển động bất thường nhờ cảm biến gia tốc, cảm biến nghiêng và chuyển động. Ngoài ra, hệ thống còn có thể kết nối với module GPS và SIM để gửi vị trí và cảnh báo khẩn cấp đến người thân trong trường hợp xảy ra tai nạn.

IoT đóng vai trò là nền tảng kết nối giữa các thiết bị cảm biến, vi xử lý và phần mềm. Dữ liệu thu thập được từ các cảm biến và camera được truyền về bộ xử lý trung tâm (Raspberry Pi) để phân tích, xử lý và đưa ra quyết định điều khiển như bật relay khởi động, phát âm thanh cảnh báo hoặc gửi dữ liệu lên ứng dụng di động. Sự kết hợp giữa mũ bảo hiểm thông minh và nền tảng IoT giúp hệ thống hoạt động linh hoạt, đồng thời hỗ trợ lưu trữ, thống kê và giám sát hành vi người điều khiển một cách hiệu quả theo thời gian thực.

2.2 Ứng dụng học sâu trong nhận diện hình ảnh

Học sâu (Deep Learning) là một lĩnh vực trong trí tuệ nhân tạo, mô phỏng cách hoạt động của não bộ con người để xử lý và phân tích dữ liệu đầu vào có cấu trúc phức tạp, đặc biệt là hình ảnh. Trong hệ thống mũ bảo hiểm thông minh, học sâu được sử dụng để nhận diện khuôn mặt người điều khiển phương tiện, qua đó xác định tình trạng đội mũ (có đội, không đội). Điều này được thực hiện thông qua camera tích hợp và một mô hình học sâu đã được huấn luyện trước.

Hai kiến trúc học sâu được sử dụng phổ biến trong bài toán nhận diện hình ảnh là CNN (Convolutional Neural Network) và YOLO (You Only Look Once). CNN hoạt động bằng cách sử dụng các lớp tích chập để tự động trích xuất đặc trưng từ hình ảnh đầu vào. Các lớp chính trong CNN bao gồm: lớp tích chập (convolutional layer), lớp kích hoạt (activation layer), lớp gộp (pooling layer) và lớp kết nối đầy đủ (fully connected layer). Đây là mô hình hiệu quả cho phân loại và phân đoạn ảnh.

Trong khi đó, YOLO là mô hình nhận diện đối tượng theo thời gian thực. Ưu điểm lớn nhất của YOLO là khả năng phát hiện nhiều đối tượng trong một lần xử lý duy nhất, giúp tối ưu tốc độ xử lý. YOLO hoạt động bằng cách chia ảnh đầu vào thành các ô lưới và dự đoán bounding box cùng với class probability cho mỗi ô, từ đó xác định vị trí và nhãn của đối tượng.

Hệ thống sử dụng mô hình MobileNet SSD, kết hợp kiến trúc MobileNet để giảm số tham số với SSD giúp phát hiện nhanh các đối tượng. Mô hình này được triển khai dưới định dạng TensorFlow Lite giúp chạy tốt trên thiết bị nhúng như Raspberry Pi Zero 2W. Ảnh đầu vào sẽ được resize về kích thước 300x300 pixel, sau đó thực hiện suy luận để xác định các bounding box, class ID và confidence score của các đối tượng. Chỉ các đối tượng có độ tin cậy trên 50% mới được giữ lại để phân tích.

2.3 Đánh giá mô hình học sâu trong nhận diện ảnh

Trong hệ thống mũ bảo hiểm thông minh, việc sử dụng mô hình học sâu là yếu tố cốt lõi để nhận diện hình ảnh khuôn mặt người điều khiển và đánh giá trạng thái đội mũ. Do đó, việc đánh giá hiệu quả của mô hình học sâu là cần thiết để đảm bảo tính chính xác và độ tin cậy trong điều kiện thực tế. Các tiêu chí quan trọng thường được sử dụng để đánh giá bao gồm độ chính xác (accuracy), tốc độ xử lý (inference time hoặc FPS – frames per second) và khả năng tổng quát (generalization). Độ chính xác phản ánh tỷ lệ nhận diện đúng các đối tượng trong ảnh đầu vào so với tổng số dự đoán. Tuy nhiên, trong bài toán nhận diện đối tượng cụ thể như phát hiện khuôn mặt hay khẩu trang, kính mắt, các chỉ số như Precision, Recall và đặc biệt là chỉ số mAP (mean Average Precision) được sử dụng để đánh giá chi tiết hơn.

Chỉ số mAP được tính bằng trung bình của các giá trị AP (Average Precision) theo từng lớp đối tượng, được định nghĩa bởi công thức:

$$mAP = \frac{1}{N}\sum_{i=1}^{N} AP_i \qquad (2.1)$$

Trong đó N là số lượng các lớp đối tượng (ví dụ: mũ bảo hiểm, người lái xe, v.v.) và AP_i là Average Precision cho lớp đối tượng thứ *i*. Việc sử dụng mAP giúp mô tả đầy đủ mức độ chính xác trong việc phát hiện và phân loại đối tượng của mô hình học sâu.

Bên cạnh độ chính xác, tốc độ xử lý cũng là yếu tố quan trọng trong hệ thống thời gian thực. Mô hình cần đạt tối thiểu 15–30 FPS để có thể phát hiện và đưa ra cảnh báo kịp thời. Trong hệ thống này, mô hình MobileNet SSD được sử dụng vì có kiến trúc gọn nhẹ, xử lý nhanh và tiêu thụ ít tài nguyên, phù hợp cho các thiết bị nhúng như Raspberry Pi Zero 2W. Cuối cùng, khả năng tổng quát của mô hình cần được kiểm tra để đảm bảo mô hình vẫn hoạt động hiệu quả trên các hình ảnh có điều kiện ánh sáng, góc nhìn hoặc môi trường khác với dữ liệu huấn luyện. Phương pháp kiểm tra chéo (cross-validation) hoặc sử dụng tập kiểm tra độc lập (hold-out test set) thường được áp dụng để đánh giá yếu tố này.

2.4 Đánh giá thuật toán học máy trong phân tích dữ liệu

Thuật toán học máy là công cụ quan trọng trong phân tích dữ liệu, giúp khai thác thông tin, nhận diện mẫu và đưa ra dự đoán từ dữ liệu thô. Hệ thống mũ bảo hiểm thông minh không chỉ phụ thuộc vào khả năng nhận diện hình ảnh, mà còn dựa vào các cảm biến như gia tốc, va chạm, rung động và góc nghiêng để thu thập dữ liệu số, phục vụ cho quá trình phân tích hành vi và đánh giá mức độ an toàn của người điều khiển. Việc đánh giá các thuật toán học máy được ứng dụng trong quá trình này là cần thiết nhằm đảm bảo rằng hệ thống phản hồi đúng và chính xác trong việc phát hiện hành vi nguy hiểm hoặc tình huống bất thường.

Với các mô hình học có giám sát, chỉ số accuracy là một trong những tiêu chí đánh giá phổ biến. Chỉ số này được tính dựa trên bốn thành phần chính: TP (True

Positive – dự đoán đúng dương tính), TN (True Negative – dự đoán đúng âm tính), FP (False Positive – dự đoán sai dương tính), và FN (False Negative – dự đoán sai âm tính). Công thức tính như sau:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{2.2}$$

Chỉ số này cho biết mô hình đã đưa ra bao nhiêu dự đoán đúng so với tổng số dự đoán. Trong một số trường hợp, khi dữ liệu mất cân bằng (ví dụ số trường hợp có va chạm ít hơn nhiều so với không có va chạm), thì các chỉ số Precision, Recall hoặc F1-score sẽ được ưu tiên hơn để đánh giá hiệu quả mô hình.

Mô hình học có giám sát được đánh giá hiệu quả khi có khả năng phân loại đúng phần lớn các tình huống thực tế. Tuy nhiên, trong trường hợp dữ liệu mất cân bằng, chẳng hạn như số lượng mẫu "không có va chạm" lớn hơn nhiều so với "có va chạm" thì chỉ số Accuracy có thể gây hiểu nhầm về hiệu suất. Khi đó, cần kết hợp thêm các chỉ số như Precision và Recall để đánh giá khả năng phát hiện đúng tình huống nguy hiểm và giảm thiểu cảnh báo sai.

Bên cạnh độ chính xác, mô hình cũng cần đảm bảo khả năng tổng quát, tức là khả năng áp dụng hiệu quả trên dữ liệu mới chưa từng gặp. Mô hình tổng quát tốt sẽ giữ được hiệu suất cao trong điều kiện thực tế, nơi dữ liệu có thể thay đổi theo môi trường và hành vi người dùng. Để kiểm tra, các phương pháp như cross-validation hoặc hold-out validation thường được áp dụng nhằm đánh giá độ tin cậy của mô hình trên các bộ dữ liệu kiểm thử độc lập, tránh hiện tượng quá khớp với dữ liệu huấn luyện (overfitting).

CHƯƠNG 3. THIẾT KẾ VÀ PHÁT TRIỂN MŨ BẢO HIỂM IOT HỖ TRỢ GIÁM SÁT AN TOÀN GIAO THÔNG VÀ CẢNH BÁO NGUY CƠ TAI NẠN

3.1 Mô hình hệ thống tổng thể

3.1.1 Mục tiêu kỹ thuật và kiến trúc tổng thể

Hệ thống mũ bảo hiểm thông minh tích hợp công nghệ IoT được thiết kế nhằm đáp ứng các mục tiêu kỹ thuật trọng yếu: đảm bảo khả năng giám sát an toàn cho người điều khiển phương tiện trong thời gian thực; phát hiện nhanh và chính xác các tình huống nguy hiểm; cung cấp cảnh báo kịp thời; đồng thời duy trì hiệu năng ổn định trên nền tảng phần cứng nhúng có tài nguyên hạn chế như Raspberry Pi Zero 2W và vi điều khiển ESP32.

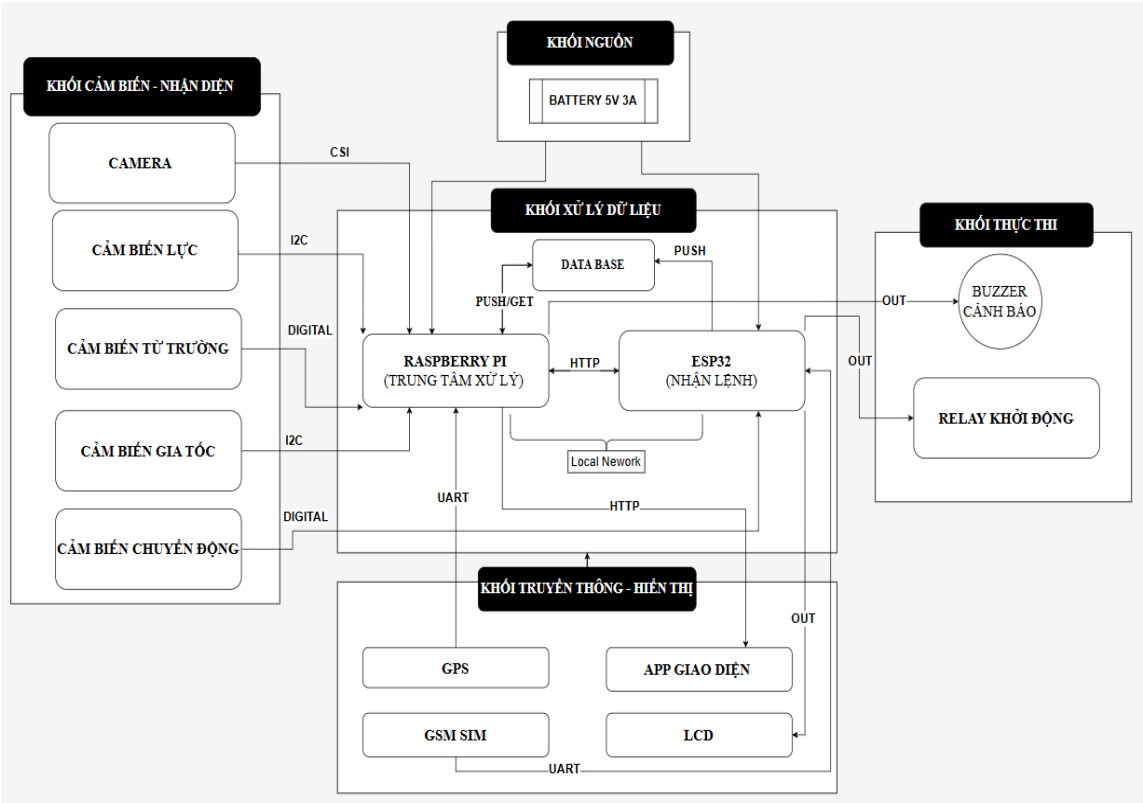
Trước hết, hệ thống phải đảm bảo khả năng nhận diện chính xác trạng thái đội mũ của người lái xe. Thông qua camera tích hợp và thuật toán học sâu MobileNet SSD, hệ thống có thể xác định xem người dùng có đội mũ bảo hiểm hay không, kể cả trong các trường hợp người điều khiển đeo khẩu trang hoặc kính mắt. Bên cạnh đó, các cảm biến lực được bố trí bên trong mũ cho phép phát hiện xem mũ có được đội đúng cách, ôm sát đầu người dùng hay không. Chức năng này giúp ngăn chặn trường hợp người dùng chỉ treo mũ hoặc đội không đúng cách để qua mặt hệ thống.

Song song với đó, hệ thống còn có nhiệm vụ phát hiện các hành vi nguy hiểm trong quá trình vận hành phương tiện. Dữ liệu thu thập từ cảm biến gia tốc MPU6050, cảm biến rung SW420 và cảm biến nghiêng SW520 được phân tích theo thời gian thực nhằm nhận diện các tình huống như tăng tốc đột ngột, va chạm, hoặc té ngã. Khi phát hiện sự kiện bất thường, hệ thống sẽ lập tức kích hoạt cảnh báo bằng loa buzzer, đồng thời gửi thông báo và thông tin vị trí thông qua module SIM900A và GPS NEO-6M đến các số điện thoại khẩn cấp đã được cài đặt trước.

Về mặt kiến trúc, hệ thống được tổ chức thành ba lớp chức năng chính. Lớp đầu tiên là lớp thu thập dữ liệu, bao gồm camera và các cảm biến vật lý (FSR, Hall, MPU6050, SW420, SW520). Lớp thứ hai là lớp xử lý và điều khiển trung tâm, sử dụng Raspberry Pi Zero 2W làm bộ xử lý chính, kết hợp với vi điều khiển ESP32 đóng vai trò chấp hành phần cứng (bật/tắt relay khởi động xe, phát cảnh báo). Lớp thứ ba là lớp truyền thông và phản hồi khẩn cấp, với các module GPS, GSM và giao diện ứng dụng di động cho phép theo dõi và giám sát tình trạng an toàn từ xa.

Hệ thống áp dụng mô hình giao tiếp phân tầng giữa Raspberry Pi và ESP32, sử dụng giao thức HTTP REST API trên mạng Wi-Fi cục bộ. Cách tiếp cận này không chỉ đảm bảo tính tương thích cao giữa các thiết bị mà còn giúp giảm độ trễ truyền thông, tăng khả năng mở rộng và dễ bảo trì về sau. Ngoài ra, các dữ liệu về trạng thái vận hành và lịch sử cảnh báo cũng được đồng bộ hóa với dịch vụ Google Sheets nhằm hỗ trợ lưu trữ lâu dài và phục vụ phân tích hành vi người dùng.

3.1.2 Sơ đồ khối hệ thống



Hình 3-1 Sơ đồ khối hệ thống

Sơ đồ khối tổng thể của hệ thống mũ bảo hiểm thông minh dùng để giám sát và cảnh báo tai nạn được tích hợp IoT trong hệ thống được thể hiện trong Hình 3-1. Hệ thống trên được chia thành năm khối chức năng chính: khối cảm biến – nhận diện, khối xử lý dữ liệu, khối thực thi, khối truyền thông – hiển thị và khối nguồn. Các khối được kết nối với nhau thông qua các chuẩn giao tiếp phổ biến như CSI, I2C, UART, digital GPIO và HTTP nội bộ.

Khối cảm biến – nhận diện bao gồm camera, cảm biến lực (FSR và RFP), cảm biến từ trường (Hall), cảm biến gia tốc (MPU6050) và cảm biến chuyển động (SW420, SW520). Camera kết nối với Raspberry Pi thông qua giao tiếp CSI để xử lý hình ảnh và nhận diện trạng thái đội mũ. Các cảm biến lực và gia tốc giao tiếp qua I2C, trong khi cảm biến từ trường và chuyển động sử dụng digital input. Các tín hiệu từ khối này phản ánh tình trạng đội mũ đúng cách, phát hiện rung động bất thường hoặc va chạm.

Khối xử lý dữ liệu sử dụng Raspberry Pi làm trung tâm xử lý. Dữ liệu cảm biến được thu thập, phân tích và lưu trữ vào cơ sở dữ liệu nội bộ (database). Raspberry Pi cũng đảm nhận việc phân tích ảnh bằng mô hình học sâu để xác định người dùng có đội mũ bảo hiểm hay không. Khi đạt điều kiện an toàn, Raspberry Pi gửi lệnh điều khiển thông qua HTTP nội bộ đến vi điều khiển ESP32, đóng vai trò là đơn vị thực thi tín hiệu phần cứng. Kết nối giữa Raspberry Pi và ESP32 sử dụng mạng nội bộ (local network) trong môi trường tích hợp.

Khối thực thi bao gồm buzzer cảnh báo và relay khởi động xe, được điều khiển trực tiếp bởi ESP32. Khi hệ thống phát hiện hành vi vi phạm như không đội mũ hoặc có va chạm, buzzer sẽ phát tín hiệu cảnh báo và relay sẽ ngắt, ngăn không cho xe khởi động. Mô hình này có nét tương đồng với nghiên cứu của các cảm biến rung và module GSM được tích hợp vào mũ bảo hiểm để phát hiện tai nạn và gửi cảnh báo tự động đến số điện thoại đã định sẵn (S. Sekar và cộng sự, 2022) .

Khối truyền thông – hiển thị bao gồm module GPS, SIM GSM, LCD và giao diện ứng dụng (app). Raspberry Pi và ESP32 sử dụng UART để giao tiếp với GPS và module SIM nhằm gửi tin nhắn khẩn cấp và cung cấp vị trí người điều khiển trong

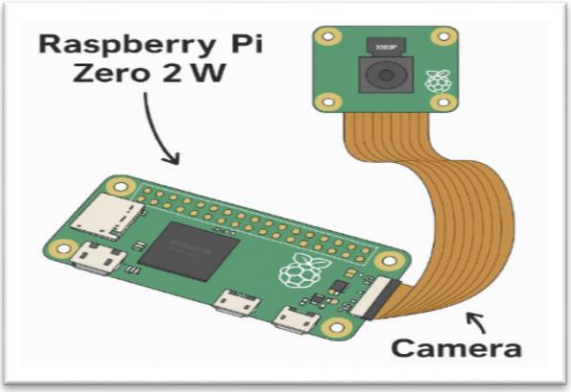
trường hợp xảy ra tai nạn. Màn hình LCD có thể hiển thị trạng thái hệ thống trực tiếp, trong khi ứng dụng di động giúp người dùng giám sát tình trạng an toàn từ xa thông qua giao diện người dùng.

Khối nguồn trong hệ thống mũ bảo hiểm IoT sử dụng pin 5V–3A để cung cấp năng lượng cho toàn bộ thiết bị, bao gồm Raspberry Pi Zero 2W, ESP32, các cảm biến và các module truyền thông như GPS và GSM. Nguồn pin phải đảm bảo dòng điện đủ lớn để đáp ứng yêu cầu tiêu thụ đồng thời của nhiều linh kiện, đồng thời tích hợp mạch bảo vệ quá dòng, quá áp và ngắn mạch nhằm tăng độ an toàn và tuổi thọ hệ thống. Tùy theo nhu cầu sử dụng, hệ thống có thể mở rộng thêm cổng sạc USB hoặc module sạc năng lượng mặt trời nhằm hỗ trợ hoạt động liên tục trong điều kiện thực địa.

3.2 Thiết kế phần cứng

3.2.1 Khối cảm biến và nhận diện

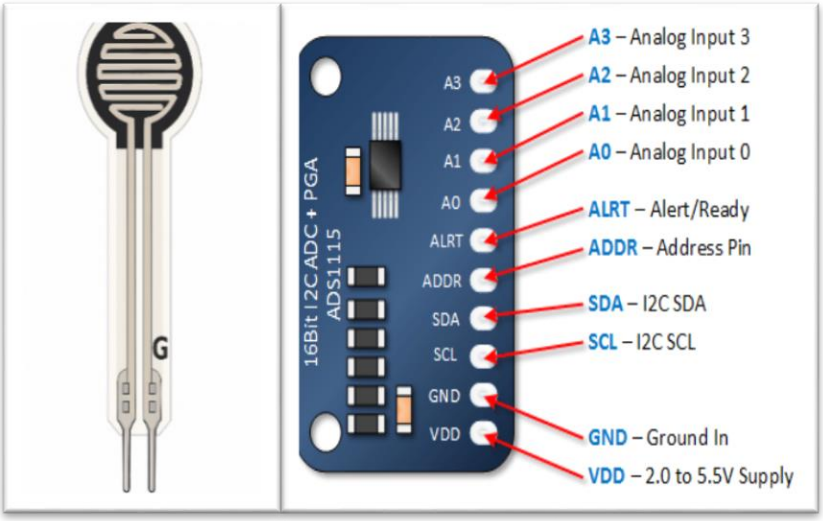
Khối cảm biến và nhận diện là bộ phận đầu vào quan trọng, thu thập dữ liệu về trạng thái đội mũ và hành vi người điều khiển. Các cảm biến được gắn trực tiếp trên mũ hoặc thân xe, cho phép hệ thống giám sát liên tục và phản ứng theo thời gian thực. Cách tiếp cận này tương tự mô hình tích hợp cảm biến rung và GPS trong nghiên cứu về mũ bảo hiểm thông minh (Sobhana và cộng sự, 2021).



CHình 3-2 Camera với Raspberry Pi

Thành phần đầu tiên trong khối này là camera, sử dụng giao tiếp CSI (Camera Serial Interface) để kết nối với Raspberry Pi như Hình 3-2. Camera có nhiệm vụ ghi

lại hình ảnh khuôn mặt người điều khiển, sau đó được xử lý bởi thuật toán học sâu tích hợp trong hệ thống nhằm xác định người dùng có đội mũ bảo hiểm hay không. Việc nhận diện hình ảnh được thiết kế để đảm bảo độ chính xác cao ngay cả trong điều kiện người dùng đeo khẩu trang hoặc kính.

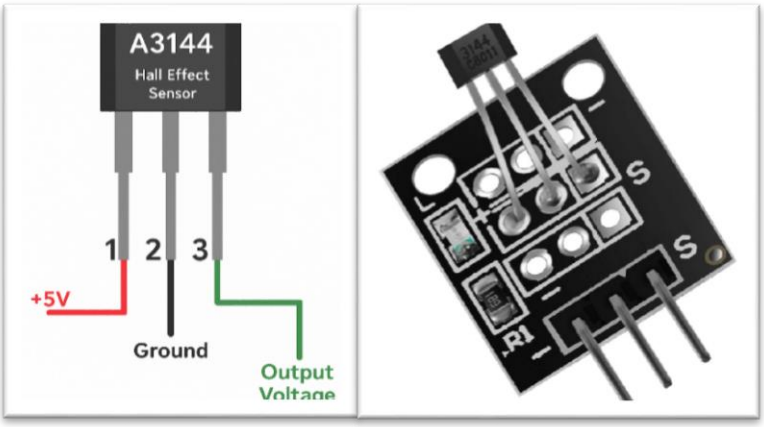


Hình 3-3 Cảm biến lực và bộ chuyển đổi ADC ADS1115

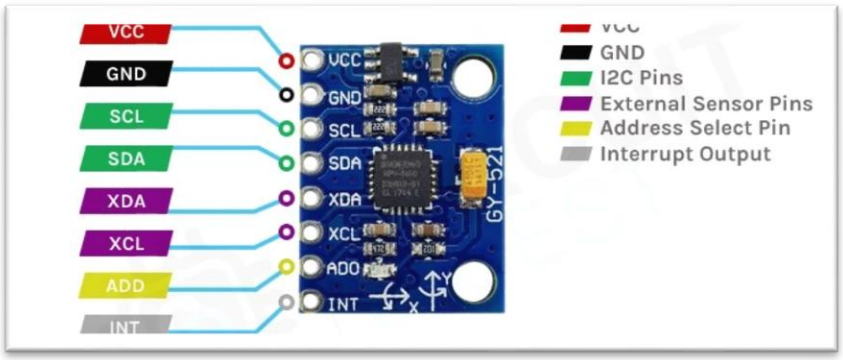
Thành phần tiếp theo trong khối là cảm biến lực như Hình 3-3 bên trái, được bố trí tại 4 điểm tiếp xúc bên trong lòng mũ nhằm phát hiện áp lực từ đầu người dùng khi đội mũ. Việc sử dụng 4 cảm biến giúp hệ thống đánh giá chính xác và đồng đều lực phân bố ở các vị trí gồm trán, đỉnh đầu và 2 bên đầu, từ đó xác định mũ đã được đội đúng cách hay chưa và xác định lực va chạm nếu có xảy ra. Các cảm biến lực này cho tín hiệu đầu ra ở dạng analog, không thể đọc trực tiếp bằng Raspberry Pi do bo mạch này không tích hợp sẵn bộ chuyển đổi ADC. Thông số kỹ thuật của cảm biến lực được trình bày tại Bảng A.2 – Phụ lục A.

Để khắc phục, mô hình sử dụng module ADC ADS1115 như Hình 3-3 bên phải, một bộ chuyển đổi tương tự và số với độ phân giải 16-bit, giao tiếp thông qua I2C. Module này có thể đọc đồng thời nhiều kênh đầu vào, phù hợp với việc đọc dữ liệu từ 4 cảm biến lực một cách chính xác và ổn định. Sau khi chuyển đổi sang tín hiệu số, dữ liệu sẽ được gửi về Raspberry Pi để xử lý và xác định điều kiện đội mũ hợp lệ. Thông số kỹ thuật của module ADC ADS1115 được nêu tại Bảng A.2 – Phụ lục A.

Cảm biến từ trường cũng được tích hợp tại vị trí quai mũ bảo hiểm nhằm phát hiện trạng thái gài quai thông qua sự hiện diện hoặc không có mặt của từ trường từ nam châm gắn trên phần quai. Khi nam châm ở gần cảm biến từ trường được kết nối với Raspberry Pi được minh họa trong Hình 3-4, hệ thống sẽ xác định rằng quai mũ chưa được gài đúng cách và do đó không cho phép khởi động xe nhằm đảm bảo an toàn. Ngược lại, khi cảm biến không phát hiện từ trường (tức là nam châm đã được kéo ra khỏi vùng từ trường do quai đã được gài đúng vị trí), hệ thống sẽ xác nhận người dùng đã gài quai hợp lệ và cho phép khởi động xe. Thông số của cảm biến Hall trình bày ở Bảng A.2 – Phụ lục A.



Hình 3-4 Cảm biến Hall với Raspberry Pi



Hình 3-5 Cảm biến gia tốc MPU6050

Ngoài ra, hệ thống cũng được trang bị cảm biến gia tốc MPU6050 được thể hiện trong Hình 3-5, có chức năng đo lường chuyển động ba chiều của mũ bảo hiểm nhằm phát hiện các hiện tượng như tăng tốc đột ngột, rung động mạnh hoặc thay đổi tư thế bất thường trong quá trình di chuyển. Dữ liệu thu được từ cảm biến gia tốc được kết hợp với cảm biến chuyển động để xác định các tình huống tiềm ẩn nguy cơ

tai nạn, chẳng hạn như va chạm, trượt ngã, hoặc mất kiểm soát khi vận hành phương tiện. Thông số kỹ thuật chi tiết của cảm biến MPU6050 được trình bày tại Bảng A.2- Phụ lục A.

Bổ trợ cho việc phát hiện các trạng thái chuyển động bất thường, hệ thống còn tích hợp hai cảm biến SW420 và SW520 nhằm nhận biết rung động kéo dài và dao động mạnh từ phía phương tiện. Cảm biến SW420 phản ứng nhạy với rung dọc theo trục thẳng đứng, trong khi SW520 phát hiện chuyển động lắc ngang. Khi cả hai cảm biến này cùng kích hoạt liên tục trong một khoảng thời gian nhất định, hệ thống sẽ xác định là có khả năng xảy ra va chạm hoặc mất kiểm soát, từ đó kích hoạt quy trình cảnh báo và xử lý khẩn cấp. Bảng A.2 – Phụ lục A trình bày chi tiết thông số 2 cảm biến này.

Tất cả các tín hiệu thu thập từ khối cảm biến và nhận diện sẽ được truyền về Raspberry Pi để xử lý và phân tích. Dựa trên kết quả phân tích, hệ thống sẽ đưa ra các quyết định điều khiển tương ứng, như cho phép khởi động xe, phát cảnh báo hoặc gửi tín hiệu khẩn cấp. Việc tích hợp đồng bộ nhiều loại cảm biến trong cùng một hệ thống giúp nâng cao khả năng nhận biết các tình huống thực tế, từ đó tăng cường mức độ an toàn và tính chủ động cho người điều khiển phương tiện. Một số nghiên cứu trước đó đã tích hợp các cảm biến như cảm biến rung và GPS vào mũ bảo hiểm thông minh nhằm phát hiện va chạm, xác định vị trí và giám sát hành vi người lái (S. Sobhana và cộng sự, 2021). Các tín hiệu sau xử lý được truyền đến thiết bị di động hoặc trung tâm điều khiển để cảnh báo kịp thời.

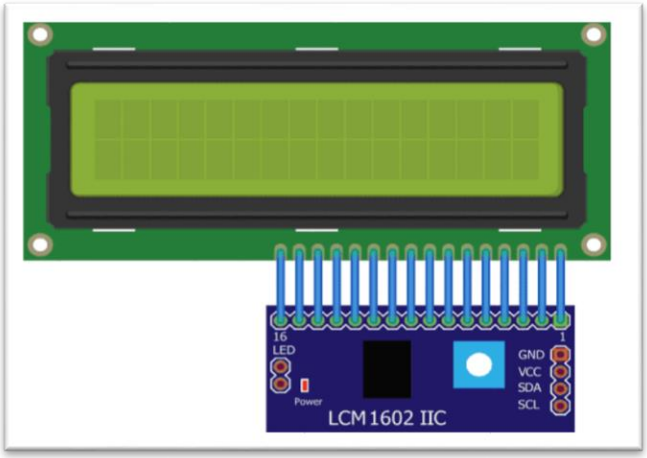
3.2.2 Khối truyền thông và hiển thị

Khối truyền thông và hiển thị bao gồm các thành phần chính: module GPS, module SIM GSM, màn hình LCD và giao diện ứng dụng di động (app). Đây là khối chức năng đóng vai trò kết nối hệ thống với môi trường bên ngoài, đặc biệt trong các tình huống khẩn cấp hoặc khi cần theo dõi trạng thái an toàn từ xa.

Raspberry Pi và ESP32 được cấu hình để giao tiếp với module GPS nằm bên phải Hình 3-6 và module SIM nằm bên trái Hình 3-6, thông qua chuẩn UART (Universal Asynchronous Receiver/Transmitter). Khi hệ thống phát hiện sự cố như

va chạm, mất cân bằng hoặc trạng thái nguy hiểm khác, Raspberry Pi sẽ thu thập dữ liệu tọa độ từ module GPS, sau đó chuyển thông tin này sang module SIM để thực hiện gửi tin nhắn khẩn cấp (SMS) đến số điện thoại đã được cấu hình trước, cơ chế tương tự cũng được triển khai trong nghiên cứu với module GSM đảm nhiệm chức năng liên lạc tự động khi có tai nạn của (Sekar và cộng sự, 2022). Trong trường hợp module SIM không có tín hiệu mạng hoặc gọi không thành công, hệ thống sẽ sử dụng kết nối Wi-Fi để gửi email khẩn cấp đến người thân đã đăng ký trước, đảm bảo cảnh báo được gửi đi kịp thời. Thông số kỹ thuật của module SIM900A và GPS NEO-6M được trình bày tương ứng tại Bảng A.2 – Phụ lục A.

Hình 3-6 Module GSM SIM900A và GPS NEO6M



Hình 3-7 Màn hình LED LCD I2C

Màn hình LCD 16x02 giao tiếp I2C sử dụng hai chân SCL (đồng hồ) và chân SDA (dữ liệu) để giao tiếp với ESP32. Màn hình LCD như Hình 3-7 này sẽ được tích hợp để hiển thị các thông tin trạng thái hệ thống trong thời gian thực, như "An toàn", "Chưa đội mũ", "Lỗi cảm biến" hoặc "Đang gửi cảnh báo". Điều này giúp người điều

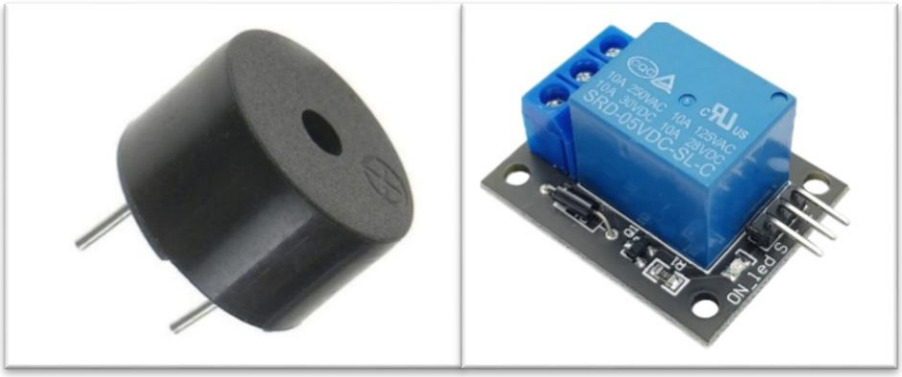
khiển có thể theo dõi trực tiếp tình trạng mũ bảo hiểm và hệ thống, góp phần nâng cao mức độ tương tác và chủ động điều chỉnh hành vi nếu cần thiết.

Khối truyền thông và hiển thị đóng vai trò quan trọng trong việc kết nối giữa hệ thống mũ bảo hiểm thông minh và người dùng, đảm bảo tính phản ứng nhanh, giám sát liên tục và hỗ trợ kịp thời trong các tình huống khẩn cấp.

3.2.3 Khối thực thi

Khối thực thi trong hệ thống mũ bảo hiểm IoT chịu trách nhiệm thực hiện các hành động phản hồi cuối cùng, chuyển đổi kết quả đánh giá từ khối xử lý trung tâm thành các tín hiệu điều khiển cụ thể tác động lên phần cứng. Trong mô hình này, khối thực thi bao gồm hai thành phần chính: buzzer cảnh báo được điều khiển trực tiếp bởi Raspberry Pi và relay điều khiển mạch khởi động xe, được điều khiển bởi ESP32.

Hình 3-8 nằm bên trái là buzzer được sử dụng như một thiết bị cảnh báo âm thanh, giúp thông báo trực tiếp đến người điều khiển khi phát hiện các tình huống vi phạm như: không đội mũ, đội mũ sai cách, không gài quai hoặc xảy ra va chạm. Raspberry Pi là thiết bị chịu trách nhiệm xử lý tín hiệu cảm biến và phân tích hình ảnh, do đó trung tâm này được cấu hình để xuất tín hiệu điều khiển trực tiếp đến buzzer thông qua chân GPIO. Điều này giúp hệ thống có thể phát ra cảnh báo tức thời, ngay khi phát hiện vi phạm, mà không cần thông qua ESP32, đảm bảo độ trễ thấp và phản ứng nhanh.



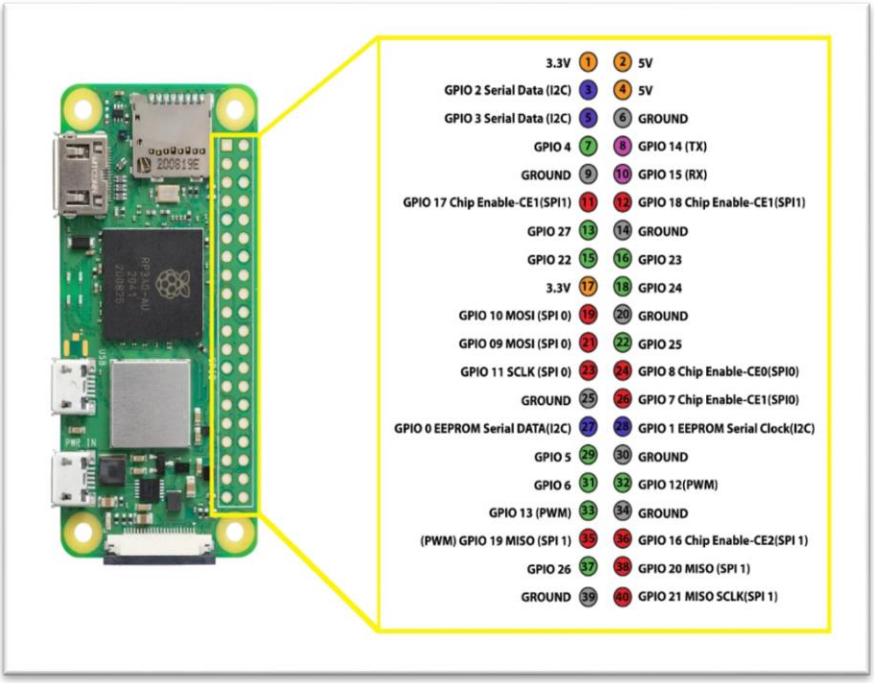
Hình 3-8 Buzzer và module relay 5VDC

Trong khi đó, ESP32 đảm nhận nhiệm vụ điều khiển mạch khởi động của xe thông qua module relay 1 kênh được minh họa phía bên phải trong Hình 3-8. Sau khi

Raspberry Pi hoàn tất các kiểm tra về điều kiện an toàn và xác nhận người điều khiển đủ điều kiện để vận hành xe, trung tâm hệ thống sẽ gửi một lệnh HTTP đến ESP32. ESP32 sẽ xử lý lệnh này và kích hoạt relay trong một khoảng thời gian ngắn (khoảng 1–2 giây), cho phép dòng điện chạy qua mạch đề, từ đó khởi động xe. Trong trường hợp không đạt điều kiện an toàn, relay sẽ không được kích hoạt và xe không thể khởi động.

3.2.4 Khối xử lý dữ liệu điều khiển

Khối xử lý dữ liệu điều khiển đóng vai trò trung tâm trong hệ thống mũ bảo hiểm thông minh, đảm nhận nhiệm vụ thu thập, phân tích dữ liệu từ cảm biến và hình ảnh đầu vào, từ đó đưa ra các quyết định điều khiển tương ứng. Khối này bao gồm hai thành phần chính: Raspberry Pi Zero 2W và vi điều khiển ESP32, được kết nối với nhau thông qua giao thức HTTP nội bộ thông qua mạng Wi-Fi cục bộ.

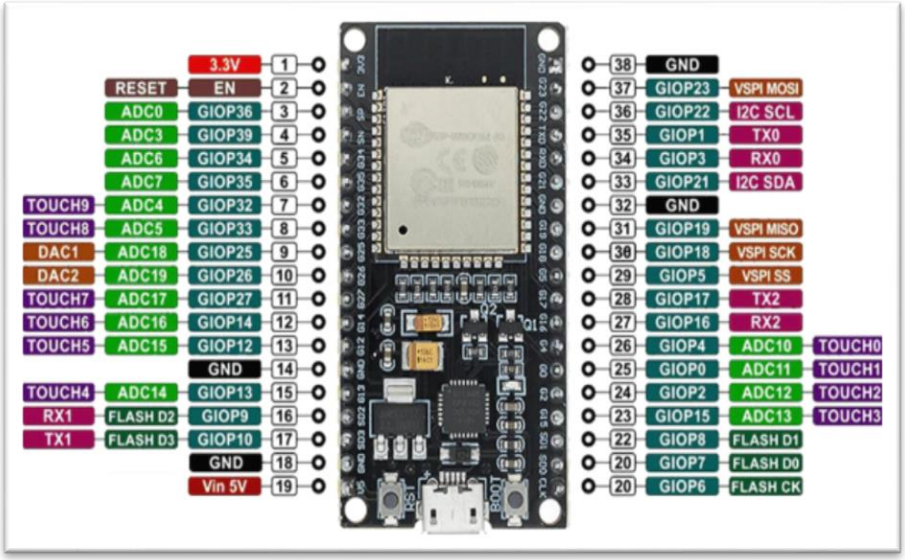


Hình 3-9 Raspberry Pi Zero 2W

Trong mô hình mũ bảo hiểm IoT này, Raspberry Pi Zero 2W như được minh họa trong Hình 3-9, đóng vai trò là "đầu não" trung tâm, xử lý toàn bộ thông tin thu thập được từ các cảm biến lực, cảm biến từ trường, cảm biến chuyển động và dữ liệu hình ảnh từ camera. Dựa trên thuật toán học sâu MobileNet SSD, Raspberry Pi nhận diện

trạng thái đội mũ của người dùng (có đội, đội đúng cách hay không) đồng thời đánh giá điều kiện gài quai và trạng thái chuyển động của người điều khiển. Sau khi phân tích, nếu hệ thống xác nhận rằng tất cả điều kiện an toàn đã được thỏa mãn, Raspberry Pi sẽ gửi một lệnh HTTP đến ESP32 để thực thi hành động tương ứng, ví dụ như cho phép xe khởi động. Ngược lại, nếu có vi phạm, hệ thống sẽ phát ra cảnh báo và từ chối khởi động xe nhằm đảm bảo an toàn.

Raspberry Pi là một máy tính nhúng nhỏ gọn, có khả năng chạy hệ điều hành Linux và hỗ trợ đầy đủ các giao tiếp phần cứng như GPIO, I2C, UART, CSI và kết nối mạng Wi-Fi. Với hiệu năng xử lý tương đối mạnh, bộ nhớ RAM linh hoạt và khả năng lập trình đa dạng (Python, C/C++), Raspberry Pi cho phép thực hiện các tác vụ tính toán phức tạp như xử lý ảnh, phân tích dữ liệu cảm biến, lưu trữ cơ sở dữ liệu và giao tiếp mạng. Thông số kỹ thuật của Raspberry Pi Zero 2W được dùng trong mô hình được thể hiện chi tiết thông qua Bảng A.4 – Phụ lục A.



Hình 3-10 Vi điều khiển ESP32

Trong mô hình mũ bảo hiểm IoT, vi điều khiển ESP32 đảm nhận vai trò là thiết bị chấp hành – thực hiện các hành động điều khiển cuối cùng dựa trên kết quả phân tích và đánh giá từ bộ xử lý trung tâm Raspberry Pi Zero 2W. Sau khi Raspberry Pi hoàn tất việc xử lý ảnh (dựa trên mô hình học sâu MobileNet SSD) và phân tích các tín hiệu từ hệ thống cảm biến (cảm biến lực, từ trường, gia tốc), chương trình sẽ

gửi một lệnh HTTP đến ESP32 nhằm kích hoạt hoặc từ chối việc khởi động xe tùy theo điều kiện an toàn được xác nhận.

ESP32 là một module điều khiển mạnh mẽ và linh hoạt được sử dụng phổ biến trong các ứng dụng IoT. Hình 3-10 điển hình cho mẫu ESP32 được phát triển bởi Espressif Systems, là một series các vi điều khiển trên một vi mạch giá rẻ, năng lượng thấp có hỗ trợ WiFi và dual-mode Bluetooth. Dòng ESP32 sử dụng bộ vi xử lý Tensilica Xtensa LX6 ở cả hai biến thể lõi kép và lõi đơn,và bao gồm các công tắc antenna tích hợp, RF balun, bộ khuếch đại công suất, bộ khuếch đại thu nhiễu thấp, bộ lọc và module quản lý năng lượng. Thông số kỹ thuật của ESP32 được thể hiện chi tiết trong Bảng A.3 – Phụ lục A.

Việc truyền nhận dữ liệu giữa Raspberry Pi Zero 2W và ESP32 được thực hiện trong điều kiện mạng cục bộ. Trong môi trường này, Raspberry Pi đảm nhận vai trò giao tiếp và xử lý dữ liệu từ các cảm biến nhận diện và điều khiển cảnh báo, trong khi đó ESP32 đảm nhận vai trò thực thi theo lệnh điều khiển của Raspberry Pi.

Đầu tiên, Raspberry Pi sử dụng giao thức I2C để giao tiếp với bộ chuyển đổi tín hiệu ADC ADS1115, qua đó thu thập dữ liệu từ bốn cảm biến lực được bố trí tại các vị trí trọng yếu bên trong mũ bảo hiểm. Trong đó, hai cảm biến đặt tại đỉnh đầu và trán đóng vai trò nhận diện người đội mũ, trong khi hai cảm biến còn lại đặt ở hai bên đầu có nhiệm vụ phát hiện va chạm, đặc biệt trong trường hợp người đội mũ gặp tai nạn có lực tác động mạnh từ phía trái hoặc phải. Sau khi xác nhận sự hiện diện của người đội mũ thông qua cảm biến lực, hệ thống tiếp tục sử dụng camera kết nối với Raspberry Pi qua cổng CSI để thực hiện bước xác minh bổ sung về hình ảnh.

Cụ thể, dữ liệu hình ảnh từ camera sẽ được xử lý bởi mô hình học sâu MobileNet SSD, hoạt động theo cơ chế nhận diện theo khối và phân loại đối tượng. Mô hình này đã được tối ưu hóa để nhận diện khuôn mặt người ngay cả khi người dùng đeo khẩu trang hoặc kính, từ đó đảm bảo độ chính xác cao trong các tình huống thực tế. Nếu người dùng được xác minh đã đội mũ đúng cách, hệ thống sẽ chuyển sang bước kiểm tra điều kiện gài quai nón. Tại đây, cảm biến từ trường Hall A3144 được gắn tại vành mũ, kết hợp với nam châm gắn trên dây quai, giúp phát hiện trạng

thái gài. Nếu cảm biến phát hiện tín hiệu từ trường ở khoảng cách gần (tức là nam châm vẫn nằm trong vùng từ), hệ thống xác định rằng dây quai chưa được gài và sẽ từ chối gửi lệnh kích hoạt relay cho ESP32. Ngược lại, khi cảm biến không ghi nhận từ trường, hệ thống kết luận rằng quai nón đã được gài đúng cách, từ đó cho phép ESP32 kích hoạt relay và cho phép xe máy khởi động.

Sau khi hoàn tất quá trình nhận diện và cho phép khởi động xe, hệ thống sẽ tiếp tục thực hiện chức năng giám sát người dùng theo thời gian thực trong suốt quá trình sử dụng mũ bảo hiểm và điều khiển phương tiện. Raspberry Pi sẽ giao tiếp với cảm biến gia tốc MPU6050 thông qua giao thức I2C để theo dõi các hành vi như tăng tốc đột ngột, xoay đầu, nghiêng đầu, hoặc té ngã. Đây là những dữ liệu quan trọng nhằm phát hiện các tình huống nguy hiểm trong quá trình vận hành. Song song đó, các cảm biến SW420 và SW520 được kết nối với ESP32 thông qua tín hiệu số (digital input), có nhiệm vụ theo dõi chuyển động và phát hiện tai nạn ở cấp độ thiết bị ngoại vi. Việc phân chia xử lý giữa Raspberry Pi và ESP32 giúp hệ thống vận hành ổn định và có khả năng phản hồi nhanh chóng với nhiều loại tín hiệu cùng lúc.

Raspberry Pi và ESP32 kết nối với nhau qua giao thức HTTP trên mạng cục bộ, đảm bảo trao đổi dữ liệu điều khiển và phân tích diễn ra nhanh chóng, ổn định. Đồng thời, cả hai thiết bị giao tiếp với module định vị GPS NEO-6M và SIM900A qua UART, cho phép xác định vị trí người dùng. Khi phát hiện va chạm hoặc hành vi bất thường, hệ thống sẽ lấy tọa độ từ GPS và sử dụng SIM để thực hiện cuộc gọi hoặc gửi tin nhắn khẩn cấp đến số điện thoại đã được thiết lập trước, giúp phản ứng kịp thời trong các tình huống nguy hiểm.

3.2.5 Khối nguồn

Khối nguồn đóng vai trò cung cấp năng lượng cho toàn bộ hệ thống mũ bảo hiểm IoT, bao gồm các thành phần chính như Raspberry Pi Zero 2W, vi điều khiển ESP32, các cảm biến, các module và các thiết bị ngoại vi khác. Nguồn năng lượng chính của hệ thống được lựa chọn là power bank thương mại chuẩn 5V–3A, sử dụng cell pin Li-ion tích hợp sẵn cùng mạch sạc và mạch bảo vệ bên trong. Loại nguồn này đảm bảo cung cấp dòng điện ổn định và liên tục cho các thiết bị tiêu thụ điện năng

cao như Raspberry Pi Zero 2W và module SIM900A. Với thiết kế nhỏ gọn, dễ tháo lắp và tiện lợi khi mang theo, power bank giúp tăng tính cơ động cho hệ thống mũ bảo hiểm IoT.

Bảng 3-1 Công suất tiêu ước tính của hệ thống

Thành phần	Công suất ước tính
Raspberry Pi Zero 2W	1.5 W
ESP32	0.3 ~ 0.4 W
SIM900A (khi gửi SMS/call)	~1.0 W
GPS NEO-6M	~0.06 W
MPU6050 + SW420 + SW520	~0.05 W
Buzzer + LCD	~0.1 ~ 0.2 W
Tổng cộng (khi peak)	≈ 2.5 ~ 3.0 W

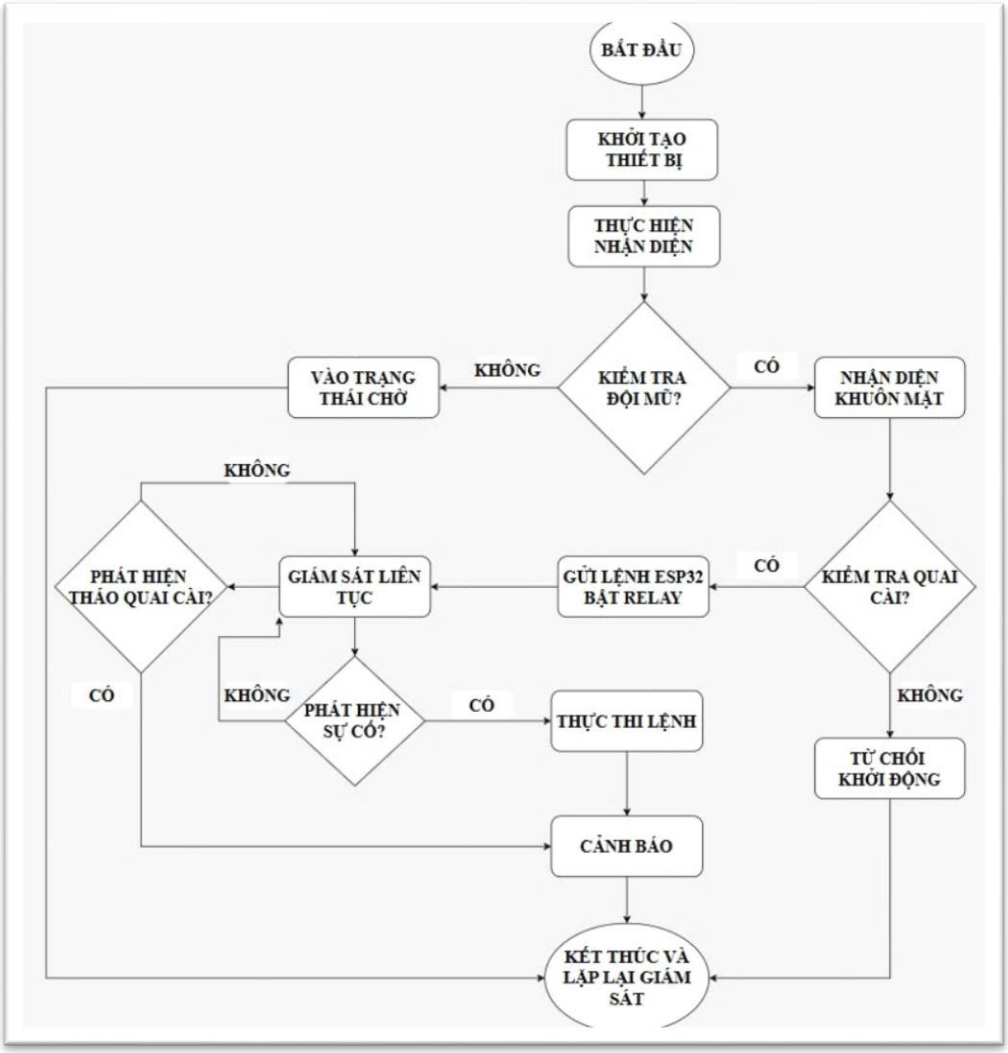
Tính toán cho thấy, khi toàn bộ hệ thống hoạt động đồng thời ở trạng thái tải cao nhất (bao gồm xử lý ảnh, gửi dữ liệu, phát cảnh báo và vận hành các cảm biến), mức tiêu thụ điện năng ước tính vào khoảng 2.5 ~ 3W, trong đó Raspberry Pi Zero 2W tiêu thụ trung bình khoảng 1.5W, còn lại là cho ESP32, các cảm biến và các module truyền thông. Bảng 3-1 mô tả chi tiết thông số công suất ước tính được dùng trong hệ thống.

3.3 Thiết kế phần mềm

3.3.1 Lưu đồ giải thuật hệ thống

Lưu đồ giả thuật hệ thống của mô hình trong Hình 3-11 không chỉ mô tả logic xử lý tuần tự của hệ thống mà còn thể hiện rõ cơ chế phản hồi theo vòng lặp, giúp hệ thống hoạt động liên tục trong suốt quá trình sử dụng. Một điểm đáng chú ý là quá trình xử lý được chia thành hai giai đoạn rõ rệt: giai đoạn xác minh điều kiện an toàn trước khi khởi động xe (bao gồm kiểm tra đội mũ, nhận diện khuôn mặt và gài quai) và giai đoạn giám sát thời gian thực sau khi phương tiện đã được khởi động.

Ngoài ra, nhánh xử lý tình huống khẩn cấp được thiết kế song song với chức năng ghi nhận trạng thái hệ thống, cho thấy khả năng vừa phản ứng nhanh với sự cố vừa hỗ trợ lưu trữ dữ liệu phục vụ đánh giá và phân tích sau này. Cấu trúc giải thuật theo hướng phân nhánh và tự động hóa như vậy là nền tảng vững chắc để hệ thống có thể mở rộng thêm các chức năng nâng cao như học hành vi người dùng, cảnh báo sớm nguy cơ tai nạn, hoặc tích hợp với nền tảng giám sát thông minh trong các phiên bản tương lai.



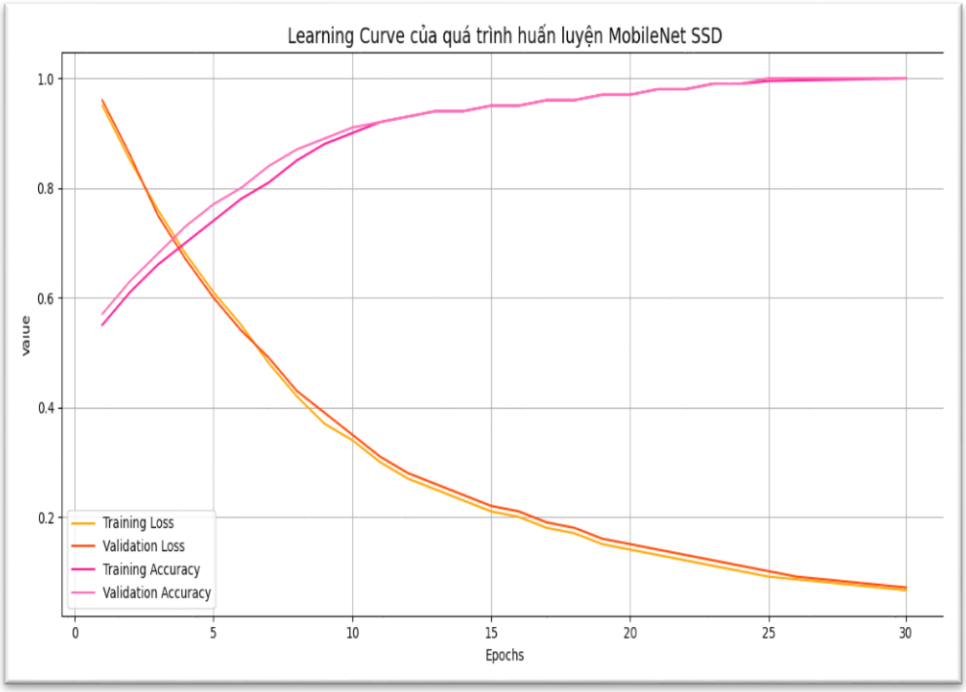
Hình 3-11 Lưu đồ giải thuật hệ thống

3.3.2 Xây dựng chương trình nhận diện hình ảnh

Trong hệ thống mũ bảo hiểm IoT hỗ trợ giám sát an toàn giao thông, chức năng nhận diện hình ảnh đóng vai trò then chốt nhằm đảm bảo người điều khiển

phương tiện thực sự đội mũ bảo hiểm đúng cách trước khi được phép khởi động xe. Đặc biệt, hệ thống cần phải nhận diện chính xác trong các trường hợp người điều khiển đeo khẩu trang hoặc kính mắt, đây những yếu tố có thể làm nhiễu quá trình nhận diện khuôn mặt thông thường.

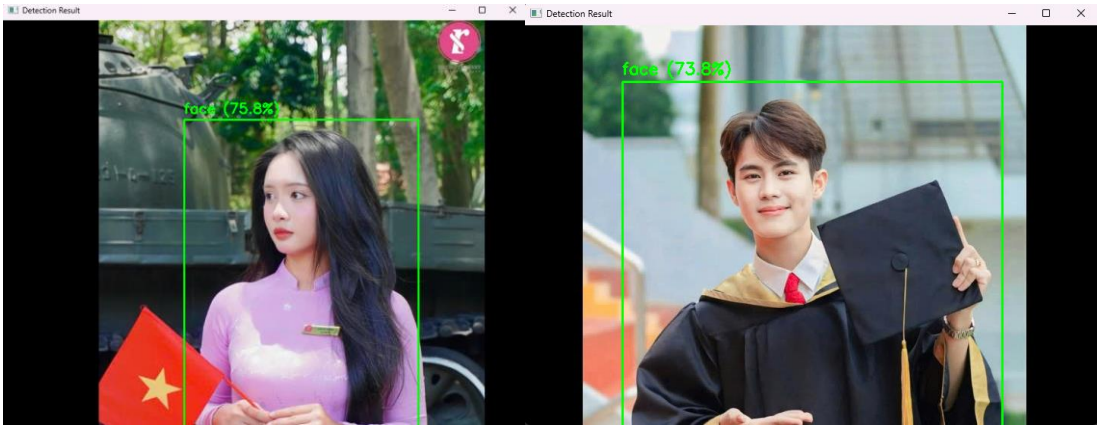
Để đáp ứng yêu cầu này, hệ thống lựa chọn mô hình MobileNet SSD làm thuật toán chính cho quá trình nhận diện hình ảnh. Mô hình này kết hợp giữa mạng nerual tích chập MobileNet, đây là một kiến trúc CNN nhẹ, được tối ưu hóa cho thiết bị có tài nguyên hạn chế, cùng với thuật toán SSD có khả năng phát hiện đối tượng theo cơ chế một bước, cho phép xử lý nhanh và phù hợp với bài toán thời gian thực. Cụ thể, MobileNet sử dụng kỹ thuật convolution tách chiều sâu (depthwise separable convolution), giúp giảm thiểu số lượng tham số và khối lượng tính toán, đồng thời vẫn giữ được khả năng trích xuất đặc trưng hình ảnh hiệu quả. Việc kết hợp với SSD giúp hệ thống phát hiện nhiều đối tượng cùng lúc, chỉ trong một lần suy luận, từ đó đảm bảo hiệu năng cao ngay trên phần cứng hạn chế như Raspberry Pi Zero 2W.



Hình 3-12 Learning Curve của quá trình huấn luyện mô hình MobileNet SSD

Mục tiêu nhận diện ảnh trong hệ thống đặt ra là phát hiện các đối tượng thuộc ba lớp chính: person (người), mask (khẩu trang) và glasses (kính mắt). Ngưỡng độ

chính xác yêu cầu là từ 50% trở lên, nhằm đảm bảo khả năng vận hành tin cậy trong môi trường thực tế, nơi người dùng có thể thay đổi trạng thái đeo khẩu trang hoặc kính. Khi phát hiện được ít nhất một trong ba đối tượng trên, hệ thống xác định người điều khiển đã đảm bảo các yêu cầu an toàn cơ bản để khởi động phương tiện.



Hình 3-13 Kết quả nhận diện của mô hình

Quá trình huấn luyện mô hình MobileNet SSD được thực hiện trên nền tảng Google Colab kết hợp cùng mã nguồn của TensorFlow, với tập dữ liệu được thu thập được từ môi trường không gian mạng phổ biến (Nguyễn Chiến Thắng, 2022). Mô hình được huấn luyện trong 30 epoch, sử dụng thuật toán tối ưu hóa Adam, với dữ liệu được chia theo tỷ lệ 80% cho huấn luyện và 20% cho kiểm thử. Kết quả huấn luyện được ghi nhận và theo dõi thông qua công cụ TensorBoard trên nền tảng Google Colab, từ đó xuất đồ thị Learning Curve thể hiện sự hội tụ của mô hình trong quá trình huấn luyện. Hình 3-12 biểu diễn đồ thị Learning Curve cho thấy giá trị hàm mất mát (Loss) giảm dần đều theo số lượng epoch, trong khi độ chính xác (Accuracy) tăng lên ổn định. Sau khoảng 25 epoch, độ chính xác trên tập kiểm thử đạt trên 95%, đồng thời hàm mất mát duy trì ở mức thấp, cho thấy mô hình học hiệu quả và không xảy ra hiện tượng quá khớp (overfitting). Quá trình xuất đồ thị Learning Curve được thực hiện bằng cách lấy dữ liệu từ TensorBoard và trực quan hóa bằng thư viện matplotlib, giúp đánh giá trực quan chất lượng mô hình sau khi huấn luyện. Kết quả này chứng tỏ mô hình MobileNet SSD sau khi được tinh chỉnh hoàn toàn đáp ứng yêu cầu về tốc độ suy luận và độ chính xác cần thiết cho hệ thống nhận diện hình ảnh trên mũ bảo hiểm IoT.

Kết quả nhận diện ảnh sau khi triển khai mô hình MobileNet SSD cho thấy khả năng phát hiện khuôn mặt người điều khiển đạt độ chính xác tốt trong các điều kiện khác nhau. Hình 3-13 thể hiện ví dụ kết quả nhận diện trên hai ảnh đầu vào có góc chụp và điều kiện ánh sáng khác biệt. Trong các trường hợp này, mô hình có thể nhận diện đúng vùng khuôn mặt với độ tin cậy lần lượt là 75.8% và 73.8%, đồng thời vẽ chính xác bounding box bao quanh đối tượng. Các kết quả nhận diện này phù hợp với ngưỡng yêu cầu (> 50%) đặt ra cho hệ thống, đảm bảo khả năng xác thực trạng thái đội mũ khi áp dụng trong thực tế. Bên cạnh đó, việc mô hình duy trì được hiệu năng ổn định trên nhiều mẫu ảnh khác nhau cũng cho thấy tính tổng quát tốt, giúp tăng độ tin cậy khi triển khai trên thiết bị nhúng như Raspberry Pi Zero 2W.

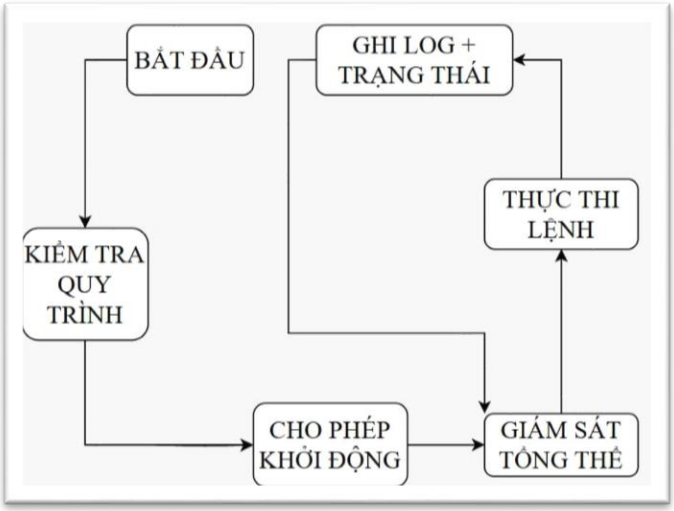
3.3.3 Xây dựng chương trình giám sát hệ thống

Chương trình giám sát và cảnh báo là thành phần trung tâm của hệ thống mũ bảo hiểm IoT, chịu trách nhiệm thu thập và phân tích dữ liệu từ các cảm biến lực, cảm biến từ trường Hall, cảm biến gia tốc, cảm biến rung, cảm biến nghiêng và hình ảnh từ camera. Dựa trên kết quả phân tích, chương trình sẽ đưa ra các quyết định điều khiển nhằm đảm bảo an toàn cho người điều khiển phương tiện. Toàn bộ chương trình được xây dựng và triển khai trên nền tảng Raspberry Pi Zero 2W, sử dụng ngôn ngữ Python kết hợp với các thư viện chuyên dụng như tflite_runtime, sklearn, google-api-python-client, RPi.GPIO và các thư viện xử lý ngoại vi khác.

Sơ đồ luồng xử lý của chương trình được thể hiện trong Hình 3-14, từ bước đầu là khởi động, hệ thống lần lượt thực hiện nhận diện hình ảnh, kiểm tra quai gài và cho phép khởi động xe khi đủ điều kiện an toàn. Sau khi khởi động, hệ thống chuyển sang giám sát tổng thể, liên tục phân tích dữ liệu từ các cảm biến để phát hiện nguy cơ tai nạn và thực hiện cảnh báo kịp thời. Ngoài ra, hệ thống luôn chờ lệnh thực thi từ ứng dụng di động để đảm bảo khả năng phản hồi linh hoạt trong quá trình sử dụng thực tế.

Quy trình hoạt động của chương trình được chia thành hai giai đoạn chính là kiểm tra điều kiện an toàn ban đầu và giám sát liên tục sau khi xe được khởi động. Ở giai đoạn đầu, hệ thống sẽ kiểm tra lực từ cảm biến FSR để xác nhận người dùng đã

đội mũ đúng cách kết hợp nhận diện hình ảnh qua mô hình MobileNet SSD và kiểm tra trạng thái gài quai bằng cảm biến Hall A3144. Khi đủ điều kiện, Raspberry Pi gửi lệnh HTTP cho ESP32 kích hoạt relay khởi động xe, nếu không, hệ thống sẽ từ chối khởi động xe. Khi xe đã khởi động, chương trình chuyển sang giám sát liên tục dữ liệu từ MPU6050 được phân tích bằng mô hình Random Forest để nhận diện các hành vi như té ngã, va chạm, tăng tốc đột ngột, cảm biến Hall kiểm tra quai gài, cảm biến SW420 và SW520 giúp phát hiện rung động hoặc nghiêng xe để truyền tin về cho Raspberry Pi xác thực lại tình trạng chung để đảm bảo độ chính xác. Nếu phát hiện nguy hiểm, hệ thống sẽ ghi nhận để giao tiếp truyền tin và phát cảnh báo, đồng thời ghi sự kiện kèm thông tin GPS lên Google Sheets. Ngoài ra, nếu trong quá trình cảnh báo hệ thống không thể thực hiện cuộc gọi hay gửi tin qua module SIM do mất sóng, chức năng gửi email tức thời kết hợp với hai phương thức gửi tin nhắn và gọi điện sẽ đóng vai trò phụ trợ để đảm bảo được chức năng truyền thông tin cảnh báo của hệ thống.



Hình 3-14 Sơ đồ luồng xử lý của hệ thống

Để nâng cao hiệu quả nhận diện các tình huống nguy hiểm trong quá trình giám sát liên tục, hệ thống sử dụng thuật toán học máy Random Forest nhằm phân tích dữ liệu dạng bảng thu thập từ cảm biến MPU6050 (Tiệp Vũ,2021). Cảm biến này cung cấp các giá trị gia tốc (ax, ay, az) và vận tốc góc (gx, gy, gz) trên ba trục không gian. Trước khi huấn luyện mô hình, vị trí gắn cảm biến của mũ bảo hiểm phải được

xác định nhằm biết được vị trí của trục x nằm ở đâu so với mặt đất (trạng thái đứng yên ghi nhận ax ≈ +1g). Trong quá trình thu thập dữ liệu, vị trí gắn cảm biến MPU6050 đã được cố định để đảm bảo tính nhất quán giữa dữ liệu huấn luyện và dữ liệu vận hành thực tế. Cụ thể, cảm biến được gắn trên mũ sao cho khi ở trạng thái đứng yên, các giá trị ghi nhận trung bình đạt được là ax ≈ +1.06g, ay ≈ 0.00g, az ≈ +0.15 ~ 0.20g. Việc chọn trục X hướng gần vuông góc với mặt đất giúp quá trình phân tích và huấn luyện mô hình đạt độ chính xác cao hơn, đồng thời loại bỏ ảnh hưởng của thay đổi tư thế hoặc vị trí lắp đặt cảm biến. Ngoài trục X làm trục tham chiếu, các giá trị ay và az đóng vai trò nhận diện các dao động theo phương ngang khi xảy ra rung động, nghiêng hoặc va chạm. Bên cạnh đó, các giá trị vận tốc góc gx, gy, gz từ cảm biến gyroscope giúp hệ thống phát hiện các chuyển động quay hoặc lật đột ngột của mũ, là dấu hiệu quan trọng để nhận diện tình huống té ngã hoặc va chạm.

Hình 3-15 Biểu đồ dao động của mũ bảo hiểm

Biểu đồ ở Hình 3-15 thể hiện dao động của ba trục gia tốc (ax, ay, az) thu được từ cảm biến MPU6050 trong năm trạng thái hoạt động khác nhau của mũ bảo hiểm,

gồm: normal, fall, shake, still và head_shake. Dữ liệu được thu thập thực tế từ cảm biến khi tiến hành mô phỏng các hành vi tương ứng, mỗi trạng thái lấy mẫu 100 khung dữ liệu liên tiếp từ tập huấn luyện. Kết quả cho thấy, trong trạng thái normal, ax dao động ổn định quanh ~1g, ay và az dao động nhỏ quanh 0, phản ánh trạng thái ổn định của mũ. Khi xảy ra fall, các trục gia tốc dao động mạnh và không ổn định, đặc biệt là ax và az là đặc trưng cho tình huống té ngã. Trạng thái shake cho thấy ay và az có biên độ dao động lớn hơn normal nhưng không đến mức hỗn loạn như fall. Ở trạng thái still, các trục dao động rất nhỏ, gần như ổn định tuyệt đối, trong khi trạng thái head_shake thể hiện các dao động có tần suất cao trên ay và az do chuyển động quay đầu. Qua biểu đồ có thể nhận thấy rằng việc kết hợp dữ liệu từ các trục ax, ay, az giúp mô hình Random Forest phân biệt rõ ràng các hành vi khác nhau, từ đó nâng cao độ chính xác trong nhận diện tình huống thực tế.

3.3.4 Cấu trúc giao tiếp giữa các cảm biến và bộ xử lý

Hệ thống mũ bảo hiểm IoT tích hợp nhiều thành phần phần cứng và phần mềm, liên kết chặt chẽ nhằm đảm bảo khả năng giám sát, cảnh báo và phản ứng kịp thời khi có sự cố. Để các thành phần này phối hợp vận hành ổn định và đạt hiệu quả cao, quá trình giao tiếp truyền dữ liệu giữa các thiết bị phải được thiết kế hợp lý, đồng bộ, đáp ứng các yêu cầu về độ trễ thấp và độ tin cậy cao. Cấu trúc giao tiếp trong hệ thống bao gồm: giao tiếp giữa Raspberry Pi Zero 2W và các cảm biến; giao tiếp giữa ESP32 và các cảm biến ngoại vi, module SIM900A; giao tiếp giữa Raspberry Pi và ESP32 thông qua HTTP REST API; và quy cách vận hành tổng thể của giao thức này trong môi trường mạng cục bộ. Thiết kế phân tầng như vậy giúp hệ thống đạt được sự linh hoạt, dễ mở rộng và có khả năng vận hành bền vững trong các điều kiện thực tế. Bảng 3-2 mô tả chi tiết các giao thức mà các thành phần giao tiếp với nhau trong hệ thống..

Bảng 3-2 Tóm tắt giao thức giao tiếp

Thiết bị	Cảm biến hoặc module	Giao thức giao tiếp
Raspberry Pi	MPU6050	I2C
Raspberry Pi	ADS1115 + FSR + RFP	I2C

Raspberry Pi	Hall A3144, Buzzer	GPIO Digital
Raspberry Pi	Camera	CSI
Raspberry Pi	GPS NEO-6M	UART
Raspberry Pi ⇔ ESP32	Điều khiển chấp hành	HTTP REST API (Flask)
ESP32	SW420, SW520, Relay	GPIO Digital
ESP32	LCD-I2C	I2C
ESP32	GSM SIM900A	UART (AT Command)

Trong hệ thống mũ bảo hiểm IoT, Raspberry Pi Zero 2W đóng vai trò trung tâm xử lý dữ liệu cảm biến và giám sát tổng thể. Việc giao tiếp giữa Raspberry Pi và các cảm biến được xây dựng dựa trên các giao thức tiêu chuẩn phù hợp với từng loại tín hiệu: I2C cho các cảm biến tuyến tính và ADC ngoài, CSI cho camera truyền hình ảnh độ phân giải cao, UART cho module GPS, và GPIO digital cho cảm biến trạng thái. Cách sử dụng các giao thức này giúp đảm bảo tốc độ truyền dữ liệu ổn định, giảm thiểu độ trễ, đồng thời tận dụng tối đa tài nguyên phần cứng sẵn có trên Raspberry Pi. Đặc biệt, việc khai thác cổng CSI cho camera và giao tiếp I2C cho các cảm biến cho phép Raspberry Pi thu nhận dữ liệu đồng bộ, phục vụ các thuật toán nhận diện và phân tích hành vi một cách chính xác và hiệu quả trong thời gian thực.

Vi điều khiển ESP32 trong hệ thống chủ yếu đảm nhiệm vai trò chấp hành và giám sát ngoại vi. Các cảm biến ngoại vi như cảm biến rung SW420, cảm biến nghiêng SW520, cùng module SIM900A được kết nối trực tiếp với ESP32 thông qua các giao thức đơn giản nhưng hiệu quả như GPIO Digital và UART. Thiết kế này giúp ESP32 có thể xử lý và phản ứng nhanh chóng đối với các sự kiện ngoại vi, giảm tải cho Raspberry Pi và đảm bảo khả năng đáp ứng tốt trong các tình huống khẩn cấp. Đặc biệt, việc giao tiếp UART giữa ESP32 và module GSM SIM900A cho phép ESP32 có thể thực hiện trực tiếp các tác vụ gọi điện hoặc gửi SMS cảnh báo. Điều này giúp hệ thống có khả năng hoạt động độc lập và duy trì liên lạc ngay cả khi Raspberry Pi tạm thời không sẵn sàng hoặc mất kết nối mạng cục bộ.

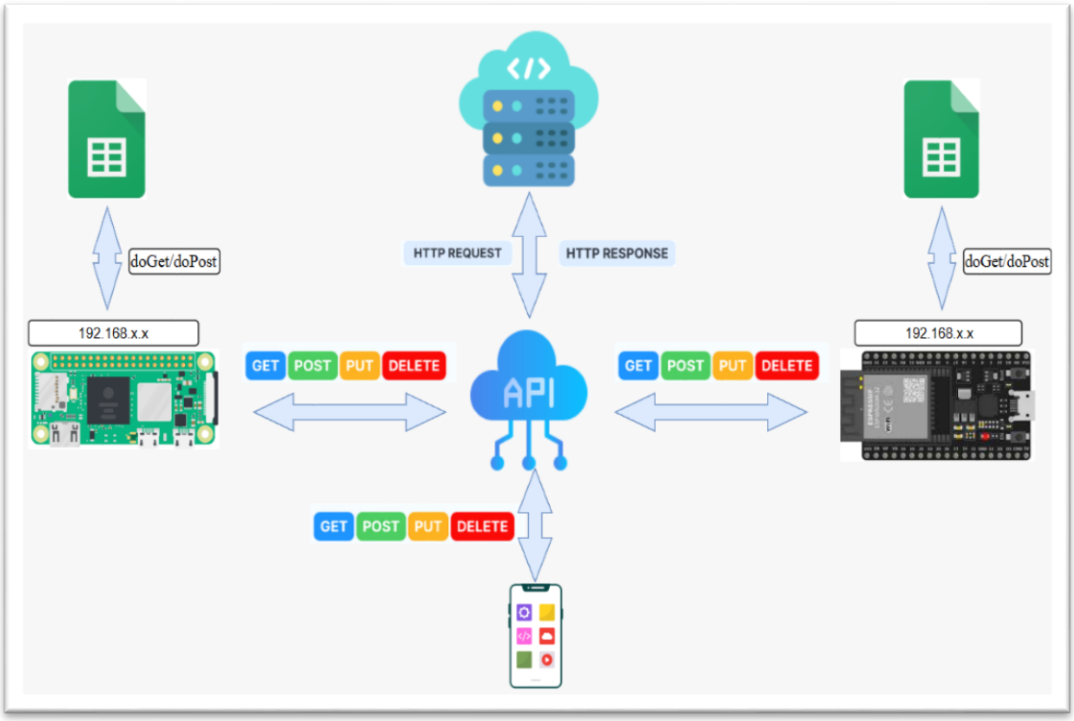
3.3.5 Giao thức truyền thông giữa Raspberry Pi và ESP32

Giao tiếp giữa Raspberry Pi và ESP32 trong hệ thống đóng vai trò là kênh truyền thông nội bộ quan trọng nhất, điều phối toàn bộ hoạt động của khối chấp hành. Trong kiến trúc này, Raspberry Pi được cấu hình làm máy chủ HTTP server (Flask), còn ESP32 đóng vai trò client, liên tục chờ nhận lệnh từ Raspberry Pi hoặc thực thi các request khi được yêu cầu. Phương thức giao tiếp được xây dựng dựa trên mô hình client-server thông qua giao thức HTTP/1.1, đảm bảo sự đơn giản và khả năng mở rộng. Mỗi khi Raspberry Pi hoàn thành quá trình phân tích dữ liệu từ cảm biến và mô hình học sâu, hệ thống sẽ tạo các HTTP request (theo phương thức GET hoặc POST), gửi trực tiếp đến ESP32 thông qua mạng Wi-Fi nội bộ. Ví dụ, khi đủ điều kiện an toàn, Raspberry Pi sẽ gửi request GET /api/relay?state=on đến ESP32, yêu cầu kích hoạt relay khởi động xe. Các request khác có thể bao gồm lệnh bật/tắt buzzer hoặc thay đổi trạng thái cảm biến phụ trợ.

Về phía ESP32, firmware được thiết kế để có một vòng lặp lắng nghe các endpoint REST API được định nghĩa sẵn. Khi nhận được một HTTP request từ Raspberry Pi, ESP32 sẽ phân tích các tham số trong URL hoặc phần body của request, thực thi lệnh tương ứng trên phần cứng (bật/tắt GPIO cho relay hoặc buzzer), sau đó trả về một HTTP response với mã trạng thái 200 “OK” hoặc thông báo lỗi nếu có sự cố. Toàn bộ quá trình giao tiếp diễn ra trong thời gian thực, với độ trễ trung bình trong mạng LAN cục bộ là dưới 50 ms, đảm bảo khả năng phản hồi kịp thời trong các tình huống khẩn cấp.

HTTP (Hypertext Transfer Protocol) là một giao thức tầng ứng dụng hoạt động theo mô hình client-server, trong đó client chủ động gửi các request và server phản hồi bằng các response tương ứng. Trong hệ thống mũ bảo hiểm IoT, HTTP đóng vai trò như một lớp giao vận chính giữa Raspberry Pi và ESP32, cho phép truyền tải các lệnh điều khiển và nhận phản hồi trạng thái một cách đơn giản và dễ kiểm thử. Các phương thức HTTP được sử dụng trong hệ thống bao gồm GET (dùng để truy vấn trạng thái hoặc gửi các lệnh có tính chất idempotent) và POST (dùng để gửi các lệnh có nội dung hoặc tham số thay đổi).

REST API (Representational State Transfer - Application Programming Interface) là một phong cách thiết kế API hiện đại, tuân thủ các nguyên lý REST, giúp hệ thống đạt tính mở rộng và linh hoạt cao. Các nguyên tắc cơ bản của REST bao gồm: phân tách rõ ràng giữa client và server (giúp các thành phần có thể phát triển hoặc nâng cấp một cách độc lập), tính stateless (mỗi request HTTP là độc lập, không lưu trạng thái phiên làm việc, giúp hệ thống dễ mở rộng và xử lý song song hiệu quả), khả năng cacheable (có thể áp dụng kỹ thuật lưu cache để tăng tốc độ phản hồi khi cần), tính uniform interface (tài nguyên được định danh qua URI và các hành động được thực hiện bằng các phương thức HTTP tiêu chuẩn như GET, POST, PUT, DELETE), và khả năng layered system (cho phép hệ thống có thể phân tầng hoặc mở rộng thông qua các middleware hoặc proxy).



Hình 3-16 Kiến trúc giao tiếp của hệ thống

Hình 3-16 minh họa tổng quan kiến trúc giao tiếp HTTP REST API trong hệ thống mũ bảo hiểm IoT, cho thấy sự tương tác giữa các thành phần: Raspberry Pi Zero 2W, ESP32, dịch vụ Google Sheets và ứng dụng di động. Cả Raspberry Pi và ESP32 đều được gán địa chỉ IP trong mạng LAN nội bộ (192.168.x.x), cho phép hai

thiết bị này trao đổi trực tiếp thông qua các phương thức HTTP chuẩn như GET, POST, PUT và DELETE. Raspberry Pi đóng vai trò là server Flask chính trong hệ thống, định nghĩa các REST endpoint để điều khiển các hành vi chấp hành (relay, buzzer) hoặc ghi nhận trạng thái từ ESP32. Ngoài ra, thông qua các hàm doGet hoặc doPost, cả Raspberry Pi và ESP32 đều có khả năng tương tác với dịch vụ Google Sheets để đồng bộ địa chỉ IP và hỗ trợ quá trình giao tiếp. Một thành phần quan trọng trong sơ đồ là ứng dụng di động, được thiết kế để giao tiếp trực tiếp với tầng REST API của Raspberry Pi và ESP32, cho phép người dùng điều khiển hoặc giám sát hệ thống từ xa. Kiến trúc REST API hai chiều này giúp hệ thống đạt được tính mở rộng cao, hỗ trợ giao tiếp đa nền tảng và đảm bảo khả năng vận hành linh hoạt trong môi trường mạng cục bộ.

Việc sử dụng mô hình HTTP REST API giữa Raspberry Pi và ESP32 không chỉ giúp đơn giản hóa thiết kế hệ thống, mà còn tạo ra một giao diện truyền thông tiêu chuẩn hóa, giúp dễ dàng kiểm tra, giám sát hoặc mở rộng các chức năng mới mà không cần thay đổi phần cứng. Đặc biệt, các lệnh từ Raspberry Pi được xây dựng theo mô hình idempotent (một lệnh gửi nhiều lần vẫn đảm bảo kết quả không thay đổi), đảm bảo an toàn cho các tình huống mất gói hoặc kết nối mạng tạm thời.

3.4 Xây dựng giao diện người dùng

3.4.1 Định hướng thiết kế giao diện người dùng

Trong hệ thống mũ bảo hiểm thông minh IoT, ứng dụng di động đóng vai trò là kênh giao tiếp chính giữa người dùng và hệ thống giám sát an toàn. Để đáp ứng tốt yêu cầu về khả năng sử dụng, tính thân thiện và hiệu quả trong các tình huống thực tế, việc thiết kế giao diện người dùng (UI) cần tuân thủ các định hướng rõ ràng, vừa đảm bảo tính trực quan, dễ thao tác, vừa đảm bảo tính phản hồi nhanh và đồng bộ dữ liệu theo thời gian thực.

Mục tiêu thiết kế giao diện của hệ thống tập trung vào việc cung cấp cho người dùng một trải nghiệm nhất quán và dễ hiểu. Giao diện phải giúp người dùng nắm bắt nhanh các thông tin trọng yếu như trạng thái an toàn của mũ bảo hiểm, cảnh báo nguy

hiểm, lịch sử sự kiện và vị trí hiện tại. Đồng thời, các thao tác điều khiển thiết bị, cấu hình cảm biến hoặc quản lý thông tin cá nhân cũng cần được bố trí hợp lý, dễ tiếp cận ngay cả khi người dùng không có nền tảng kỹ thuật chuyên sâu.

Về mặt thiết kế kỹ thuật, ứng dụng sử dụng kiến trúc client-server dựa trên REST API để giao tiếp với Raspberry Pi và ESP32, cho phép ứng dụng đồng bộ dữ liệu liên tục và kiểm soát được các trạng thái phần cứng từ xa. Tốc độ phản hồi và độ ổn định được ưu tiên hàng đầu để đảm bảo rằng các lệnh điều khiển (như bật/tắt relay, phát cảnh báo) cụ thể thực thi với độ trễ tối thiểu, dưới 50 ms trong mạng nội bộ.

Ngoài ra, do đối tượng người dùng có thể bao gồm nhiều nhóm tuổi và mức độ am hiểu công nghệ khác nhau, giao diện được thiết kế theo hướng tối giản, các biểu tượng trực quan kết hợp với thông báo rõ ràng giúp nâng cao khả năng sử dụng trong mọi tình huống. Các biểu đồ trực quan như Pie Chart, Bar Chart và biểu đồ đường thời gian cũng được tích hợp nhằm giúp người dùng dễ dàng theo dõi và đánh giá các hành vi an toàn trong quá trình vận hành phương tiện.

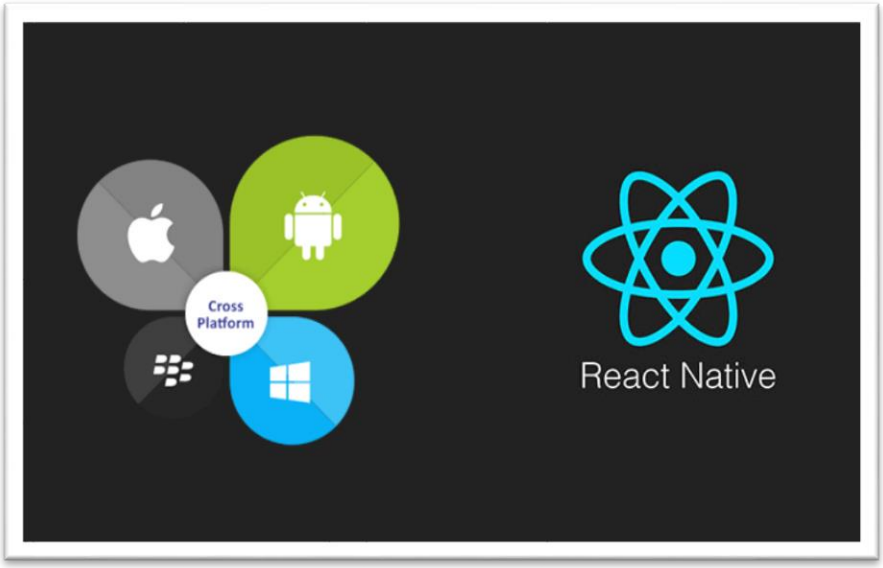
Việc xây dựng giao diện trên nền tảng React Native giúp ứng dụng đạt được tính đa nền tảng, dễ dàng triển khai trên cả hệ điều hành Android và iOS, đồng thời tạo điều kiện thuận lợi cho việc nâng cấp, bảo trì hoặc mở rộng chức năng trong các phiên bản tương lai.

3.4.2 Kiến trúc ứng dụng di động

Ứng dụng di động của hệ thống mũ bảo hiểm IoT được phát triển trên nền tảng React Native có giao diện như Hình 3-17, cho phép xây dựng các ứng dụng đa nền tảng cho cả Android và iOS chỉ từ một mã nguồn duy nhất. Với đặc thù là một hệ thống giám sát an toàn giao thông thời gian thực, kiến trúc của ứng dụng được thiết kế theo hướng phân tầng, giúp đảm bảo khả năng phản hồi nhanh, đồng bộ hóa dữ liệu liên tục và duy trì sự ổn định khi vận hành trong môi trường mạng cục bộ.

Trong tầng giao diện người dùng, các màn hình chức năng được xây dựng nhằm cung cấp trải nghiệm trực quan và thân thiện. Người dùng có thể dễ dàng theo dõi trạng thái của mũ bảo hiểm, nhận thông báo về các tình huống nguy hiểm và thực

hiện các thao tác điều khiển thiết bị. Giao diện được tối ưu để hiển thị các thông tin quan trọng một cách rõ ràng, đồng thời hỗ trợ các biểu đồ trực quan giúp người dùng nhanh chóng nắm bắt tình trạng an toàn của thiết bị và hành vi lái xe.



Hình 3-17 Nền tảng React Native

Phía dưới lớp giao diện là tầng xử lý logic ứng dụng, nơi triển khai các cơ chế kiểm soát cảm biến, điều khiển relay khởi động, phát cảnh báo âm thanh và đồng bộ dữ liệu với hệ thống. Tất cả các hành động từ phía người dùng đều được xử lý tại tầng này, sau đó được chuyển thành các yêu cầu HTTP REST API gửi tới Raspberry Pi. Việc xây dựng tầng logic riêng biệt giúp ứng dụng duy trì sự linh hoạt, dễ bảo trì và thuận tiện cho việc mở rộng các chức năng mới trong tương lai.

Tầng giao tiếp dữ liệu đóng vai trò cầu nối giữa ứng dụng và phần cứng của hệ thống. Thông qua các API REST do Raspberry Pi cung cấp, ứng dụng có thể gửi các lệnh điều khiển như bật/tắt cảm biến, kích hoạt relay hoặc buzzer, đồng thời truy vấn trạng thái hiện tại của mũ bảo hiểm. Quá trình trao đổi dữ liệu giữa ứng dụng và Raspberry Pi được thực hiện qua mạng Wi-Fi cục bộ với độ trễ trung bình dưới 50 ms, đảm bảo các phản hồi từ hệ thống được cập nhật nhanh chóng. Dữ liệu trao đổi sử dụng định dạng JSON giúp tăng tính tương thích và dễ dàng mở rộng khi cần.

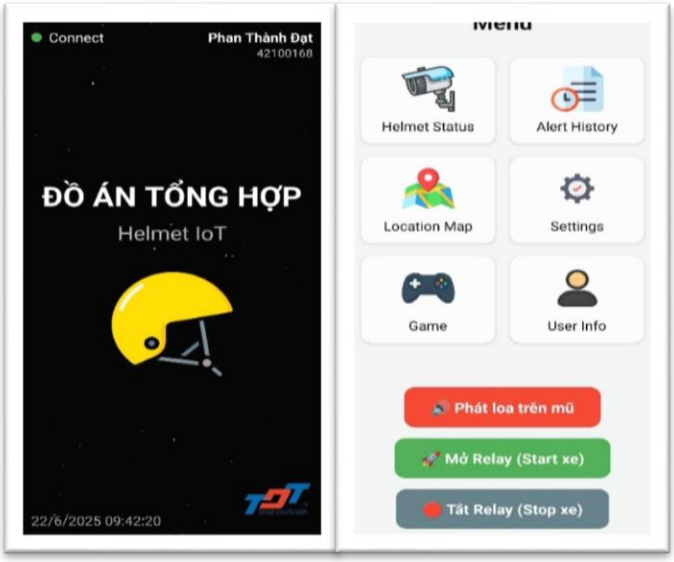
Song song với việc giao tiếp trực tiếp với Raspberry Pi, ứng dụng còn tích hợp khả năng đồng bộ hóa dữ liệu với Google Sheets thông qua các API xây dựng trên

nền tảng Google Apps Script. Các thông tin quan trọng như trạng thái an toàn, lịch sử cảnh báo và vị trí GPS của mũ bảo hiểm được cập nhật định kỳ lên Google Sheets, giúp người dùng dễ dàng theo dõi, lưu trữ và phân tích thông tin lâu dài.

Ngoài ra, để đảm bảo tính ổn định của kết nối, ứng dụng được thiết kế với cơ chế tự động cập nhật địa chỉ IP của Raspberry Pi từ Google Sheets. Bằng cách này, ứng dụng luôn có thể xác định chính xác địa chỉ của Raspberry Pi ngay cả khi có sự thay đổi mạng nội bộ. Cùng với đó, cơ chế polling liên tục với tần suất 3–5 giây cho phép ứng dụng duy trì trạng thái đồng bộ với phần cứng, giúp người dùng luôn nhận được các thông tin mới nhất từ hệ thống.

3.4.3 Các thành phần giao diện chính

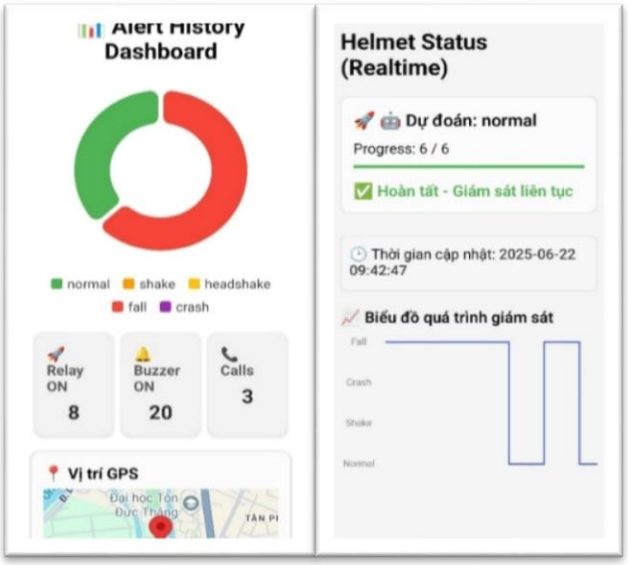
Ứng dụng di động của hệ thống mũ bảo hiểm IoT được thiết kế gồm nhiều thành phần giao diện chính, mỗi thành phần phục vụ một chức năng cụ thể nhằm giúp người dùng dễ dàng theo dõi, kiểm soát và tương tác với hệ thống. Các thành phần này được tổ chức dưới dạng các màn hình chức năng, cho phép chuyển đổi linh hoạt thông qua menu điều hướng chính của ứng dụng.



Hình 3-18 Giao diện Dashboard và Menu

Giao diện Dashboard được minh họa ở Hình 3-18 bên trái đóng vai trò là trang tổng quan khi người dùng khởi động ứng dụng. Tại đây, các thông tin cơ bản như tên người dùng, mã số sinh viên, ngày giờ hiện tại và trạng thái kết nối của hệ thống được

hiển thị một cách rõ ràng và trực quan. Từ Dashboard, người dùng có thể truy cập nhanh vào Menu chính để điều hướng đến các chức năng chuyên sâu hơn.



Hình 3-19 Giao diện Alert History và Helmet Status

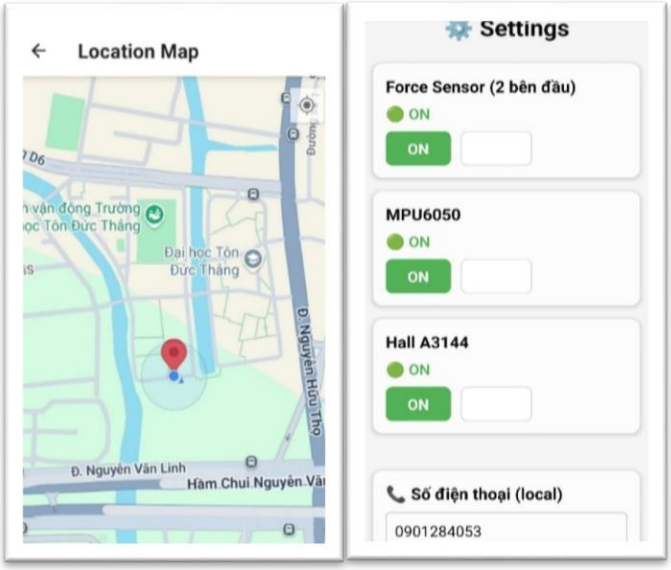
Giao diện Menu được thể hiện ở hình 3-18 bên phải cung cấp các lối tắt trực tiếp đến các tính năng quan trọng của hệ thống, bao gồm: giám sát trạng thái mũ bảo hiểm, xem lịch sử cảnh báo, theo dõi vị trí, cấu hình cảm biến, kiểm tra cảnh báo video và quản lý thông tin cá nhân. Thiết kế của Menu hướng tới sự đơn giản, dễ hiểu, giúp người dùng có thể thao tác thuận tiện ngay cả khi đang sử dụng ngoài thực địa.

Giao diện Helmet Status được minh họa qua Hình 3-19 bên phải cho phép người dùng theo dõi trạng thái vận hành hiện tại của mũ bảo hiểm theo thời gian thực. Các thông tin về tiến trình kiểm tra an toàn, kết quả nhận diện hình ảnh, trạng thái cảm biến, cũng như biểu đồ diễn biến hành vi từ cảm biến MPU6050 được trình bày một cách sinh động và dễ hiểu. Nhờ đó, người dùng có thể nhận biết ngay các tình huống nguy hiểm hoặc bất thường.

Giao diện Alert History được mình họa ở hình 3-19 bên trái giúp người dùng xem lại toàn bộ lịch sử các cảnh báo đã xảy ra. Dữ liệu được trực quan hóa dưới dạng biểu đồ tròn và biểu đồ cột, thể hiện tần suất các loại hành vi nguy hiểm như rung động, va chạm hoặc té ngã. Ngoài ra, các thông tin liên quan đến số lần kích hoạt

relay, buzzer hoặc cuộc gọi khẩn cấp cũng được thống kê rõ ràng, hỗ trợ người dùng đánh giá mức độ an toàn trong quá trình vận hành.

Giao diện Location Map được mình họa qua Hình 3-20 bên trái cung cấp chức năng theo dõi vị trí của mũ bảo hiểm hoặc người điều khiển phương tiện. Thông qua GPS tích hợp, vị trí hiện tại của thiết bị được hiển thị trực tiếp trên bản đồ số, giúp người dùng và người thân có thể biết được vị trí chính xác khi có sự cố xảy ra.



Hình 3-20 Giao diện Location Map và Settings

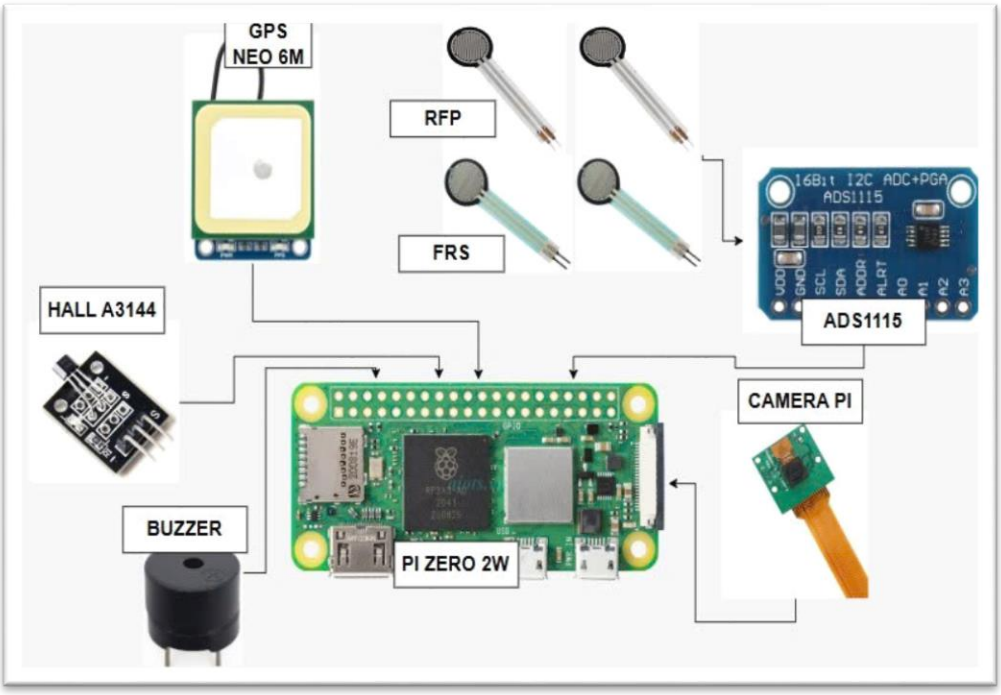
Giao diện Settings được thể hiện ở Hình 3-20 bên phải cho phép người dùng chủ động cấu hình các thành phần cảm biến như Force Sensor, MPU6050 hoặc cảm biến Hall, cũng như cập nhật và lưu số điện thoại liên hệ khẩn cấp. Ngoài ra, người dùng có thể chủ động gửi yêu cầu kích hoạt cuộc gọi đến người thân thông qua chức năng "Gọi chủ động" được tích hợp trong giao diện này.

Giao diện Game đang trong quá trình phát triển chưa hoàn thiện nhưng đây là nơi để tích hợp các tính năng tương lai mà người dùng mong muốn hoặc có thể tạo nên một trò chơi tương tác được với mũ bảo hiểm IoT bằng các chức năng thu thập dữ liệu của cảm biến. Cuối cùng là giao diện User Info cho phép người dùng quản lý thông tin cá nhân, bao gồm các thông tin cơ bản như họ tên, mã số sinh viên, lớp học, chuyên ngành và số điện thoại.

CHƯƠNG 4. KẾT QUẢ THI CÔNG VÀ THỰC NGHIỆM

4.1 Kết quả thiết kế hệ thống

Sau quá trình nghiên cứu, thiết kế và triển khai, hệ thống mũ bảo hiểm thông minh IoT đã được hoàn thiện, bao gồm cả phần cứng và phần mềm. Toàn bộ hệ thống được triển khai theo đúng các yêu cầu kỹ thuật đặt ra trong đề tài, đáp ứng đầy đủ về chức năng, tính ổn định, khả năng giao tiếp giữa các thành phần và khả năng tích hợp thực tế trên mũ bảo hiểm. Kết quả thiết kế thể hiện qua việc lắp ráp đầy đủ các linh kiện phần cứng, lập trình và vận hành phần mềm trên các thiết bị, đồng thời tích hợp thành công giao tiếp giữa các khối chức năng. Về phần cứng, hệ thống được chia thành hai nhóm chính: hệ thống 1 sử dụng Raspberry Pi Zero 2W làm bộ xử lý trung tâm và hệ thống 2 sử dụng ESP32 làm vi điều khiển chấp hành.

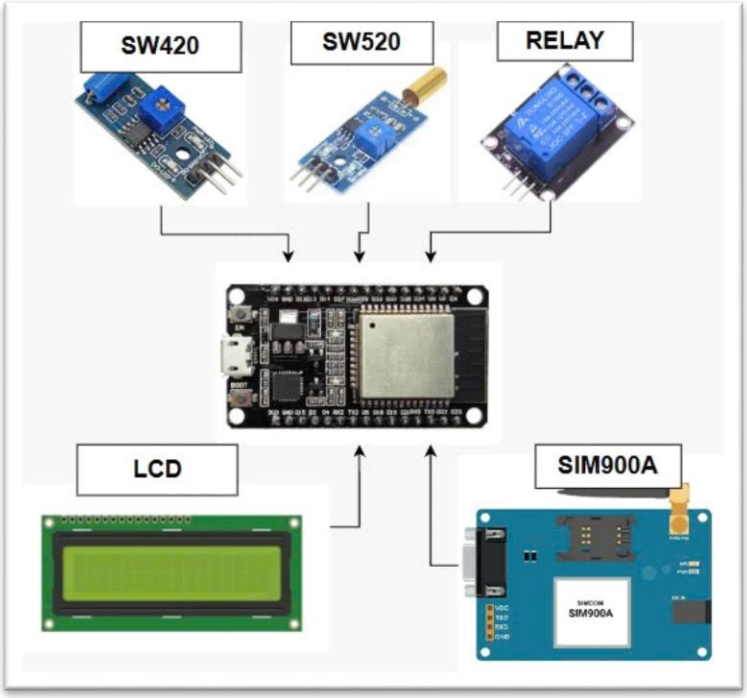


Hình 4-1 Linh kiện sử dụng ở hệ thống 1

Hệ thống 1 có nhiệm vụ thu thập dữ liệu từ các cảm biến MPU6050, ADS1115 (đọc tín hiệu từ 2 cảm biến RFP602 và 2 cảm biến FSR402), module GPS NEO6M,

Camera Pi và phát cảnh báo qua Buzzer. Các kết nối phần cứng được thiết lập thông qua các giao tiếp I2C, UART và GPIO, đảm bảo khả năng truyền nhận dữ liệu thời gian thực. Hình 4-1 minh họa các linh kiện được sử dụng trong hệ thống 1.

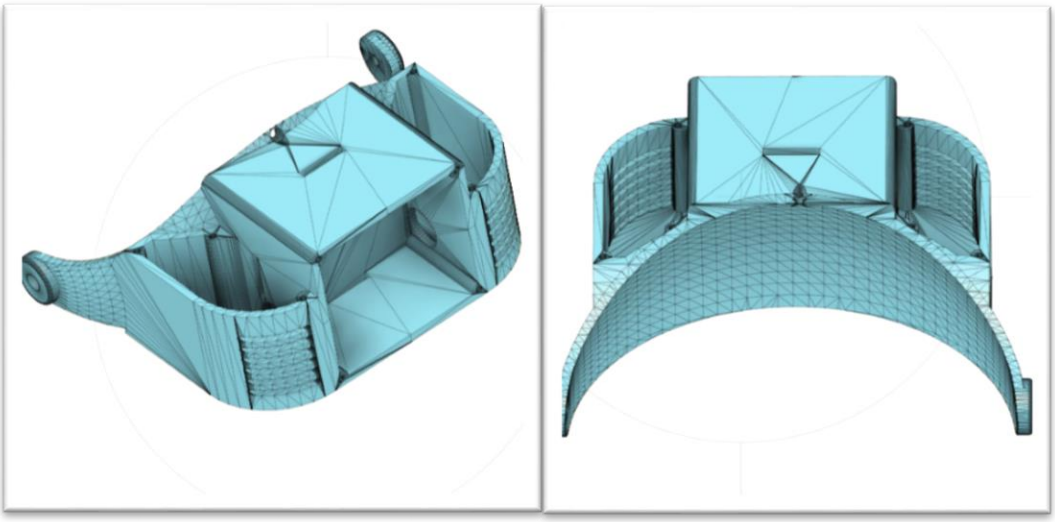
Song song với đó, hệ thống 2 thực hiện xử lý tín hiệu từ cảm biến rung SW420, cảm biến nghiêng SW520, điều khiển relay, phát cảnh báo âm thanh và gửi tin nhắn SMS thông qua module SIM900A. Ngoài ra, ESP32 còn có khả năng hiển thị trạng thái thông qua màn hình LCD và nhận lệnh điều khiển từ Raspberry Pi Zero 2W. Hình 4-2 trình bày các linh kiện được sử dụng trong hệ thống 2 của mô hình.



Hình 4-2 Linh kiện sử dụng ở hệ thống 2

Về phần mềm, chương trình trên Raspberry Pi được phát triển bằng ngôn ngữ Python, tích hợp các thư viện giao tiếp với cảm biến, xử lý hình ảnh và nhận diện trạng thái đội mũ. Chương trình trên ESP32 được lập trình bằng nền tảng PlatformIO trên phần mềm Visual Studio Code, đảm nhiệm các chức năng điều khiển thiết bị ngoại vi và thực hiện các giao tiếp REST API với Raspberry Pi. Bên cạnh đó, ứng dụng di động được xây dựng trên nền tảng React Native, đóng vai trò là giao diện người dùng chính, cho phép giám sát trạng thái hệ thống theo thời gian thực, nhận cảnh báo, điều khiển từ xa và đồng bộ dữ liệu với Google Sheets.

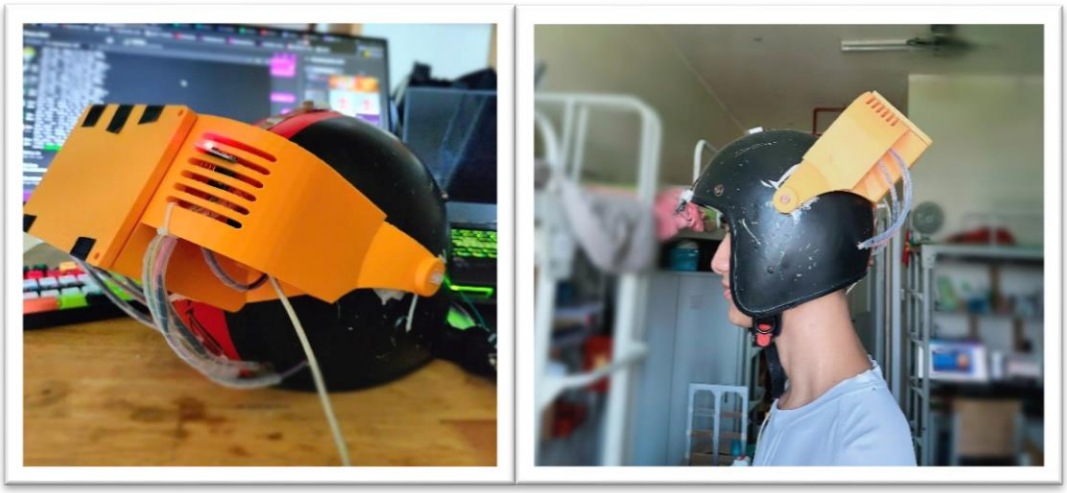
Ngoài các thành phần linh kiện điện tử, hệ thống còn được thiết kế bổ sung hộp đựng chuyên dụng nhằm bảo vệ và cố định các module phần cứng khi tích hợp vào mũ bảo hiểm. Hộp đựng được thiết kế bằng phần mềm SolidWorks, đảm bảo độ chính xác cao về kích thước và tối ưu hóa không gian lắp đặt. Việc sử dụng thiết kế 3D giúp dễ dàng kiểm tra, hiệu chỉnh cấu trúc trước khi sản xuất thực tế. Sau khi thiết kế hoàn chỉnh, hộp đựng được gia công bằng công nghệ in 3D với chất liệu nhựa chịu va đập, phù hợp với các điều kiện sử dụng thực tế trên phương tiện giao thông. Thiết kế hộp không chỉ giúp bảo vệ các linh kiện khỏi các tác động cơ học bên ngoài mà còn đảm bảo khả năng thoát nhiệt và bố trí dây dẫn một cách gọn gàng, thuận tiện cho quá trình bảo trì và nâng cấp sau này. Hình 4-3 thể hiện mô hình thiết kế 3D của hộp đựng linh kiện trong hệ thống.



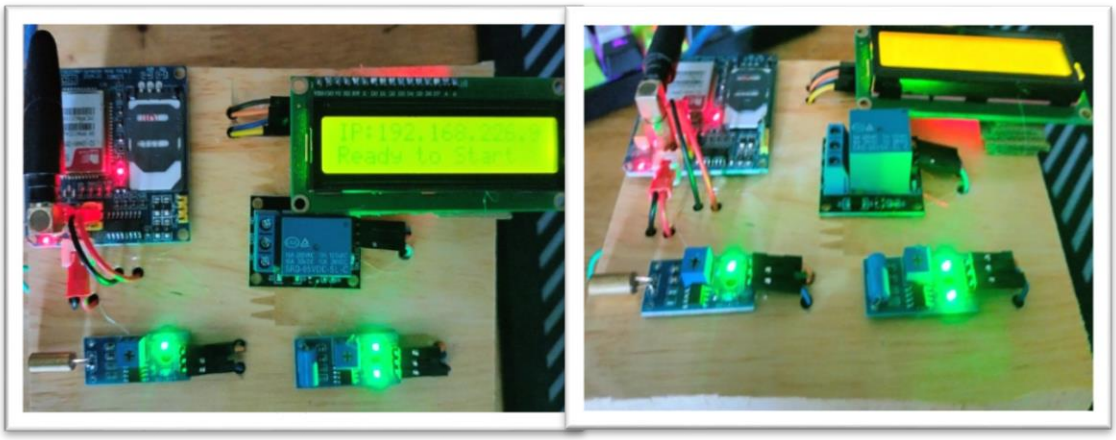
Hình 4-3 Mô hình 3D của hộp đựng linh kiện

Hệ thống sau khi lắp ráp hoàn chỉnh đã được tích hợp trực tiếp lên mũ bảo hiểm thực tế thông qua hộp đựng thiết kế bằng phần mềm SolidWorks và gia công bằng công nghệ in 3D. Thiết kế hộp đựng được tối ưu để ôm sát phần sau của mũ, đảm bảo sự gọn nhẹ, giảm lực cản không khí khi di chuyển và đồng thời bảo vệ an toàn cho các module điện tử bên trong. Các dây dẫn tín hiệu và dây nguồn được bố trí hợp lý, sử dụng ống gen co và dây rút để cố định chắc chắn dọc theo thành mũ, giúp giảm thiểu hiện tượng rung lắc hoặc đứt gãy trong quá trình sử dụng. Vị trí lắp đặt được tính toán nhằm đảm bảo không gây cản trở cho người dùng khi đội mũ, đồng

thời không làm tăng đáng kể trọng lượng mũ, giữ được sự thoải mái khi vận hành. Qua quá trình lắp ráp thực tế và kiểm tra thử nghiệm, hệ thống cho thấy khả năng vận hành ổn định, các thành phần được cố định chắc chắn, đảm bảo khả năng chống rung và chịu va đập khi di chuyển trên đường. Hình 4-4 trình bày hình ảnh lắp ráp thực tế của hệ thống mũ bảo hiểm thông minh IoT.



Hình 4-4 Mô hình nón bảo hiểm hoàn thiện



Hình 4-5 Mô hình giả lập hệ thống trên phương tiện

Mô hình phần cứng được thiết kế và lắp ráp trên một bề mặt phẳng nhằm mô phỏng vị trí gắn thực tế trên thân phương tiện. Cấu trúc gồm các thành phần chính như module SIM900A, vi điều khiển ESP32, relay điều khiển nguồn, màn hình hiển thị LCD I2C và các cảm biến SW420, SW520. Mạch được bố trí gọn gàng, đảm bảo khả năng quan sát và kết nối dễ dàng khi lắp đặt hoặc kiểm tra. Trong mô hình thực tế ở Hình 4-5, module SIM900A (bên trái) thực hiện chức năng liên lạc khẩn cấp

thông qua cuộc gọi. Màn hình LCD I2C (bên phải) hiển thị địa chỉ IP và trạng thái hệ thống. Phía dưới là relay khởi động và hai cảm biến rung: SW420 phát hiện rung động theo phương thẳng đứng, SW520 phản ứng với dao động nghiêng. Hai cảm biến này hoạt động song song để phát hiện các tình huống như va chạm, rung lắc mạnh hoặc trượt ngã kéo dài. Nếu cả hai cảm biến kích hoạt liên tục trong thời gian định ngưỡng (≥ 3 giây), hệ thống sẽ xử lý sự kiện và đưa ra quyết định cảnh báo gọi điện và gửi email.

Kết luận, qua quá trình thiết kế, lắp ráp và thử nghiệm ban đầu, hệ thống mũ bảo hiểm thông minh IoT đã được hoàn thiện đầy đủ cả về phần cứng lẫn phần mềm. Các linh kiện được bố trí hợp lý, đảm bảo an toàn khi tích hợp lên mũ bảo hiểm thực tế. Phần mềm vận hành ổn định, giao tiếp giữa các thành phần diễn ra chính xác và có độ trễ thấp. Toàn bộ hệ thống đã sẵn sàng bước vào giai đoạn kiểm thử thực tế nhằm đánh giá hiệu quả hoạt động và khả năng áp dụng trong các tình huống phù hợp với mục tiêu thực nghiệm mô hình học thuật.

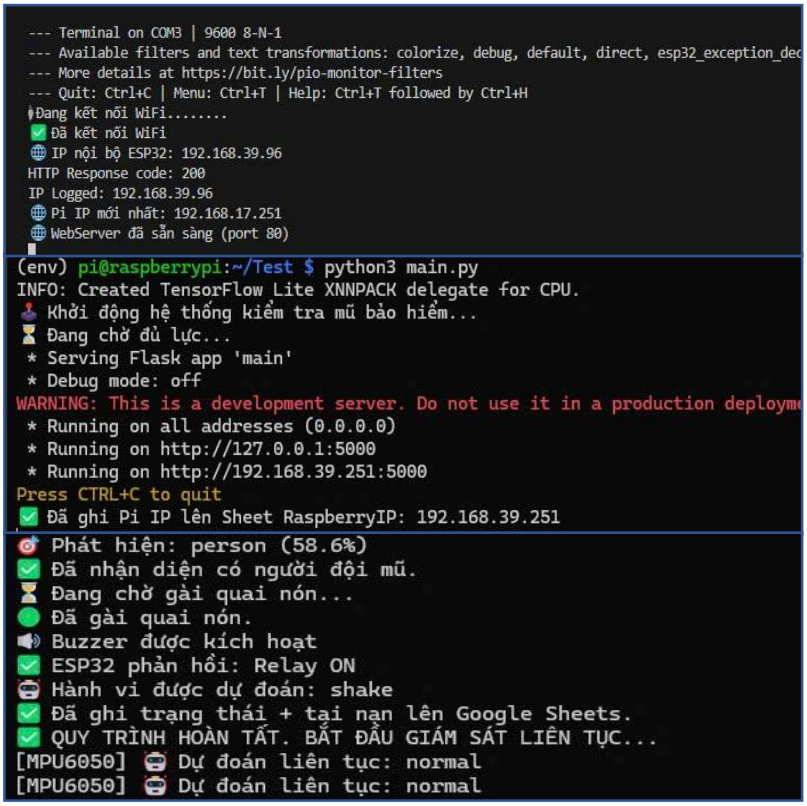
4.2 Kiểm thử hệ thống và vận hành thực tế

Sau khi hoàn thiện phần cứng và phần mềm, hệ thống mũ bảo hiểm thông minh IoT đã được tiến hành kiểm thử thực tế nhằm đánh giá khả năng vận hành, độ ổn định và mức độ đáp ứng các yêu cầu kỹ thuật đề ra. Quá trình kiểm thử được thực hiện trên mũ bảo hiểm thực tế đã lắp đặt đầy đủ các linh kiện, tích hợp với hộp đựng 3D và kết nối với ứng dụng di động. Các thử nghiệm được tiến hành trong các điều kiện vận hành thực tế trên đường phố với nhiều tình huống khác nhau để kiểm tra khả năng đáp ứng của hệ thống.

Quá trình kiểm thử được chia thành hai giai đoạn chính. Ở giai đoạn đầu, kiểm thử khả năng khởi động hệ thống: khi người dùng đội mũ, hệ thống lần lượt kiểm tra các điều kiện an toàn bao gồm: lực cảm biến FSR đạt ngưỡng yêu cầu, nhận diện hình ảnh người đội mũ có độ chính xác trên 50% qua mô hình MobileNet SSD, và trạng thái gài quai mũ đúng cách thông qua cảm biến Hall. Khi hội đủ các điều kiện, Raspberry Pi sẽ gửi lệnh HTTP cho ESP32 để kích hoạt relay cho phép xe khởi động.

Qua quá trình thử nghiệm, hệ thống vận hành ổn định với độ trễ trung bình từ khi đội mũ đến khi cho phép khởi động xe là khoảng 5–7 giây, đảm bảo tính tiện lợi và an toàn cho người sử dụng.

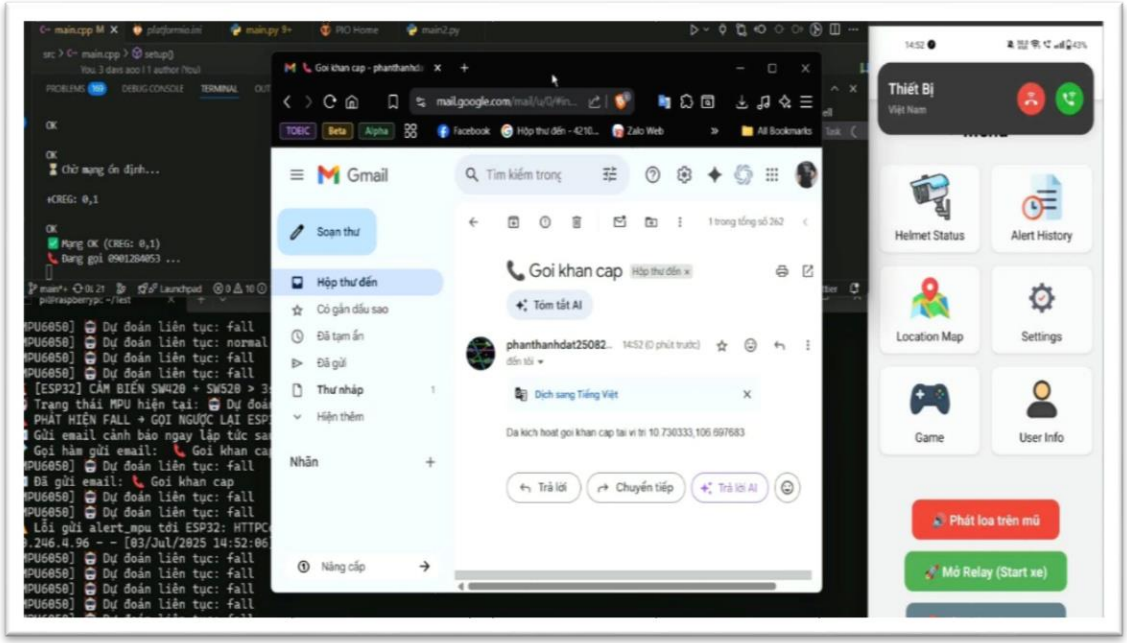
Trong giai đoạn tiếp theo, hệ thống chuyển sang trạng thái giám sát liên tục trong quá trình vận hành xe. Các cảm biến MPU6050, SW420 và SW520 hoạt động đồng bộ, ghi nhận liên tục các hành vi của người điều khiển. Khi phát hiện các hành vi nguy hiểm hoặc tình huống va chạm, hệ thống phát cảnh báo tức thời qua buzzer và gửi thông báo về ứng dụng di động. Đặc biệt, khi mô phỏng các tình huống va chạm nghiêm trọng, hệ thống đã tự động thực hiện cuộc gọi khẩn cấp thông qua module SIM900A với thời gian phản hồi trung bình dưới 10 giây kể từ khi phát hiện sự kiện.



Hình 4-6 Giai đoạn chạy thử hệ thống

Quá trình kiểm thử thực tế được ghi nhận qua các nhật ký hoạt động của hệ thống. Hình 4-6 minh họa sơ bộ quá trình khởi động và vận hành thực tế của hệ thống mũ bảo hiểm thông minh IoT. Trong đó thể hiện các bước: Raspberry Pi khởi động

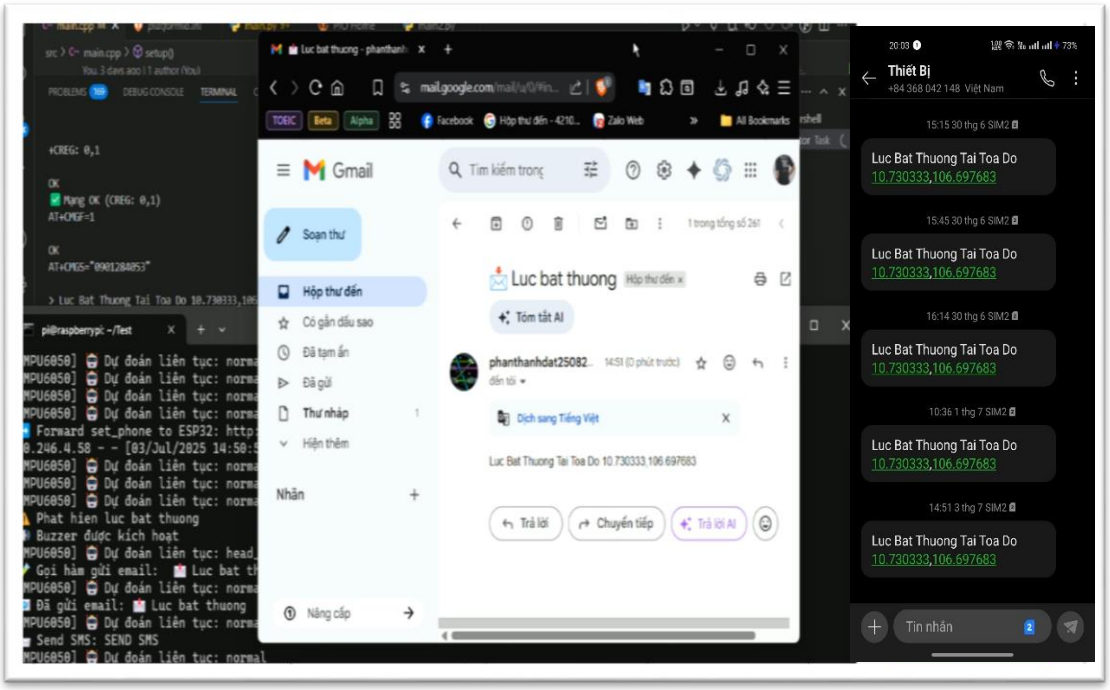
dịch vụ Flask, kiểm tra cảm biến lực và nhận diện người đội mũ bằng mô hình học sâu MobileNet SSD, kiểm tra trạng thái gài quai mũ, ghi nhận IP thiết bị lên Google Sheets, điều khiển relay khởi động xe thông qua ESP32. Bên cạnh đó, ESP32 vận hành WebServer, kết nối Wi-Fi, phản hồi các lệnh điều khiển từ Raspberry Pi. Sau khi hoàn tất các bước kiểm tra an toàn, hệ thống chuyển sang chế độ giám sát liên tục, sử dụng cảm biến MPU6050 và các cảm biến phụ trợ để nhận diện hành vi vận hành, đồng thời ghi nhận và phản hồi kịp thời các tình huống nguy hiểm. Nhật ký hoạt động cho thấy các thành phần phần cứng và phần mềm phối hợp ổn định, đáp ứng đúng yêu cầu kỹ thuật đề ra.



Hình 4-7 Kiểm thử chức năng giám sát

Để kiểm chứng khả năng phản ứng trong tình huống thực tế, hệ thống đã được thử nghiệm với kịch bản mô phỏng tai nạn nhằm đánh giá tính chính xác và độ tin cậy của cơ chế cảnh báo khẩn cấp. Hình 4-7 ghi lại quá trình vận hành của hệ thống trong một tình huống mô phỏng tai nạn thực tế. Tại thời điểm này, cảm biến MPU6050 liên tục ghi nhận trạng thái "fall", đồng thời hai cảm biến rung SW420 và SW520 cùng phát hiện rung động kéo dài trên 3 giây. ESP32 ngay lập tức gửi tín hiệu cảnh báo về Raspberry Pi thông qua API. Sau khi xác nhận điều kiện tai nạn thực sự xảy ra, hệ thống tiến hành đồng thời hai tác vụ khẩn cấp, một là gửi thư điện tử cảnh

báo đến địa chỉ được cấu hình sẵn, trong đó đính kèm tọa độ GPS tại thời điểm sự cố (10.730333, 106.697683) và hai là kích hoạt cuộc gọi tự động đến số điện thoại người thân thông qua module SIM900A. Trong tình huống thực nghiệm được minh họa tại Hình 4-8, hệ thống ghi nhận một kịch bản giám sát trong đó xuất hiện lực tác động lên hai cảm biến được bố trí ở hai bên vành phía trong của mũ, khu vực không tiếp xúc trực tiếp với đầu người đội. Ngay lập tức, hệ thống xác định đây là dấu hiệu bất thường trong quá trình giám sát và nhanh chóng phát tín hiệu điều khiển buzzer phát ra âm thanh cảnh báo tại chỗ. Đồng thời, hệ thống thực hiện quy trình ghi nhận sự kiện và gửi tin nhắn SMS đến số điện thoại đã được thiết lập trước. Để tăng độ tin cậy, một email dự phòng với nội dung cảnh báo tương tự cũng được gửi đến địa chỉ định sẵn, nhằm đảm bảo thông tin vẫn được chuyển đi trong trường hợp gặp sự cố như mất tín hiệu hoặc lỗi gửi tin nhắn.



Hình 4-8 Kiểm thử chức năng giám sát bất thường

Qua quá trình kiểm thử thực tế, hệ thống đã thể hiện khả năng phản ứng nhanh chóng và chính xác trước các tình huống mô phỏng tai nạn. Các bước xử lý được hiển thị rõ ràng trong nhật ký dòng lệnh, cho thấy quy trình vận hành tuân thủ đúng theo thiết kế ban đầu. Song song đó, giao diện ứng dụng di động vẫn duy trì hoạt động ổn

định, hỗ trợ người dùng theo dõi trạng thái hệ thống theo thời gian thực và điều khiển từ xa thông qua các chức năng như phát loa cảnh báo, khởi động hoặc tắt động cơ xe, cũng như giám sát vị trí GPS của người điều khiển. Kết quả kiểm thử xác nhận sự phối hợp nhịp nhàng và hiệu quả giữa các thành phần phần cứng, phần mềm và giao diện người dùng. Điều này không chỉ đảm bảo độ tin cậy mà còn nâng cao tính kịp thời của hệ thống trong việc xử lý và phản hồi các tình huống khẩn cấp, góp phần tăng cường an toàn cho người tham gia giao thông.

4.3 Đánh giá hiệu suất hệ thống

Hiệu suất của hệ thống được đánh giá thông qua các chỉ số về tốc độ xử lý, độ chính xác nhận diện, độ trễ phản hồi và thời lượng hoạt động, được trình bày trong Bảng 4-1. Qua kết quả đo đạc, hệ thống cho thấy khả năng xử lý ổn định, đáp ứng yêu cầu kỹ thuật đề ra. Mô hình nhận diện hình ảnh hoạt động tốt ngay trên nền tảng phần cứng hạn chế, với tốc độ xử lý phù hợp cho ứng dụng thời gian thực. Giao tiếp giữa Raspberry Pi và ESP32 có độ trễ rất thấp, đảm bảo khả năng phản hồi nhanh với các tình huống bất thường

Bảng 4-1 Hiệu suất quan sát được của hệ thống

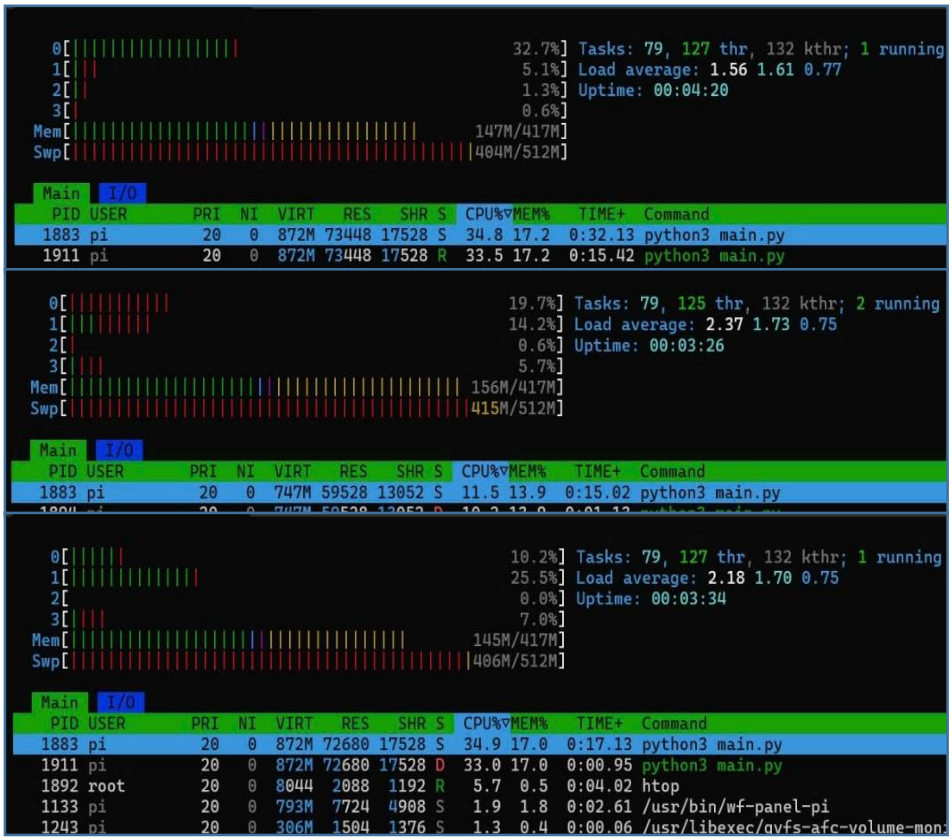
Thông số đánh giá	Giá trị quan sát/đo được
Thời gian khởi động Raspberry Pi và Flask server	~10–15 giây
Thời gian kiểm tra an toàn (FSR + Hall + nhận diện ảnh)	~5–10 giây
Tốc độ xử lý MobileNet SSD trên Pi Zero 2W	~2.5–3 FPS
Độ chính xác nhận diện ảnh (ánh sáng tốt)	≥ 60%
Độ chính xác nhận diện ảnh (ánh sáng yếu)	40-60%

Ngưỡng nhận diện đạt yêu cầu	≥ 50%
Độ trễ giao tiếp HTTP giữa Pi ↔ ESP32 (LAN)	< 50 ms
Thời gian phản hồi buzzer khi phát hiện fall/crash	< 1 giây
Thời gian từ phát hiện sự kiện đến thực hiện cuộc gọi/gửi sms/gửi email	< 10 giây
Thời gian hoạt động liên tục (battery 5000 mAh)	~2.5–3 giờ

Qua các chỉ số trên, có thể nhận thấy hiệu suất của hệ thống đáp ứng tốt các yêu cầu kỹ thuật đề ra. Quá trình kiểm tra điều kiện an toàn trước khi khởi động xe diễn ra nhanh, thuận tiện cho người dùng. Mô hình nhận diện ảnh MobileNet SSD cho kết quả chính xác và ổn định trên nền tảng phần cứng hạn chế như Raspberry Pi Zero 2W. Độ trễ trong giao tiếp giữa Raspberry Pi và ESP32 ở mức rất thấp, đảm bảo khả năng phản hồi kịp thời khi xảy ra các tình huống nguy hiểm. Các hành động cảnh báo và gọi điện khẩn cấp đều được thực hiện với tốc độ phù hợp, có khả năng hỗ trợ hiệu quả trong các tình huống thực tế. Thời lượng hoạt động khi sử dụng nguồn pin di động thương mại đáp ứng tốt nhu cầu sử dụng liên tục trong các chuyến đi thông thường.

Hiệu suất sử dụng CPU (%) và RAM (MB) của Raspberry Pi Zero 2W được ghi nhận thông qua công cụ htop và quá trình theo dõi thực tế khi chạy các tác vụ chính của hệ thống. Kết quả ghi nhận được thể hiện rõ qua hình 4-9. Cụ thể, khi hệ thống ở chế độ chờ khởi động (Idle), mức sử dụng CPU duy trì ở mức thấp khoảng 8%, RAM sử dụng ổn định quanh mức 100 MB. Trong các giai đoạn xử lý yêu cầu cao hơn như nhận diện hình ảnh và giám sát liên tục, mức sử dụng CPU dao động từ 35% đến 55%, trong khi RAM sử dụng ở mức từ 140 MB đến 160 MB. Khi thực hiện các tác vụ cảnh báo và thực hiện cuộc gọi thông qua SIM900A, mức sử dụng tài

nguyên tăng cao hơn trong thời gian ngắn nhưng vẫn đảm bảo ổn định cho toàn hệ thống.

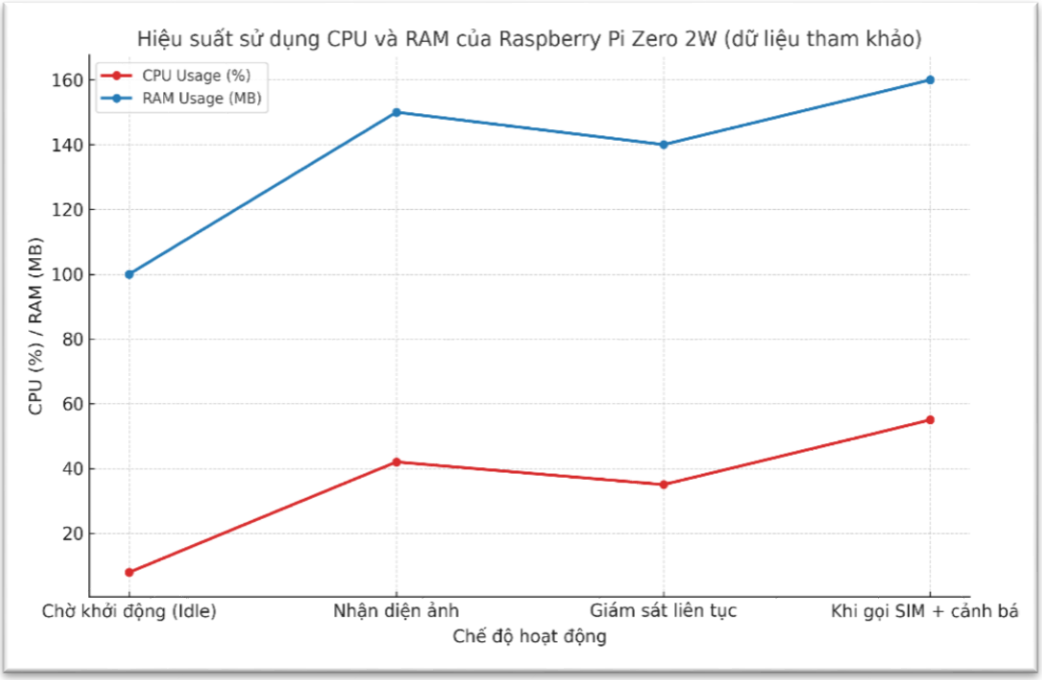


Hình 4-9 Hiệu suất hoạt động của CPU&RAM của Raspberry Pi Zero 2W

Giá trị trung bình của mức sử dụng CPU và RAM trên Raspberry Pi Zero 2W được thống kê theo từng trạng thái hoạt động chính của hệ thống, bao gồm: chế độ chờ khởi động, xử lý nhận diện hình ảnh, giám sát liên tục và thực hiện cảnh báo. Hình 4-9 minh họa mức sử dụng tài nguyên trung bình trong từng trạng thái, cho thấy hệ thống hoạt động ổn định với mức tiêu thụ phần cứng không vượt quá giới hạn của thiết bị. Trong khi đó, Hình 4-10 thể hiện diễn biến thay đổi của mức tiêu thụ CPU và RAM theo thời gian trong quá trình vận hành thực tế. Biểu đồ phản ánh rõ sự dao động tài nguyên tương ứng với các giai đoạn xử lý khác nhau, từ đó cho thấy tính ổn định và khả năng thích ứng của hệ thống với các yêu cầu xử lý thời gian thực.

Đối với ESP32, mức sử dụng RAM heap dao động từ 40 KB khi ở chế độ chờ đến khoảng 50 KB khi đồng thời nhận lệnh HTTP từ Raspberry Pi và thực hiện các

tác vụ như kích hoạt cảnh báo hoặc cuộc gọi khẩn cấp. Mức sử dụng bộ nhớ của ESP32 phù hợp với giới hạn phần cứng và đảm bảo an toàn cho hoạt động lâu dài.



Hình 4-10 Biểu đồ sử dụng CPU và RAM của Raspberry Pi Zero 2W

Các kết quả trên cho thấy hệ thống sử dụng hợp lý tài nguyên phần cứng, đảm bảo khả năng vận hành ổn định trong suốt quá trình hoạt động. Việc khai thác tài nguyên không vượt quá ngưỡng cho phép của các thiết bị nhúng như Raspberry Pi Zero 2W và ESP32, đồng thời vẫn đảm bảo đáp ứng tốt yêu cầu về thời gian thực cho các tác vụ xử lý, nhận diện và cảnh báo. Điều này cũng mở ra tiềm năng cho việc mở rộng hoặc tích hợp thêm các tính năng mới trong các phiên bản tiếp theo của hệ thống.

4.4 Một số hạn chế trong quá trình thi công

Trong quá trình nghiên cứu và triển khai thực tế hệ thống mũ bảo hiểm thông minh IoT, bên cạnh những kết quả đạt được, nhóm thực hiện cũng ghi nhận một số hạn chế và thách thức nhất định.

Trước hết, về phần cứng, việc lựa chọn Raspberry Pi Zero 2W làm bộ xử lý trung tâm mang lại ưu điểm về kích thước nhỏ gọn và tiết kiệm năng lượng. Tuy nhiên, với cấu hình phần cứng hạn chế (CPU ARM Cortex-A53, RAM 512 MB), hiệu

suất xử lý của mô hình học sâu MobileNet SSD chỉ đạt khoảng 2.5–3 FPS. Điều này chưa đủ để đáp ứng yêu cầu nhận diện hình ảnh ở tốc độ cao khi xe di chuyển nhanh hoặc trong các môi trường có điều kiện ánh sáng phức tạp.

Trong các thử nghiệm thực tế, nhóm nhận thấy độ chính xác của thuật toán nhận diện ảnh bị ảnh hưởng đáng kể bởi điều kiện môi trường, đặc biệt là ánh sáng yếu hoặc ban đêm. Độ chính xác trong các điều kiện này dao động từ 40% đến 60%, có thể dẫn đến các trường hợp cảnh báo sai hoặc từ chối khởi động xe dù người điều khiển đã đội mũ đúng cách. Đây là điểm cần được cải thiện thông qua việc huấn luyện thêm dữ liệu trong các môi trường ánh sáng khác nhau.

Hệ thống cảm biến lực mặc dù hoạt động ổn định trong môi trường thử nghiệm, nhưng trong thực tế, kết quả đo có thể bị ảnh hưởng bởi cách đội mũ hoặc bởi lực phân bố không đồng đều trên đầu người dùng. Điều này có thể gây ra các cảnh báo sai, làm ảnh hưởng đến trải nghiệm của người dùng.

Ngoài ra, trong quá trình vận hành tại các khu vực có sóng di động yếu, thời gian gửi cảnh báo khẩn cấp qua cuộc gọi hoặc tin nhắn có thể tăng lên trên 10 giây hoặc có thể không bắt sóng được, ảnh hưởng đến khả năng hỗ trợ kịp thời khi xảy ra sự cố. Việc tối ưu hóa cơ chế gửi cảnh báo khẩn cấp trong các môi trường mạng yếu cần được tiếp tục nghiên cứu.

Về phần mềm, mặc dù mô hình Random Forest sử dụng cho phân tích hành vi từ dữ liệu cảm biến MPU6050 cho độ chính xác cao trên tập dữ liệu hiện tại, tuy nhiên mô hình chưa được tinh chỉnh cho các phong cách lái xe đa dạng hoặc các tình huống hiếm gặp, điều này có thể làm giảm độ chính xác trong một số trường hợp đặc biệt. Cuối cùng, hệ thống hiện tại chưa hỗ trợ cơ chế cập nhật phần mềm từ xa (OTA). Việc nâng cấp phần mềm trên Raspberry Pi và ESP32 vẫn thực hiện thủ công, gây khó khăn khi cần triển khai các bản cập nhật hoặc sửa lỗi sau khi hệ thống đã được lắp đặt trên thiết bị thật.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Qua quá trình nghiên cứu và triển khai, hệ thống mũ bảo hiểm thông minh IoT hỗ trợ giám sát an toàn giao thông và cảnh báo nguy cơ tai nạn đã được xây dựng thành công và đáp ứng được các yêu cầu kỹ thuật đã đề ra. Hệ thống đã thực hiện tốt các chức năng chính như: nhận diện chính xác trạng thái đội mũ, phát hiện các hành vi lái xe nguy hiểm, cảnh báo khẩn cấp khi xảy ra tai nạn, đồng thời cho phép người dùng giám sát trạng thái an toàn và lịch sử hoạt động thông qua ứng dụng di động. Việc tích hợp thêm cơ chế gửi email khẩn cấp như một biện pháp dự phòng khi SIM mất sóng đã góp phần nâng cao độ tin cậy và khả năng phản ứng của hệ thống trong các tình huống bất lợi.

Về phần cứng, các thành phần cảm biến, vi xử lý và các module truyền thông được tích hợp gọn gàng vào mũ bảo hiểm, đảm bảo tính thẩm mỹ và phù hợp với thực tế sử dụng. Hệ thống phần mềm vận hành ổn định, các module giao tiếp (giữa Raspberry Pi và ESP32, giữa ứng dụng di động và hệ thống) có độ trễ thấp, đảm bảo phản ứng kịp thời với các tình huống nguy hiểm.

Đặc biệt, việc ứng dụng mô hình học sâu MobileNet SSD và các thuật toán học máy Random Forest đã góp phần nâng cao khả năng nhận diện, tăng độ tin cậy cho hệ thống trong điều kiện vận hành thực tế. Khả năng kết nối dữ liệu với dịch vụ Google Sheets và giao diện ứng dụng di động giúp việc giám sát và quản lý hành vi lái xe trở nên dễ dàng, trực quan hơn.

Tuy nhiên, hệ thống vẫn còn một số hạn chế cần khắc phục, như tốc độ xử lý còn hạn chế do phần cứng Raspberry Pi Zero 2W chưa đủ mạnh, khả năng nhận diện hình ảnh trong điều kiện ánh sáng yếu còn thấp, thời lượng pin hoạt động chưa dài, và chưa có cơ chế cập nhật phần mềm từ xa (sử dụng Firebase OTA). Việc cải thiện các vấn đề này là cơ sở quan trọng cho các hướng phát triển tiếp theo của đề tài. Nhìn

chung, kết quả đề tài đã chứng minh tính khả thi và hiệu quả của việc ứng dụng công nghệ IoT và AI trong lĩnh vực đảm bảo an toàn giao thông cá nhân.

5.2 Hướng phát triển trong tương lai

Trong giai đoạn tiếp theo, hệ thống mũ bảo hiểm thông minh IoT hoàn toàn có thể tiếp tục được nghiên cứu và phát triển theo nhiều hướng nhằm nâng cao hơn nữa hiệu quả sử dụng, độ chính xác, độ ổn định cũng như khả năng mở rộng tính năng.

Trước hết, việc nâng cấp phần cứng xử lý là một hướng đi quan trọng. Việc thay thế Raspberry Pi Zero 2W bằng các nền tảng vi xử lý mạnh hơn như Raspberry Pi 4 hoặc các dòng vi xử lý chuyên dụng cho học máy như Nvidia Jetson Nano sẽ giúp cải thiện đáng kể tốc độ xử lý hình ảnh, cho phép hệ thống vận hành mượt mà hơn ngay cả trong các điều kiện phức tạp như di chuyển với tốc độ cao hoặc môi trường ánh sáng yếu. Bên cạnh đó, việc tối ưu thuật toán nhận diện bằng cách mở rộng tập dữ liệu huấn luyện với các mẫu ảnh đa dạng hơn về điều kiện ánh sáng, góc chụp và hành vi người dùng sẽ giúp nâng cao độ chính xác, giảm thiểu các trường hợp cảnh báo sai.

Ngoài ra, giải pháp nâng cao hiệu quả tiêu thụ năng lượng, chẳng hạn như tối ưu chu kỳ hoạt động của cảm biến hoặc tích hợp nguồn cấp điện trực tiếp từ xe máy, sẽ giúp kéo dài thời gian vận hành liên tục của thiết bị, đáp ứng tốt hơn nhu cầu sử dụng thực tế.

Về mặt phần mềm, việc bổ sung cơ chế cập nhật phần mềm từ xa là rất cần thiết để hỗ trợ việc nâng cấp, vá lỗi hoặc mở rộng tính năng một cách dễ dàng và nhanh chóng sau khi hệ thống đã được triển khai. Đồng thời, phát triển thêm các chức năng phân tích hành vi lái xe, cảnh báo sớm nguy cơ tai nạn dựa trên dữ liệu lịch sử, kết hợp với các nền tảng giám sát giao thông thông minh – tương tự đề xuất trong nghiên cứu tích hợp hệ thống camera, truyền dữ liệu MQTT và nền tảng đám mây sẽ giúp gia tăng giá trị ứng dụng thực tiễn của hệ thống (S. A. Khan và cộng sự, 2024).

DANH MỤC TÀI LIỆU THAM KHẢO

A. Tài liệu tham khảo:

Tiếng Việt

Tiệp Vũ (2021). *Machine Learning cho dữ liệu dạng bảng*. Truy cập tháng 4 năm 2025, từ https://machinelearningcoban.com/tabml_book/intro.html.

Nguyễn Chiến Thắng (2022). *Train model SSD MobileNet với Tensorflow 2 trên Colab*. Truy cập tháng 4 năm 2025, từ https://miai.vn/2022/01/03/train-model-ssd-mobilenet-voi-tensorflow-2-tren-colab-mi-ai/.

Tiếng Anh

S. Sekar, L. Jaivenkatesh, S. Aravind Kumar, K. Dinesh Kumar, and N. Jeevanantham (2022), *"IoT based smart helmet," International Research Journal of Engineering and Technology (IRJET)*, vol. 9, no. 4, pp. 3277–3285.

S. Sobhana, S. R. Sowmeeya, M. Srinathji, and S. Tamilselvan (2021), *"Smart helmet," IOP Conference Series: Materials Science and Engineering*, vol. 1084, no. 1, p. 012116, doi: 10.1088/1757-899X/1084/1/012116.

S. A. Khan, T. Abbasi, Y. S. Zeb, M. Bilal, and S. Sardar (2024), *"Development of smart helmet using Internet of Things (IoT)," Journal of Xi'an Shiyou University, Natural Science Edition*, vol. 20, no. 7, pp. 220–226.

PHỤ LỤC

Phụ lục A: Danh sách linh kiện sử dụng

A.1 Danh sách linh kiện sử dụng

Bảng A.1 Danh sách linh kiện sử dụng trong mô hình

STT	Tên linh kiện	Số lượng	Mô tả chức năng
1	Raspberry Pi Zero 2W	1	Bộ xử lý trung tâm, thực thi mô hình AI, giao tiếp cảm biến và điều khiển thiết bị ngoại vi
2	ESP32 DevKit	1	Vi điều khiển phụ trách thực thi relay, cảnh báo, hiển thị LCD và xử lý tín hiệu từ cảm biến rung/nghiêng
3	Camera Pi (CSI interface)	1	Ghi nhận hình ảnh khuôn mặt, đầu vào cho mô hình nhận diện MobileNet SSD
4	Cảm biến lực FSR402/RFP602	4	Phát hiện người đội mũ đúng cách và ghi nhận lực va chạm từ các hướng
5	Cảm biến Hall A3144	1	Phát hiện tình trạng gài quai nón bảo hiểm (nam châm tiếp xúc)
6	MPU6050 (gia tốc + con quay)	1	Đo gia tốc và vận tốc góc, phục vụ nhận diện té ngã, rung lắc, nghiêng
7	SW420 + SW520	2	Cảm biến phát hiện rung động và nghiêng bổ sung ở thân xe hoặc mũ
8	GSM SIM900A Module	1	Gửi SMS và gọi điện thoại khẩn cấp khi xảy ra tai nạn
9	GPS NEO-6M	1	Định vị vị trí người điều khiển

10	LCD 16x2 (I2C)	1	Hiển thị trạng thái vận hành của hệ thống ESP32
11	Buzzer	1	Phát cảnh báo âm thanh khi không đảm bảo an toàn hoặc có va chạm
12	Power bank 5V–3A	1	Cấp nguồn cho toàn bộ hệ thống khi lắp thực tế vào mũ
13	ADC ADS1115	1	Chuyển đổi tín hiệu analog từ FSR sang digital cho Raspberry Pi

A.2 Thông số kỹ thuật các cảm biến chính

Bảng A.2 Thông số kỹ thuật chính của các linh kiện sử dụng trong hệ thống

STT	Tên linh kiện	Chức năng chính	Giao tiếp	Nguồn hoạt động	Thông số nổi bật
1	MPU6050	Đo gia tốc và góc quay	I2C	3.3V	±2g~±16g; ±250~2000°/s; tích hợp Gyro
2	FSR402 / RFP602	Cảm biến lực	Analog	0–5V	Điện trở biến đổi theo lực tác động
3	Hall A3144	Phát hiện từ trường (gài quai mũ)	Digital (GPIO)	3.3–5V	Ngõ ra LOW khi có từ trường
4	SW420 / SW520	Phát hiện rung/lắc	Digital (GPIO)	3.3–5V	Nhạy với rung động và dao động cơ học
5	SIM900A	Gửi SMS, gọi điện	UART	5V	Mạng GSM/GPRS, khe SIM thường
6	GPS NEO-6M	Xác định vị trí	UART	3.3–5V	Sai số ~2.5m; cập nhật 1–5Hz

7	LCD 16x2 I2C	Hiển thị trạng thái	I2C	5V	2 dòng, 16 ký tự; nền xanh, chữ đen
8	Buzzer	Cảnh báo âm thanh	GPIO (Digital)	3.3–5V	Phát âm thanh cảnh báo ngắn gọn
9	ADS1115 (ADC)	Chuyển tín hiệu analog sang digital	I2C	2.0–5.5V	4 kênh, độ phân giải 16-bit

Bảng A.3 Thông số kỹ thuật ESP32 DevKit V1

Thông số	Chi tiết
Vi xử lý	Dual-core Xtensa LX6 (32-bit), xung nhịp 160 MHz hoặc 240 MHz
RAM	512 KB SRAM
Bộ nhớ Flash	4 MB đến 16 MB
Wi-Fi	802.11 b/g/n (HT20) với hỗ trợ chế độ Access Point (AP), Station (STA) và Mesh
Bluetooth	Bluetooth v4.2 và BLE (Bluetooth Low Energy)
GPIO (Digital I/O Pins)	Lên đến 34 chân GPIO
ADC (Analog to Digital Converter)	12-bit, 18 kênh ADC
DAC (Digital to Analog Converter)	8-bit, 2 kênh DAC
UART	Lên đến 3 cổng UART
SPI	4 cổng SPI
I2C	2 cổng I2C
Tiêu thụ điện năng	Chế độ ngủ thấp nhất khoảng 10 µA, chế độ hoạt động khoảng 160 mA
Nguồn cung cấp	2.2V - 3.6V (thông thường là 3.3V)

Bảng A.4 Thông số kỹ thuật Raspberry Pi Zero 2W

Tính năng	Thông số chi tiết
Vi xử lý	Broadcom BCM2710A1
Kiến trúc CPU	ARM Cortex-A53 (64-bit)
Số nhân CPU	4 nhân
Xung nhịp CPU	1.0 GHz
RAM	512 MB LPDDR2
Kết nối Wi-Fi	802.11 b/g/n (2.4GHz)
Bluetooth	Bluetooth 4.2
Giao tiếp GPIO	40 chân GPIO (2.54mm)
Giao tiếp CSI (Camera)	1 cổng CSI
Giao tiếp I2C/SPI/UART	Có (qua header GPIO)
Hệ điều hành hỗ trợ	Raspberry Pi OS (Linux), hỗ trợ Python/C++
Nguồn cấp	5V qua chân GPIO hoặc USB
Kích thước	65mm x 30mm
Tiêu thụ điện năng (ước tính)	~0.7W – 2.5W tùy tải

Phụ lục B: Mã nguồn phát triển

B.1 Mã nguồn hệ thống 1

```
import warnings
warnings.filterwarnings("ignore", category=UserWarning)
import numpy as np
import pandas as pd
import tflite_runtime.interpreter as tflite
from PIL import Image, ImageEnhance
import os
import time
import board
import busio
import RPi.GPIO as GPIO
import requests
import socket
import smbus2
import joblib
from datetime import datetime
from threading import Thread
from flask import Flask, request
from adafruit_ads1x15.analog_in import AnalogIn
import adafruit_ads1x15.ads1115 as ADS
from google.oauth2 import service_account
from googleapiclient.discovery import build
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from datetime import datetime, timedelta
last_call_time = datetime.min  # thời gian lần gọi gần nhất
def send_emergency_email(subject, body):
    try:
        print("□ Gọi hàm gửi email: ", subject)
        msg = MIMEMultipart()
        msg['From'] = EMAIL_SENDER
        msg['To'] = EMAIL_RECEIVER
        msg['Subject'] = subject

        msg.attach(MIMEText(body, 'plain'))
```

```
        server = smtplib.SMTP_SSL('smtp.gmail.com', 465)
        server.login(EMAIL_SENDER, EMAIL_PASSWORD)
        server.sendmail(EMAIL_SENDER, EMAIL_RECEIVER, msg.as_string())
        server.quit()

        print("📧 Đã gửi email:", subject)
    except Exception as e:
        print(f"❌ Gửi email thất bại: {e}")

last_alarm_time = datetime.min  # 🆕 thời điểm báo động cuối
# === Google Sheets Config ===
SPREADSHEET_ID = '1rOX2ULAQ4fAMfDijPcDynpaQx-
rwwhKBXyNRi89T58o'
CREDENTIALS_FILE = 'credentials.json'
# ==== email ===
EMAIL_SENDER = "phanthanhdat25082003@gmail.com"
EMAIL_PASSWORD = "qwafdvecgpdnfelj"
EMAIL_RECEIVER = "phanthanhdat0825@gmail.com"
creds = service_account.Credentials.from_service_account_file(
    CREDENTIALS_FILE,
    scopes=['https://www.googleapis.com/auth/spreadsheets']
)
sheet_service = build('sheets', 'v4', credentials=creds)

# === GPIO Config ===
BUZZER_PIN = 27
HALL_PIN = 22
GPIO.setmode(GPIO.BCM)
GPIO.setup(HALL_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(BUZZER_PIN, GPIO.OUT)

# === ADC for force sensors ===
i2c = busio.I2C(board.SCL, board.SDA)
ads = ADS.ADS1115(i2c)
force_top = AnalogIn(ads, ADS.P0)
force_forehead = AnalogIn(ads, ADS.P2)
force_left = AnalogIn(ads, ADS.P1)   # A1 bên trái
force_right = AnalogIn(ads, ADS.P3)  # A3 bên phải
```

```
FORCE_THRESHOLD = 2000

# === Load label and TFLite model ===
with open("labelmap.txt", "r") as f:
    labels = f.read().splitlines()

interpreter = tflite.Interpreter(model_path="model.tflite")
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
input_height = input_details[0]['shape'][1]
input_width = input_details[0]['shape'][2]

# === Load sklearn model ===
bus = smbus2.SMBus(1)
MPU6050_ADDR = 0x68
model = joblib.load("mpu6050_rf_model.pkl")
ENABLE_FORCE = True
ENABLE_MPU = True
ENABLE_HALL = True

def init_mpu():
    bus.write_byte_data(MPU6050_ADDR, 0x6B, 0)
    time.sleep(0.1)

def read_raw_data(addr):
    high = bus.read_byte_data(MPU6050_ADDR, addr)
    low = bus.read_byte_data(MPU6050_ADDR, addr+1)
    value = ((high << 8) | low)
    if value > 32768:
        value = value - 65536
    return value

def read_mpu6050():
    ax = read_raw_data(0x3B) / 16384.0
    ay = read_raw_data(0x3D) / 16384.0
    az = read_raw_data(0x3F) / 16384.0
    gx = read_raw_data(0x43) / 131.0
    gy = read_raw_data(0x45) / 131.0
    gz = read_raw_data(0x47) / 131.0
```

```
    return ax, ay, az, gx, gy, gz

def predict_behavior(ax, ay, az, gx, gy, gz):
    input_data = pd.DataFrame([[ax, ay, az, gx, gy, gz]],
                              columns=["ax", "ay", "az", "gx", "gy", "gz"])
    return model.predict(input_data)[0]

def get_latest_ip():
    try:
        result = sheet_service.spreadsheets().values().get(
            spreadsheetId=SPREADSHEET_ID,
            range="Main!A2:M"
        ).execute()
        values = result.get('values', [])
        if values:
            return values[-1][1]
    except Exception as e:
        print("❌ Lỗi đọc IP từ Google Sheets:", e)
    return None

def get_pi_ip():
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        ip = s.getsockname()[0]
        s.close()
        return ip
    except:
        return "Unknown"


def append_status_to_sheet(detect1, detect2, detect3, conclusion,
                           accident_flag=0,
                           detection_source='',
                           esp_response='',
                           gps_location='',
                           note=''):
    try:
        current_time = datetime.now().strftime('%H:%M:%S')
        current_date = datetime.now().strftime('%d/%m/%Y %H:%M:%S')
```

```
        esp_ip = get_latest_ip() or ''
        pi_ip = get_pi_ip()

        body = {
            'values': [[
                current_date, esp_ip, pi_ip, current_time,
                detect1, detect2, detect3, conclusion,
                accident_flag, detection_source, esp_response,
                gps_location, note
            ]]
        }
        sheet_service.spreadsheets().values().append(
            spreadsheetId=SPREADSHEET_ID,
            range="Main!A2",
            valueInputOption="USER_ENTERED",
            body=body
        ).execute()
        print("✅ Đã ghi trạng thái + tai nạn lên Google Sheets.")
    except Exception as e:
        print("⚠ Lỗi ghi Google Sheets:", e)


def upload_pi_ip_only():
    try:
        current_date = datetime.now().strftime('%d/%m/%Y %H:%M:%S')
        pi_ip = get_pi_ip()

        body = {
            'values': [[
                current_date, pi_ip
            ]]
        }
        sheet_service.spreadsheets().values().append(
            spreadsheetId=SPREADSHEET_ID,
            range="RaspberryIP!A1",  # 🚀 Sheet mới tên RaspberryIP
            valueInputOption="USER_ENTERED",
            insertDataOption="INSERT_ROWS",  # luôn chèn dòng mới
            body=body
        ).execute()
```

```
        print(f"✅ Đã ghi Pi IP lên Sheet RaspberryIP: {pi_ip}")
    except Exception as e:
        print("⚠ Lỗi khi ghi Pi IP:", e)

def auto_upload_pi_ip():
    upload_pi_ip_only()  # 🚀 Ghi 1 lần duy nhất khi khởi động


def detect_image(image_path):
    img = Image.open(image_path).convert('RGB')
    img = img.rotate(180)
    img = img.resize((input_width, input_height))
    img = ImageEnhance.Sharpness(img).enhance(2)

    input_data = np.expand_dims(np.array(img, dtype=np.uint8), axis=0)
    interpreter.set_tensor(input_details[0]['index'], input_data)
    interpreter.invoke()

    classes = interpreter.get_tensor(output_details[1]['index'])[0]
    scores = interpreter.get_tensor(output_details[2]['index'])[0]

    for i in range(len(scores)):
        if scores[i] > 0.3:
            label = labels[int(classes[i])]
            print(f"🎯 Phát hiện: {label} ({scores[i]*100:.1f}%)")
            if label in ['person', 'mask', 'glasses']:
                return True
    return False

def play_buzzer():
    print("🔊 Buzzer được kích hoạt")
    try:
        for _ in range(3):
            GPIO.output(BUZZER_PIN, GPIO.HIGH)
            time.sleep(0.3)
            GPIO.output(BUZZER_PIN, GPIO.LOW)
            time.sleep(0.2)
    except Exception as e:
        print("❌ Lỗi khi phát buzzer:", e)
```

```
app = Flask(__name__)

# === Biến lưu trạng thái ===
current_status = {
    "step": "⏳ Khởi động hệ thống...",
    "progress": 0,
    "timestamp": datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
    "gx": 0.0,
    "ax": 0.0
}


def update_status(step_text, progress_value):
    global current_status
    current_status["step"] = step_text
    current_status["progress"] = progress_value
    current_status["timestamp"] = datetime.now().strftime('%Y-%m-%d
%H:%M:%S')
from flask import jsonify
@app.route('/status', methods=['GET'])
def get_status():
    return current_status



@app.route('/buzzer', methods=["GET"])
def remote_buzzer():
    Thread(target=play_buzzer).start()
    return "🔔 Buzzer từ HTTP đã chạy!"


@app.route('/api/buzzer', methods=['GET'])
def api_buzzer():
    state = request.args.get('state', 'off')
    print(f"🔔 API Buzzer yêu cầu: {state}")

    if state == 'on':
        Thread(target=play_buzzer).start()
```

```
        return "✅ Buzzer ON"
    else:
        GPIO.output(BUZZER_PIN, GPIO.LOW)
        return "✅ Buzzer OFF"

@app.route('/api/relay', methods=['GET'])
def api_relay():
    state = request.args.get('state', 'off')
    esp32_ip = get_latest_ip()
    if not esp32_ip:
        return "❌ Không có IP ESP32"

    try:
        url = f"http://{esp32_ip}/relay?state={state}"
        print(f"➡ Forward relay to ESP32: {url}")
        response = requests.get(url, timeout=3)
        return f"✅ ESP32 relay {state} → {response.text}"
    except Exception as e:
        return f"⚠ Lỗi gửi relay tới ESP32: {str(e)}"

@app.route('/api/sensor_control', methods=['GET'])
def api_sensor_control():
    global ENABLE_FORCE, ENABLE_MPU, ENABLE_HALL

    sensor = request.args.get('sensor', '')
    state = request.args.get('state', 'off')
    enabled = (state == 'on')

    print(f"🔄 Yêu cầu điều khiển sensor: {sensor}, trạng thái: {state}")

    if sensor == 'force':
        ENABLE_FORCE = enabled
        return f"✅ Force sensor set to {state}"
    elif sensor == 'mpu':
        ENABLE_MPU = enabled
        return f"✅ MPU6050 set to {state}"
    elif sensor == 'hall':
        ENABLE_HALL = enabled
```

```
        return f"✅ Hall sensor set to {state}"
    else:
        return f"❌ Không rõ sensor: {sensor}"

@app.route('/alert_sw', methods=['POST'])
def alert_sw():
    global current_status, last_call_time

    print("🚨 [ESP32] CẢM BIẾN SW420 + SW520 > 3s → BÁO VỀ PI")
    print(f"□ Trạng thái MPU hiện tại: {current_status['step']}")

    if 'fall' in current_status['step']:
        now = datetime.now()
        if (now - last_call_time) > timedelta(seconds=7):
            print("📞 PHÁT HIỆN FALL → GỌI NGƯỢC LẠI ESP32 (alert_mpu)!")
            print("📧 Gửi email cảnh báo ngay lập tức sau khi gửi alert_mpu...")
            send_emergency_email(
                subject="📞 Goi khan cap",
                body="Da kich hoat goi khan cap tai vi tri 10.730333,106.697683")
            esp32_ip = get_latest_ip()
            if esp32_ip:
                try:
                    url_alert = f"http://{esp32_ip}/alert_mpu"

                    response = requests.get(url_alert, timeout=5)
                    print(f"📩 ESP32 trả lời: {response.text.strip()}")
                except Exception as e:
                    print(f"⚠️ Lỗi gửi alert_mpu tới ESP32: {e}")
            last_call_time = now
            return "OK - Đã gọi alert_mpu cho ESP32!"
        else:
            print("□ ĐANG CHỜ 7 GIÂY... Không gọi lại.")
            return "OK - Đang chờ (không gọi lại)"
    else:
        print("ℹ️ Không phải fall → chưa gọi.")
        return "OK - Chưa gọi (chưa fall)."
@app.route('/api/set_phone', methods=['GET'])
def api_set_phone():
    esp32_ip = get_latest_ip()
```

```
    if not esp32_ip:
        return "❌ Không có IP ESP32"

    phone = request.args.get('phone', '')
    if not phone:
        return "❌ Thiếu tham số phone"

    try:
        url = f"http://{esp32_ip}/set_phone?phone={phone}"
        print(f"➡ Forward set_phone to ESP32: {url}")
        response = requests.get(url, timeout=3)
        return f"✅ ESP32 set_phone → {response.text}"
    except Exception as e:
        return f"⚠ Lỗi gửi set_phone tới ESP32: {str(e)}"

@app.route('/log_call_done', methods=['POST'])
def log_call_done():
    print("📞 Đã nhận log_call_done từ ESP32")
    # Có thể:
    # - ghi vào Google Sheet
    # - hoặc gọi app incrementCounter('call')
    # - hoặc set biến nhớ

    return "OK - Call logged"

@app.route('/api/alert_mpu')
def api_alert_mpu():
    esp32_ip = get_latest_ip()
    if not esp32_ip:
        return "❌ Không có IP ESP32"

    try:
        url_alert = f"http://{esp32_ip}/alert_mpu"


        print(f"➡ Gửi alert_mpu tới ESP32: {url_alert}")
        response = requests.get(url_alert, timeout=5)
        return f"✅ ESP32 alert_mpu → {response.text.strip()}"
```

```
    except Exception as e:
        return f"⚠ Lỗi gửi alert_mpu tới ESP32: {e}"

def continuous_mpu_monitor():
    global ENABLE_MPU
    init_mpu()
    while True:
        if not ENABLE_MPU:
            time.sleep(1)
            continue

        ax, ay, az, gx, gy, gz = read_mpu6050()
        result = predict_behavior(ax, ay, az, gx, gy, gz)
        print(f"[MPU6050] □ Dự đoán liên tục: {result}")
        current_status.update({
        "step": f"□ Dự đoán: {result}",
        "progress": 6,
        "timestamp": datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
        "gx": gx,
        "ax": ax,
          "ay": ay,
          "az": az
})
        update_status(f"□ Dự đoán: {result}", 6)
        time.sleep(3)
def monitor_force_left_right():
    global ENABLE_FORCE
    while True:
        if not ENABLE_FORCE:
            time.sleep(1)
            continue

        v_left = max(force_left.value - 300000, 0)
        v_right = force_right.value

        if v_left > FORCE_THRESHOLD or v_right > FORCE_THRESHOLD:
            print(f"⚠ Phat hien luc bat thuong")
            play_buzzer()
            send_emergency_email(
```

```
                subject="📧 Luc bat thuong",
                body="Luc Bat Thuong Tai Toa Do 10.730333,106.697683")
        # 🆕 Gửi yêu cầu gửi tin nhắn qua ESP32
        esp32_ip = get_latest_ip()
        if esp32_ip:
            try:
                msg = "Luc Bat Thuong Tai Toa Do 10.730333,106.697683"
                url_sms = f"http://{esp32_ip}/send_sms?msg={msg}"
                response = requests.get(url_sms, timeout=5)

                print(f"📤 Send SMS: {response.text.strip()}")
            except Exception as e:
                print(f"❌ ERROR SEND SMS to ESP32: {e}")

        append_status_to_sheet(
            1, 1, 1, 1,
            accident_flag=0,
            detection_source='force_sensor',
            gps_location="10.730333, 106.697683",
            note="Lực bất thường trái/phải"
        )
        time.sleep(5)
    time.sleep(0.5)



def continuous_hall_monitor():
    global ENABLE_HALL
    print("✅ QUY TRÌNH HOÀN TẤT. BẮT ĐẦU GIÁM SÁT LIÊN TỤC...")
    last_state = GPIO.LOW
    while True:
        if not ENABLE_HALL:
            time.sleep(1)
            continue

        current_state = GPIO.input(HALL_PIN)
        if current_state == GPIO.LOW and last_state == GPIO.HIGH:
            print("📍 Phát hiện nam châm mới → Kích hoạt buzzer.")
            play_buzzer()
```

```
        last_state = current_state
        time.sleep(0.5)

def main_process():
    try:
        print("🚀 Khởi động hệ thống kiểm tra mũ bảo hiểm...")
        detect1 = detect2 = detect3 = 0

        print("⏳ Đang chờ đủ lực...")
        update_status("⏳ Đang chờ đủ lực...", 1)
        while True:
            if force_top.value > FORCE_THRESHOLD and force_forehead.value >
FORCE_THRESHOLD:
                print("✅ Lực đạt yêu cầu.")
                detect1 = 1
                break
            time.sleep(1)

        print("📸 Chụp ảnh để nhận diện...")
        update_status("📸 Đang nhận diện người đội mũ...", 2)
        while True:
            os.system("libcamera-still -o image.jpg --width 320 --height 240 -n -t 100")
            if detect_image("image.jpg"):
                print("✅ Đã nhận diện có người đội mũ.")
                detect2 = 1
                break
            else:
                print("⛔ Không phát hiện người đội mũ. Chụp lại...")
                time.sleep(2)

        print("⏳ Đang chờ gài quai nón...")
        update_status("⏳ Đang chờ gài quai nón...", 3)
        while True:
            if GPIO.input(HALL_PIN) == GPIO.HIGH:
                print("✅ Đã gài quai nón.")
                update_status("✅ Đã gài quai nón, khởi động xe...", 4)
                play_buzzer()
                detect3 = 1
                break
```

```
    print("● Chưa gài quai...")
    time.sleep(1)

esp32_ip = get_latest_ip()
if esp32_ip:
    try:
        url = f"http://{esp32_ip}/relay?state=on"
        response = requests.get(url, timeout=3)
        print("✅ ESP32 phản hồi:", response.text)
    except Exception as e:
        print("⚠ Lỗi gửi đến ESP32:", e)

init_mpu()
ax, ay, az, gx, gy, gz = read_mpu6050()
behavior_result = predict_behavior(ax, ay, az, gx, gy, gz)
print(f"□ Hành vi được dự đoán: {behavior_result}")
update_status(f"□ Hành vi: {behavior_result}", 5)

abnormal_detected = behavior_result in ["fall", "crash", "abnormal"]
if abnormal_detected and esp32_ip:

    url_alert = f"http://{esp32_ip}/alert_mpu"
    try:
        response_alert = requests.get(url_alert, timeout=5)
        result = response_alert.text.strip()
        print(f"📨 ESP32 trả lời: {result}")
    except Exception as e:
        print("⚠ ESP32 không phản hồi:", e)
        result = "NO_RESPONSE"
    print("🚨 Xác nhận tai nạn → xử lý khẩn cấp...")
    play_buzzer()
    append_status_to_sheet(
            detect1, detect2, detect3, 1,
            accident_flag=1,
            detection_source="MPU6050",
            esp_response=result,
            gps_location="10.730592740620942, 106.69800229033699",
            note="Tăng tốc đột ngột và rung/nghiêng"
            )
```

```
        conclusion = 1 if detect1 and detect2 and detect3 else 0
        append_status_to_sheet(detect1, detect2, detect3, conclusion)

        Thread(target=continuous_hall_monitor, daemon=True).start()
        Thread(target=continuous_mpu_monitor, daemon=True).start()
        Thread(target=monitor_force_left_right, daemon=True).start()
        update_status("□ Đang giám sát liên tục với MPU6050...", 6)

    except KeyboardInterrupt:
        print("\n⛔ Dừng bằng tay.")
    except KeyboardInterrupt:
        print("\n⛔ Dừng bằng tay.")
        GPIO.cleanup()
        print("□ Đã giải phóng GPIO.")

if __name__ == "__main__":
    Thread(target=auto_upload_pi_ip, daemon=True).start()  # 🚀 Ghi IP lên
RaspberryIP
    Thread(target=main_process).start()
    app.run(host="0.0.0.0", port=5000)
```

B.2 Mã nguồn hệ thống 2

```
 #include <WiFi.h>
#include <HTTPClient.h>
#include <WebServer.h>
#include <HardwareSerial.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

TaskHandle_t CallTaskHandle = NULL;
portMUX_TYPE uartMux = portMUX_INITIALIZER_UNLOCKED;

const char* ssid = "DACN";
const char* password = "12345678";
```

```
const char* pi_ip_url =
"https://script.google.com/macros/s/AKfycbxazIeUl2Q9zXUBUVG-
kuc6q9RA7jbuqJPFwdN6oaU8IDeOWHtLo74hR4Kkd9XpcNDT/exec";
const char* scriptURL = "https://script.google.com/macros/s/AKfycbzyYU-
wogDP8d2snAJm4yRgYTDyovdMoSeKtGnVtBiEgNwbn8JAIRiAcpXDDhpD2m
2K/exec";

#define RELAY_PIN 15
#define SW420_PIN 25
#define SW520_PIN 26
#define SIM_TX 17
#define SIM_RX 16

LiquidCrystal_I2C lcd(0x27, 16, 2);  // LCD I2C 16x2

HardwareSerial SIM900(1);
WebServer server(80);
bool isCalling = false;
unsigned long activateStart = 0;
bool counting = false;
String pi_ip = "";
String phoneNumber = "0901284053";

void showSIMResponse() {
  portENTER_CRITICAL(&uartMux);
  while (SIM900.available()) {
    char c = SIM900.read();
    Serial.write(c);
  }
  portEXIT_CRITICAL(&uartMux);
}

bool checkNetworkReady() {
  for (int i = 0; i < 5; i++) {
    portENTER_CRITICAL(&uartMux);
    SIM900.println("AT+CREG?");
    portEXIT_CRITICAL(&uartMux);
    delay(500);
    String response = "";
```

```
    portENTER_CRITICAL(&uartMux);
    while (SIM900.available()) {
      char c = SIM900.read();
      response += c;
      Serial.write(c);
    }
    portEXIT_CRITICAL(&uartMux);

    if (response.indexOf("+CREG: 0,1") >= 0) {
      Serial.println("✅ Mạng OK (CREG: 0,1)");
      return true;
    } else {
      Serial.println("⚠ Mạng chưa sẵn sàng, thử lại...");
      delay(2000);
    }
  }
  Serial.println("❌ Mạng không sẵn sàng sau 5 lần thử!");
  return false;
}

void notifyPiCallDone();

void callPhoneNumber() {
  if (isCalling) {
    Serial.println("⊘ Đang trong cuộc gọi, bỏ qua callPhoneNumber!");
    return;
  }

  isCalling = true;
  Serial.println("📞 Bắt đầu gọi SIM900A...");

  server.stop();
  delay(100);

  portENTER_CRITICAL(&uartMux);
  SIM900.println("ATZ");
  portEXIT_CRITICAL(&uartMux);
  delay(500);
  showSIMResponse();
```

```
portENTER_CRITICAL(&uartMux);
SIM900.println("ATE0");
portEXIT_CRITICAL(&uartMux);
delay(500);
showSIMResponse();

portENTER_CRITICAL(&uartMux);
SIM900.println("AT+CSQ");
portEXIT_CRITICAL(&uartMux);
delay(500);
showSIMResponse();

portENTER_CRITICAL(&uartMux);
SIM900.println("AT+CPIN?");
portEXIT_CRITICAL(&uartMux);
delay(500);
showSIMResponse();

portENTER_CRITICAL(&uartMux);
SIM900.println("AT+CREG?");
portEXIT_CRITICAL(&uartMux);
delay(500);
showSIMResponse();

portENTER_CRITICAL(&uartMux);
SIM900.println("AT+COPS?");
portEXIT_CRITICAL(&uartMux);
delay(1000);
showSIMResponse();

portENTER_CRITICAL(&uartMux);
SIM900.println("AT+COLP=1");
portEXIT_CRITICAL(&uartMux);
delay(500);
showSIMResponse();

portENTER_CRITICAL(&uartMux);
SIM900.println("AT+CSCS=\"GSM\"");
portEXIT_CRITICAL(&uartMux);
```

```
  delay(500);
  showSIMResponse();

  Serial.println("□ Chờ mạng ổn định...");

  if (!checkNetworkReady()) {
    Serial.println("❌ Hủy cuộc gọi vì không có mạng!");
    return;
  }

  String cmd = "ATD" + phoneNumber + ";";
  delay(1000);
  portENTER_CRITICAL(&uartMux);
  SIM900.println(cmd);
  portEXIT_CRITICAL(&uartMux);

  Serial.printf("📞 Đang gọi %s ...\n", phoneNumber.c_str());

  for (int i = 0; i < 60; i++) {
    showSIMResponse();
    delay(500);
  }

  portENTER_CRITICAL(&uartMux);
  SIM900.println("ATH");
  portEXIT_CRITICAL(&uartMux);
  Serial.println("📞 Đã cúp máy.");
  delay(1000);
  showSIMResponse();
  notifyPiCallDone();

  isCalling = false;
  server.begin();
  CallTaskHandle = NULL;
  Serial.println("🌐 WebServer đã khởi động lại.");
}

void CallTask(void *parameter) {
  Serial.println("📡 Tắt WiFi để gọi SIM900A...");
```

```
  WiFi.disconnect(true);
  WiFi.mode(WIFI_OFF);
  delay(500);

  callPhoneNumber();

  Serial.println("📡 Bật lại WiFi sau khi gọi...");
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\n✅ WiFi đã kết nối lại.");
  server.begin();

  CallTaskHandle = NULL;
  vTaskDelete(NULL);
}

void sendIPtoGoogleSheet(const String& ip) {
  HTTPClient http;
  String url = String(scriptURL) + "?ip=" + ip;
  http.begin(url);
  http.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);
  int httpCode = http.GET();
  if (httpCode > 0) {
    Serial.printf("HTTP Response code: %d\n", httpCode);
    Serial.println(http.getString());
  } else {
    Serial.printf("Lỗi gửi IP: %s\n", http.errorToString(httpCode).c_str());
  }
  http.end();
}

void notifyPiCallDone() {
  if (pi_ip == "") return;
```

```
  HTTPClient http;
  String url = "http://" + pi_ip + ":5000/log_call_done";
  http.begin(url);
  int httpCode = http.POST("");
  if (httpCode > 0) {
    Serial.printf("📞 Đã báo log_call_done về Pi (%d)\n", httpCode);
  } else {
    Serial.printf("❌ Lỗi gửi log_call_done: %s\n",
http.errorToString(httpCode).c_str());
  }
  http.end();
}

void handleRelay() {
  String state = server.arg("state");
  if (state == "on") {
    digitalWrite(RELAY_PIN, HIGH);
    server.send(200, "text/plain", "Relay ON");
  } else if (state == "off") {
    digitalWrite(RELAY_PIN, LOW);
    server.send(200, "text/plain", "Relay OFF");
  } else {
    server.send(400, "text/plain", "Tham số không hợp lệ. Dùng ?state=on hoặc off");
  }
}

void sendAlertToPi() {
  if (pi_ip == "") {
    Serial.println("❌ Không có IP Pi!");
    return;
  }
  HTTPClient http;
  String url = "http://" + pi_ip + ":5000/alert_sw";
  http.begin(url);
  int httpCode = http.POST("");
  if (httpCode > 0) {
    Serial.printf("📡 Gửi cảnh báo về Pi %s, Response: %d\n", pi_ip.c_str(),
httpCode);
    Serial.println(http.getString());
```

```
  } else {
    Serial.printf("❌ Lỗi gửi cảnh báo: %s\n", http.errorToString(httpCode).c_str());
  }
  http.end();
}

void checkSensors() {
  int sw420 = digitalRead(SW420_PIN);
  int sw520 = digitalRead(SW520_PIN);

  if (sw420 == LOW && sw520 == LOW) {
    if (!counting) {
      counting = true;
      activateStart = millis();
      Serial.println("□ Cả 2 cảm biến ON → Bắt đầu đếm 3 giây...");
    } else {
      if (millis() - activateStart >= 3000) {
        Serial.println("🚨 Cảm biến SW420 + SW520 > 3s → Gửi cảnh báo về Pi!");
        sendAlertToPi();
        counting = false;
        delay(7000);
      }
    }
  } else {
    counting = false;
  }
}

void getPiIP() {
  HTTPClient http;
  http.begin(pi_ip_url);
  http.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);
  int httpCode = http.GET();
  if (httpCode > 0) {
    String ip = http.getString();
    ip.trim();
    Serial.printf("🌐 Pi IP mới nhất: %s\n", ip.c_str());
    pi_ip = ip;
  } else {
```

```
    Serial.printf("❌ Lỗi lấy IP Pi: %s\n", http.errorToString(httpCode).c_str());
  }
  http.end();
}

void handleAlertMPU() {
  Serial.println("🚨 Nhận yêu cầu ALERT_MPU từ Pi → tạo Task gọi SIM900A!");
  if (isCalling) {
    Serial.println("⊘ Đang trong cuộc gọi → từ chối ALERT_MPU");
    server.send(200, "text/plain", "BUSY - Đang trong cuộc gọi SIM900A");
    return;
  }

  if (CallTaskHandle == NULL) {
    xTaskCreatePinnedToCore(CallTask, "CallTask", 4096, NULL, 1,
&CallTaskHandle, 0);
    server.send(200, "text/plain", "CONFIRM_ACCIDENT");
  } else {
    server.send(200, "text/plain", "BUSY - Đang gọi SIM900A");
  }
}

void handleSetPhone() {
  String newPhone = server.arg("phone");
  if (newPhone.length() > 5) {
    phoneNumber = newPhone;
    Serial.printf("✅ Đã cập nhật số điện thoại mới: %s\n", phoneNumber.c_str());
    server.send(200, "text/plain", "Phone updated");
  } else {
    server.send(400, "text/plain", "Invalid phone number");
  }
}
void handleSendSMS() {
  String msg = server.arg("msg");
  if (msg.length() == 0) {
    server.send(400, "text/plain", "Thiếu tham số msg");
    return;
  }
```

```
  if (!checkNetworkReady()) {
    server.send(500, "text/plain", "SIM900A không có mạng!");
    return;
  }

  String cmd = "AT+CMGF=1";  // Text mode
  SIM900.println(cmd);
  delay(500);
  showSIMResponse();

  cmd = "AT+CMGS=\"" + phoneNumber + "\"";
  SIM900.println(cmd);
  delay(500);
  showSIMResponse();

  SIM900.print(msg);
  SIM900.write(26); // Ctrl+Z để gửi
  delay(1000);
  showSIMResponse();

  server.send(200, "text/plain", "SEND SMS");
}

void setup() {
  Serial.begin(9600);
  SIM900.begin(9600, SERIAL_8N1, SIM_RX, SIM_TX);

  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, LOW);
  pinMode(SW420_PIN, INPUT_PULLUP);
  pinMode(SW520_PIN, INPUT_PULLUP);

  Wire.begin(21, 22);
  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("WiFi Connecting...");

  WiFi.begin(ssid, password);
```

```
  Serial.print("Đang kết nối WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\n✅ Đã kết nối WiFi");
  String ip = WiFi.localIP().toString();
  Serial.print("🌐 IP nội bộ ESP32: ");
  Serial.println(ip);

  sendIPtoGoogleSheet(ip);
  getPiIP();

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("IP:");
  lcd.print(ip);
  lcd.setCursor(0, 1);
  lcd.print("Ready to Start");
  server.on("/send_sms", handleSendSMS);
  server.on("/relay", handleRelay);
  server.on("/alert_mpu", handleAlertMPU);
  server.on("/set_phone", handleSetPhone);
  server.begin();
  Serial.println("🌐 WebServer đã sẵn sàng (port 80)");
}

unsigned long lastGetPiIP = 0;
const unsigned long getPiInterval = 2 * 60 * 5000;

void loop() {
  server.handleClient();
  checkSensors();

  if (millis() - lastGetPiIP > getPiInterval) {
    getPiIP();
    lastGetPiIP = millis();
```